



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Herramienta de gestión de
calendario EPS
Documentación Técnica**



Presentado por Rebeca Cuesta Estépar
en Universidad de Burgos — 9 de junio
de 2026

Tutores: Álvar Arnaiz González, Ana Serrano
Mamolar

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	33
Apéndice B Especificación de Requisitos	43
B.1. Introducción	43
B.2. Objetivos generales	43
B.3. Catálogo de requisitos	44
B.4. Especificación de casos de uso	51
Apéndice C Especificación de diseño	73
C.1. Introducción	73
C.2. Diseño de datos	73
C.3. Diseño arquitectónico	88
C.4. Diseño procedimental	93
Apéndice D Documentación técnica de programación	101
D.1. Introducción	101
D.2. Estructura de directorios	102
D.3. Manual del programador	109

D.4. Compilación, instalación y ejecución del proyecto	114
D.5. Pruebas del sistema	119
Apéndice E Documentación de usuario	131
E.1. Introducción	131
E.2. Requisitos de usuarios	131
E.3. Instalación	133
E.4. Manual del usuario	133
Apéndice F Anexo de sostenibilización curricular	151
F.1. Introducción	151
F.2. ODS 4: Educación de calidad	151
F.3. ODS 9: Industria, innovación e infraestructura	152
F.4. ODS 12: Producción y consumo responsables	152
F.5. ODS 13: Acción por el clima	153
F.6. Conclusión	153
Bibliografía	155

Índice de figuras

A.1. Gráfico de los commits realizados durante el Sprint 1	4
A.2. Gráfico de los commits realizados durante el Sprint 2	5
A.3. Gráfico de los commits realizados durante el Sprint 3	6
A.4. Gráfico de los commits realizados durante el Sprint 4	7
A.5. Gráfico de los commits realizados durante el Sprint 5	8
A.6. Gráfico de los commits realizados durante el Sprint 6	9
A.7. Gráfico de los commits realizados durante el Sprint 7	10
A.8. Gráfico de los commits realizados durante el Sprint 8	11
A.9. Gráfico de los commits realizados durante el Sprint 9	12
A.10. Gráfico de los commits realizados durante el Sprint 10	14
A.11. Gráfico de los commits realizados durante el Sprint 11	15
A.12. Gráfico de los commits realizados durante el Sprint 12	16
A.13. Gráfico de los commits realizados durante el Sprint 13	17
A.14. Gráfico de los commits realizados durante el Sprint 14	19
A.15. Gráfico de los commits realizados durante el Sprint 15	20
A.16. Gráfico de los commits realizados durante el Sprint 16	21
A.17. Gráfico de los commits realizados durante el Sprint 17	22
A.18. Gráfico de los commits realizados durante el Sprint 18	23
A.19. Gráfico de los commits realizados durante el Sprint 19	25
A.20. Gráfico de los commits realizados durante el Sprint 20	26
A.21. Gráfico de los commits realizados durante el Sprint 21	28
A.22. Gráfico de los commits realizados durante el Sprint 22	29
A.23. Gráfico de los commits realizados durante el Sprint 23	30
A.24. Gráfico de los commits realizados durante el Sprint 24	32
 B.1. Diagrama general de casos de uso	 51
C.1. Diagrama entidad-relación	75

C.2. Modelo relacional de la base de datos	76
C.3. Arquitectura general de la aplicación	89
C.4. Diagrama de despliegue de la aplicación	93
C.5. Diagrama de secuencia correspondiente al inicio de sesión	94
C.6. Diagrama de secuencia correspondiente a la carga de horarios desde hoja de cálculo	95
C.7. Diagrama de secuencia correspondiente a la creación de una reserva puntual	96
C.8. Diagrama de secuencia correspondiente a la modificación de una reserva periódica	98
C.9. Diagrama de secuencia correspondiente a ver la ocupación de una o varias aulas	99
C.10. Diagrama de secuencia correspondiente a la modificación del tipo de día en el calendario académico	100
D.1. Estructura de directorios del backend	103
D.2. Estructura de directorios del frontend	106
E.1. Pantalla de inicio de sesión	134
E.2. Estructura general de la interfaz de usuario	135
E.3. Pantalla de gestión de reservas puntuales	136
E.4. Formulario de creación o edición de reserva puntual	138
E.5. Desplegable para generar reservas puntuales de forma periódica	139
E.6. Pantalla de ocupación de aulas	140
E.7. Pantalla de ocupación de aulas	140
E.8. Pantalla de listas con cursos académicos cargados	141
E.9. Pantalla para crear un nuevo curso	142
E.10. Pantalla del calendario académico	142
E.11. Modal para cambiar el tipo de día	143
E.12. Pantalla de carga de nuevo horario	144
E.13. Pantalla de visualización de los horarios cargados en la aplicación	145
E.14. Pantalla de visualización de horarios	145
E.15. Formulario de creación o edición de reserva periódica	146
E.16. Modal de aviso con las restricciones incumplidas por el cambio	147
E.17. Acceso a entidades en el panel de administración	148
E.18. Página con el listado de entradas para la entidad Aula	149
E.19. Formulario para la creación o edición de una asignatura	149
E.20. Ventana para el mantenimiento y la gestión global de todo el sistema	150

Índice de tablas

A.1. Costes de personal del proyecto	35
A.2. Costes de hardware del proyecto	36
A.3. Costes de software del proyecto	37
A.4. Costes de infraestructura y despliegue	38
A.5. Resumen de costes del proyecto	38
A.6. Resumen de licencias de las herramientas utilizadas	39
B.1. CU-01 Inicio de sesión	52
B.2. CU-02 Gestionar usuarios	53
B.3. CU-03 Gestionar roles	54
B.4. CU-04 Gestión de entidades	55
B.5. CU-05 Cargar horario	56
B.6. CU-06 Consultar reservas puntuales	57
B.7. CU-07 Consultar ocupación de aula	58
B.8. CU-08 Crear reserva puntual	59
B.9. CU-09 Revisar solicitudes de reserva	60
B.10. CU-10 Modificar campo solicitud	61
B.11. CU-11 Aprobar rechazar reserva puntual	62
B.12. CU-12 Gestión de reservas puntuales	63
B.13. CU-13 Consultar horario de grado	64
B.14. CU-14 Gestión de reservas periódicas	65
B.15. CU-15 Crear calendario académico	66
B.16. CU-16 Consultar calendario académico	67
B.17. CU-17 Modificar día	68
B.18. CU-18 Solicitar reserva	69
B.19. CU-19 Consultar reservas propias	70
B.20. CU-20 Editar reservas propias	71
B.21. CU-22 Gestión solicitud de reservas	72

C.1. Diccionario de datos de la tabla AULA	78
C.2. Diccionario de datos de la tabla DOCENTE	78
C.3. Diccionario de datos de la tabla ASIGNATURAS	79
C.4. Diccionario de datos de la tabla IMPARTE	80
C.5. Diccionario de datos de la tabla GRADO	80
C.6. Diccionario de datos de la tabla GRUPO	81
C.7. Diccionario de datos de la tabla RESPONSABLE	81
C.8. Diccionario de datos de la tabla RESERVA	82
C.9. Diccionario de datos de la tabla RESERVA_PUNTUAL	83
C.10. Diccionario de datos de la tabla RESERVA_PERIODICA	84
C.11. Diccionario de datos de la tabla CURSO	84
C.12. Diccionario de datos de la tabla SEMESTRE	85
C.13. Diccionario de datos de la tabla DIA	85
C.14. Diccionario de datos de la tabla LECTIVO	86
C.15. Diccionario de datos de la tabla CAMBIO_DOCENCIA	86
C.16. Diccionario de datos de la tabla EXAMEN	87
C.17. Diccionario de datos de la tabla FESTIVO	87
C.18. Diccionario de datos de la tabla TFG	87
D.1. Descripción de la estructura principal del backend	105
D.2. Descripción de la estructura principal del frontend	109
D.3. Caso de prueba CP-01: validación de restricciones con conflicto	121
D.4. Caso de prueba CP-02: validación correcta de restricciones	122
D.5. Caso de prueba CP-03: movimiento correcto de una serie de reservas	123
D.6. Caso de prueba CP-04: generación de reservas desde hoja de cálculo	124
D.7. Caso de prueba CP-05: carga de Excel con aula inexistente	125
D.8. Caso de prueba CP-06: detección de aula ocupada	126
D.9. Caso de prueba CP-07: validación de aula disponible	127
D.10. Caso de prueba CP-08: clasificación de celdas mediante expresiones regulares	128
D.11. Caso de prueba CP-09: normalización de nombres de aulas	129

Apéndice A

Plan de Proyecto Software

A.1. Introducción

En este apéndice se presenta el Plan de Proyecto Software cuya finalidad es recoger los aspectos relacionados con la organización, la planificación y la viabilidad del proyecto, dando una visión global de cómo se ha estructurado su desarrollo y de los recursos necesarios para poder haberlo llevado a cabo.

En primer lugar, se presenta la planificación temporal del proyecto, en la que se describe la distribución del proyecto durante su duración mediante una organización por *sprints*, siguiendo la metodología ágil *Scrum*. Esta planificación va a permitir identificar las distintas fases de desarrollo que tiene el proyecto, los objetivos que se han planteado durante cada etapa y la evolución del trabajo de forma ordenada.

En segundo lugar, se muestra el estudio de viabilidad, cuyo objetivo es analizar si la realización del proyecto resulta asumible desde diferentes perspectivas. Para ello, se aborda por un lado la viabilidad económica, en la que se estiman los costes relacionado al desarrollo, y, por otro, la viabilidad legal, donde se consideran aspectos normativos y legales que pueden afectar al proyecto, como el uso de herramientas o licencias, o la protección de datos.

Todo este apéndice trata de justificar tanto que el proyecto ha sido planificado de manera adecuada, como que también resulta viable desde el punto de vista de la organización, económico y global.

A.2. Planificación temporal

Para organizar el desarrollo del proyecto a lo largo de este se ha optado por utilizar una metodología ágil, más concretamente *Scrum*. Se ha elegido esta metodología debido a que permite estructurar el trabajo de una forma iterativa e incremental, de forma que facilita la adaptación ante los posibles cambios que puedan sufrir los requisitos, la detección temprana de errores y el seguimiento de manera continua del proyecto.

Por otro lado, tendríamos la posibilidad de haber elegido una metodología tradicional, donde todas las fases quedan definidas desde el inicio de manera rígida. Mientras que *Scrum* divide el desarrollo en ciclos cortos de tiempo denominados *sprints*. Donde cada sprint tiene una duración determinada y en el que se comienza estableciendo los objetivos y tareas concretas que deben completarse en ese periodo de tiempo. Lo que permite avanzar de forma progresiva, revisando al final de cada iteración los resultados obtenidos durante el *sprint* y reajustando la planificación en el caso de que fuera necesario.

El uso de *Scrum* en este proyecto resultaba el más adecuado ya que permite descomponer el proyecto en partes más pequeñas y manejables, lo que favorece una mejor organización del tiempo y un control más preciso del avance. Además, facilita priorizar las funcionalidades más importantes y centrarse en entregar valor en cada etapa.

Planificación por Sprints

La planificación temporal del proyecto se ha estructurado en varios sprints, es decir, periodos de trabajo de una duración limitada en los que se desarrollan un conjunto de objetivos o tareas específicas.

En cada uno de estos sprints se especifica la fecha de inicio y de fin, la duración, luego se establecen los objetivos o tareas que se deben tratar de conseguir, la revisión del sprint que va a determinar cómo ha ido y si han faltado cosas o qué problemas ha habido, y un gráfico con los commits que se han realizado al repositorio durante este periodo de tiempo. En alguna ocasión concreta se encuentra también un apartado de observaciones.

Este enfoque como se ha mencionado, permite dividir el proyecto en iteraciones sucesivas, de forma que al finalizar cada *sprint* se obtiene un resultado parcial que contribuye al producto final. Y permite evaluar el estado del proyecto de forma periódica, identificando posibles problemas y

desviaciones y reorganizando las tareas en función de las necesidades que se van detectando durante el desarrollo.

Sprint 1 (08/09/2025-29/09/2025)

Duración: 3 semanas

■ **Objetivos**

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Crear repositorio.
2. Comenzar a realizar la base de datos.
3. Descargar los programas necesarios para la documentación del proyecto en L^AT_EX.
4. Investigar y revisar apuntes sobre la metodología *Scrum*.
5. Investigar y revisar apuntes relacionado a los casos de uso.

■ **Observaciones**

El comienzo de la base de datos se realizó desde un boceto de diagrama entidad-relación realizado durante la reunión.

■ **Revisión del Sprint**

Se lograron realizar todos los objetivos propuestos en la reunión de planificación, aunque queda profundizar más sobre la recopilación de información respecto al funcionamiento de los casos de uso.

■ **Gráfico**

El gráfico *burndown chart* se puede ver en la figura [A.1](#)

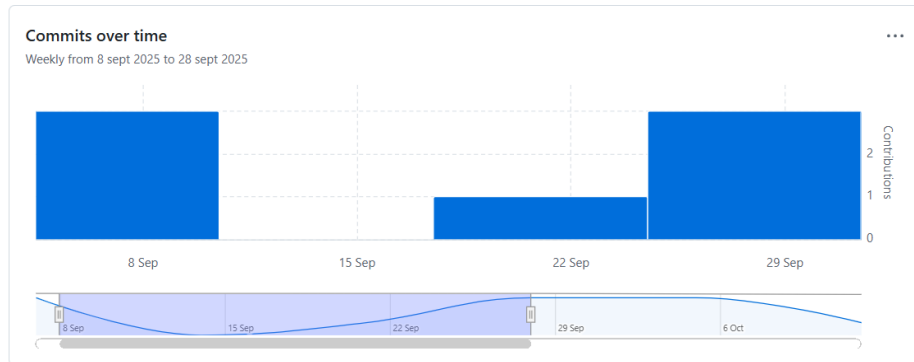


Figura A.1: Gráfico de los commits realizados durante el Sprint 1

Sprint 2 (30/09/2025-06/10/2025)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Realizar el *.gitignore*
2. Subir la plantilla de $\text{\LaTeX}$ al repositorio
3. Diseñar el diagrama entidad-relación
4. Realizar el modelo relacional
5. Comenzar con el listado de casos de uso

■ Revisión del Sprint

En este sprint, se han conseguido todos los objetivos propuesto al comienzo del mismo durante la reunión de planificación, aunque hay que realizar mejoras con el listado de casos de uso de la aplicación.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura [A.2](#)

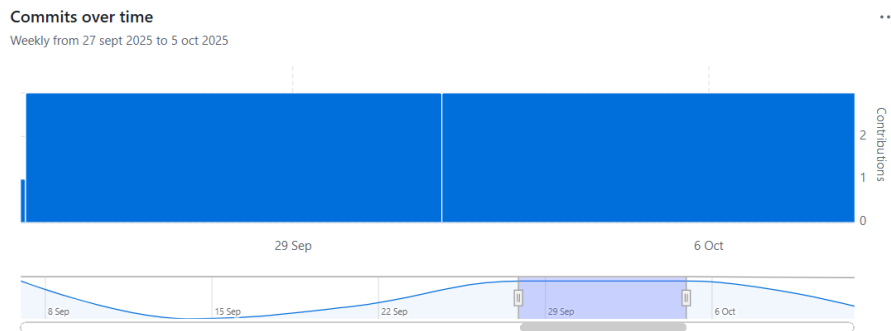


Figura A.2: Gráfico de los commits realizados durante el Sprint 2

Sprint 3 (07/10/2025-13/10/2025)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Definir lenguajes de programación y framework para el proyecto
2. Definir arquitectura
3. Documentar los tres primeros sprints
4. Definir casos de uso a alto nivel
5. Definir y extender un caso de uso

■ Revisión del Sprint

Tras la reunión de revisión del sprint se ha determinado que se han superado todos los objetivos que se propusieron en la planificación. El lenguaje propuesto para el backend ha sido Python, y el framework Django. Esta propuesta ha sido aceptada por los tutores.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura [A.3](#)

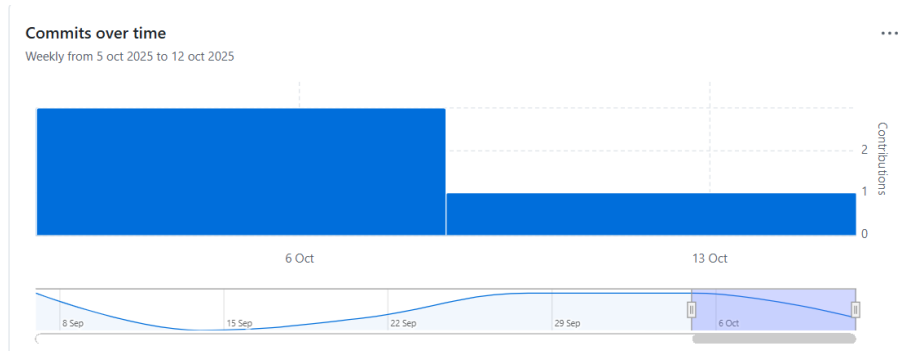


Figura A.3: Gráfico de los commits realizados durante el Sprint 3

Sprint 4 (14/10/2025-28/10/2025)

Duración: 2 semanas

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Documentar lista de requisitos
2. Comenzar a documentar casos de uso
3. Investigar funcionamiento framework Django

■ Revisión del Sprint

En la revisión del sprint se ha analizado todo bien y falta analizar más en los casos de uso ya que faltan algunos, y profundizar más en Django REST Framework.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura [A.4](#)

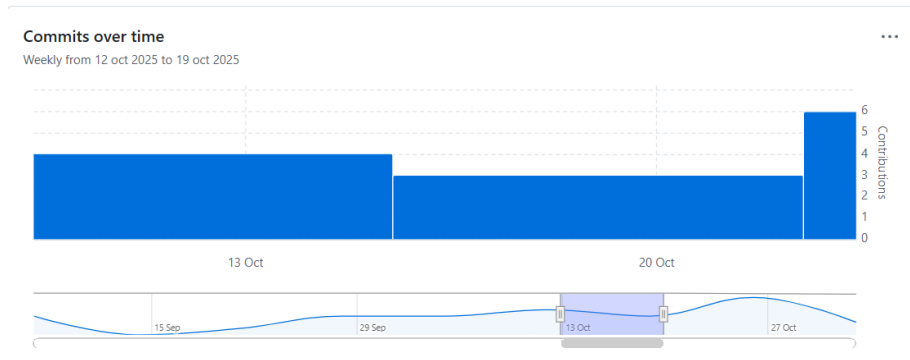


Figura A.4: Gráfico de los commits realizados durante el Sprint 4

Sprint 5 (28/10/2025-4/11/2025)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Añadir más casos de uso, ver cuáles faltan
2. Hacer diagrama de casos de uso
3. Investigar funcionamiento Django REST Framework

■ Revisión del Sprint

En la reunión de revisión del sprint, se ha visto un fallo en el diagrama de casos de uso, y en cuanto a ellos se ha decidido realizar agrupaciones en alguno de ellos para definirlos como uno solo. Y se ha realizado bien el trabajo del funcionamiento del Django REST Framework.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura [A.5](#)

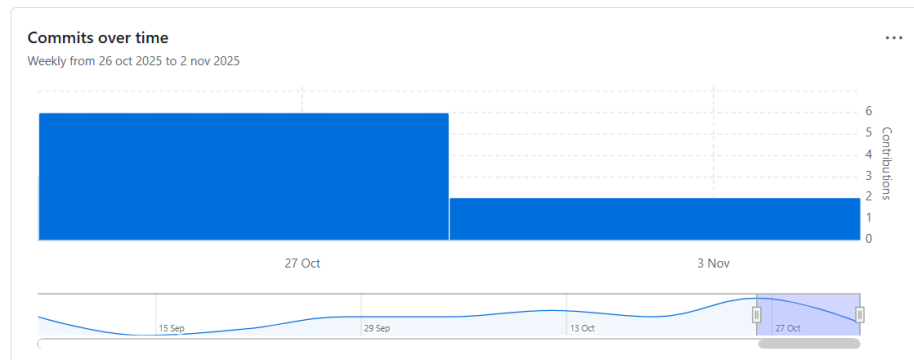


Figura A.5: Gráfico de los commits realizados durante el Sprint 5

Sprint 6 (4/11/2025-11/11/2025)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Corregir diagrama de casos de uso
2. Realizar tablas de casos de uso
3. Configurar el entorno
4. Realizar una pantalla sencilla

■ Revisión del Sprint

En esta revisión se ha determinado que se ha configurado el entorno correctamente (backend) y que se han corregido correctamente las propuestas de la revisión del sprint anterior sobre el diagrama de casos de uso, aunque se ha propuesto una mejora más. También se ha recomendado por parte de los tutores especificar más las tareas de los sprints, es decir, en vez de poner documentar casos de uso ya que es muy general ya abarca todos, para que de tiempo a hacerlo en un sprint especificar hacer casos de uso de inicio de sesión, las consultas y generación de informes, y así, ya que no he terminado de documentar todos los casos de uso por falta de tiempo y por dudas de alguno de

ellos. Tampoco se ha realizado la pantalla sencilla ya que para poder hacer la pantalla especificada requería realizar cambios en la base de datos que quise consultar con los tutores antes.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.6

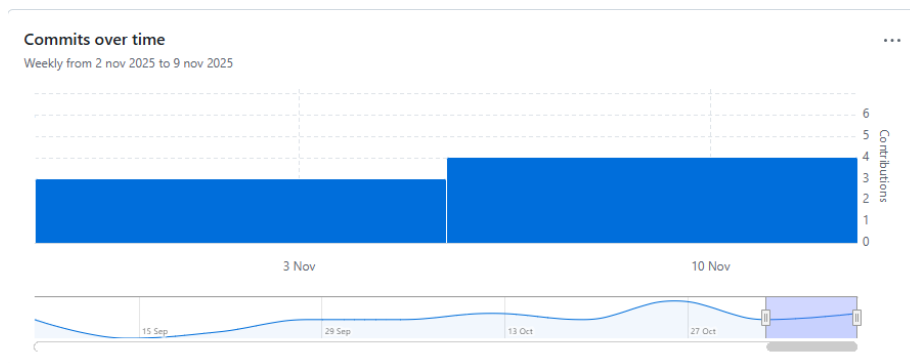


Figura A.6: Gráfico de los commits realizados durante el Sprint 6

Sprint 7 (11/11/2025-18/11/2025)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Terminar de documentar los casos de uso que quedan
2. Diseñar alguna pantalla con Pencil (menú y solicitud de reserva)
3. Investigar el funcionamiento de Django REST Framework con React
4. Programar pantalla solicitud de reserva puntual

■ Revisión del Sprint

En la revisión del sprint se han definido un par de casos de uso más que faltaban para el funcionamiento de la aplicación, y también se ha propuesto que tras la realización de la pantalla sencilla comprobar que funciona metiendo datos de prueba, lo que realizaré en el siguiente sprint.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.7

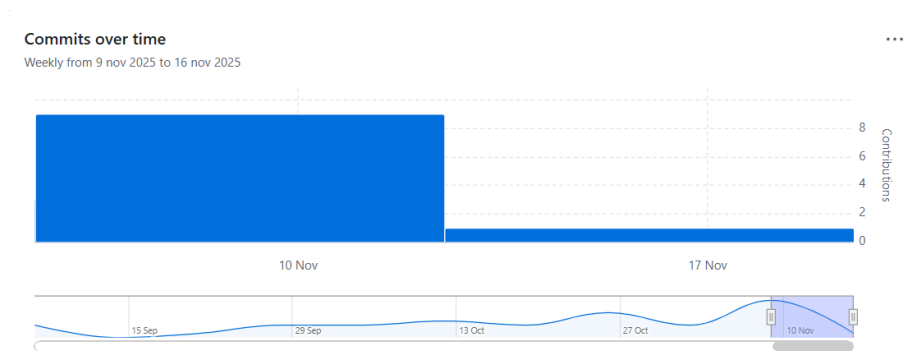


Figura A.7: Gráfico de los commits realizados durante el Sprint 7

Sprint 8 (18/11/2025-02/12/2025)

Duración: 2 semanas

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Crear datos de prueba
2. Comprobar conexión del frontend con el backend y si la reserva se guarda correctamente
3. Investigar sobre bibliotecas de calendarios

4. Implementar pantalla inicial de la definición del calendario

■ **Revisión del Sprint**

Durante la reunión de revisión de este sprint se ha visto que va a ser necesario añadir campos que describan mejor las características del aula que se va a necesitar para poder hacer un filtrado de los aulas disponibles para la reserva, también se ha recomendado mirar frameworks para mejorar la interfaz gráfica ya que de momento se encuentra muy básica. Y por último, se ha recomendado cambiar el método para poblar la base de datos, con un Excel. Y seguir investigando un poco más sobre las bibliotecas de calendario.

■ **Gráfico**

El gráfico *burndown chart* se puede ver en la figura A.8

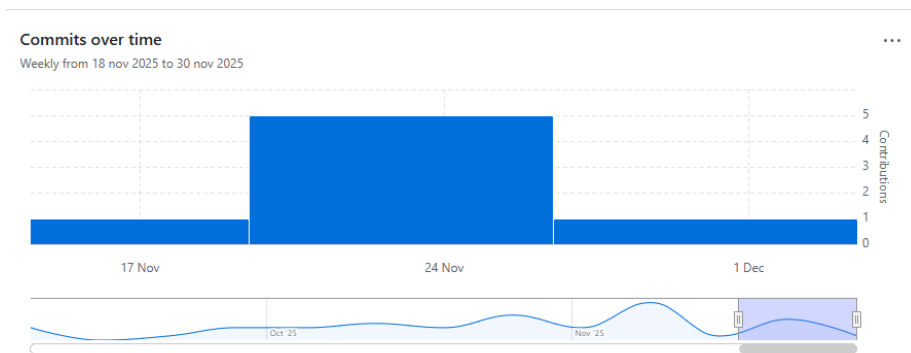


Figura A.8: Gráfico de los commits realizados durante el Sprint 8

Sprint 9 (19/12/2025-12/01/2026)

Duración: 2 semanas

■ **Objetivos**

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Realizar la página de consulta de reservas (peticiones y activas)
2. Añadir campos en la solicitud de reservas
3. Investigar sobre Tailwind y Bootstrap
4. Comenzar a realizar el script que comunique en ambas direcciones la base de datos con el Excel
5. Comenzar a comentar las técnicas y herramientas en la memoria (Django y React)
6. Elegido el framework de CSS comenzar a mejorar la estética de las pantallas

■ Revisión del Sprint

Durante la reunión de este sprint se han visto todas las dudas surgidas durante la creación de la pantalla de solicitudes pendientes principalmente, y las dudas que han ido surgiendo con el resto de tareas también. Se ha determinado que el resto de tareas estaban bien realizadas, salvo la documentación que tengo que realizar más.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.9

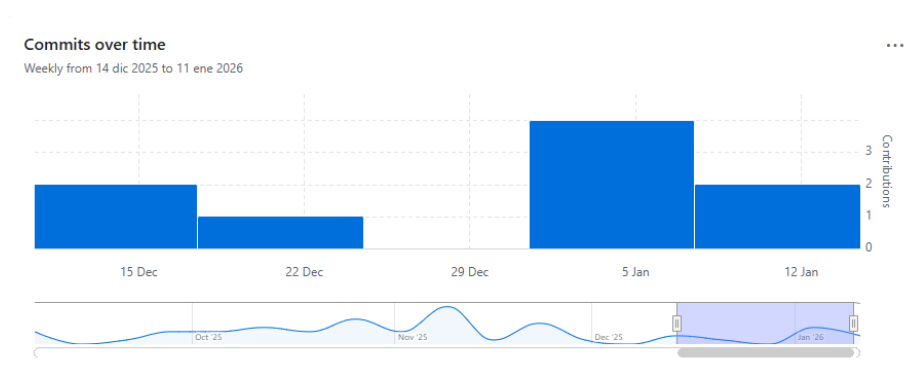


Figura A.9: Gráfico de los commits realizados durante el Sprint 9

Sprint 10 (12/01/2026-23/01/2026)

Duración: 2 semanas

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Corregir las dudas resueltas durante la revisión del sprint anterior
2. Investigar para hacer plantillas con jinja
3. Configurar la página de solicitud como plantilla para la de editar también
4. Añadir en la página de solicitudes pendientes, que permita el filtrado y la selección de varias reservas, y una ventana de confirmación cuando se generan de forma periódica.
5. Seguir documentando las técnicas y herramientas en la memoria (Tailwind y bases de datos)

■ Revisión del Sprint

Durante la reunión de este sprint, se ha determinado que los avances realizados a lo largo del sprint están bien, pero falta realizar más documentación. Finalmente no pude realizar la documentación de la memoria prevista por falta de tiempo para ello, pero se hará para el siguiente sprint.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura [A.10](#)

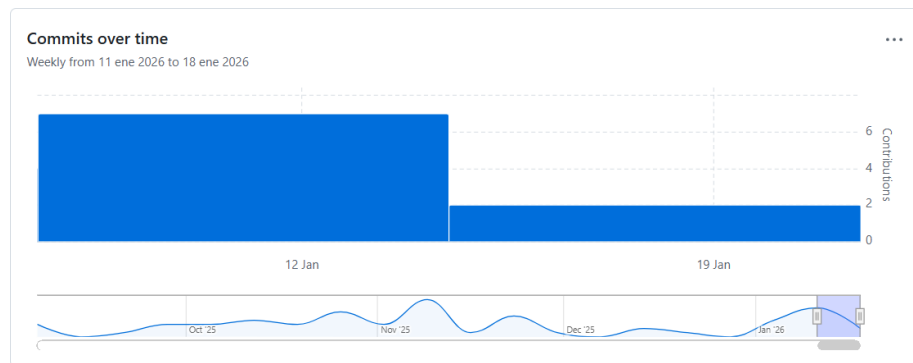


Figura A.10: Gráfico de los commits realizados durante el Sprint 10

Sprint 11 (23/01/2026-11/02/2026)

Duración: 2 semanas y media

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Buscar templates para React
2. Corrección de que aparezcan solicitudes solamente desde el día de hoy en adelante
3. Mirar bien bibliotecas de calendario y decidir
4. Empezar a ver cómo hacer pantallas con las bibliotecas de calendarios.
5. Corregir base de datos con los cambios.
6. Mirar login pasarela con la universidad
7. Documentar memoria (Tailwind, Gestión de la Base de Datos y biblioteca de calendarios escogida).

■ Revisión del Sprint

Durante este sprint se han realizado todos los objetivos propuestos salvo el de corregir la base de datos, por una duda que surgió al respecto que se ha comentado durante esta reunión, y la de mirar el login con

las pasarela con la universidad, ya que al final no se va a poder realizar de esa forma, y estamos esperando a que desde arriba nos den una respuesta a la propuesta mandada. Comentar que finalmente no se ha utilizado una biblioteca para hacer templates con react, sino que se han dividido las páginas en componentes que se van juntando en contenedores para evitar la duplicidad del código.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.11

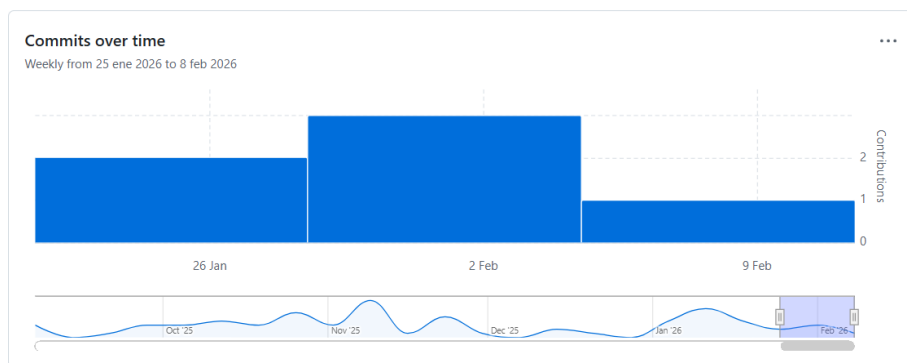


Figura A.11: Gráfico de los commits realizados durante el Sprint 11

Sprint 12 (11/02/2026-18/02/2026)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Seguir documentando, Técnicas y herramientas la biblioteca de calendarios, y comenzar a mirar los trabajos relacionados.
2. Realizar el menú de la forma propuesta, y hacer la página de selección de uno o varios aulas.

3. Modificar la base de datos
4. Descargar biblioteca de calendarios y empezar a realizar la pantalla de la ocupación por aula.

■ Revisión del Sprint

Durante la revisión de este sprint, se ha visto los avances relacionados con la documentación, que finalmente fueron los puntos 1 y 2 de la memoria, en vez de las técnicas y trabajos relacionados. El resto está todo realizado, a falta de conseguir conectar de manera correcta en la ventana de ocupación de aulas, el backend con el frontend para que muestre las reservas.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura [A.12](#)

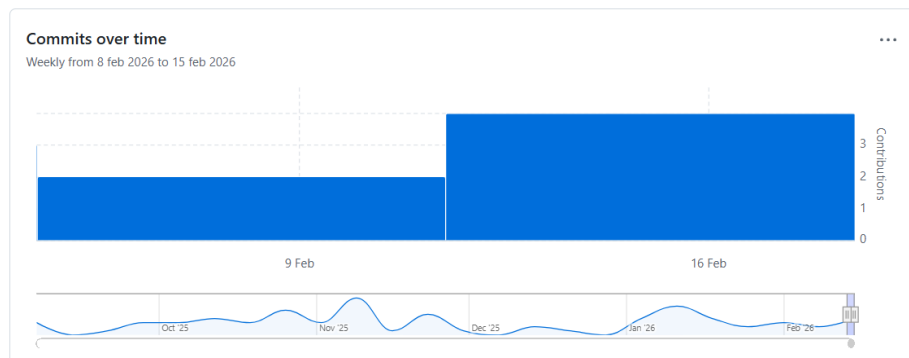


Figura A.12: Gráfico de los commits realizados durante el Sprint 12

Sprint 13 (18/02/2026-25/02/2026)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Seguir documentando, Técnicas y herramientas, la biblioteca de calendarios, y comenzar a mirar los trabajos relacionados.
2. Conectar aulas y modificar ventana para que se vean las reservas de todas las aulas seleccionadas
3. Modificar menú, eliminar opciones para flujo más sencillo
4. Juntar las solicitudes pendientes y todas las reservas en una sola página.

■ Revisión del Sprint

Durante la reunión del sprint se ha determinado un cambio respecto a la manera de calcular el color del aula en la página de ocupación con el calendario para que siempre tenga el mismo color, lo que deriva una modificación en la base de datos. Se han cumplido los objetivos de manera correcta, aunque hay que realizar algo más de documentación.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.13

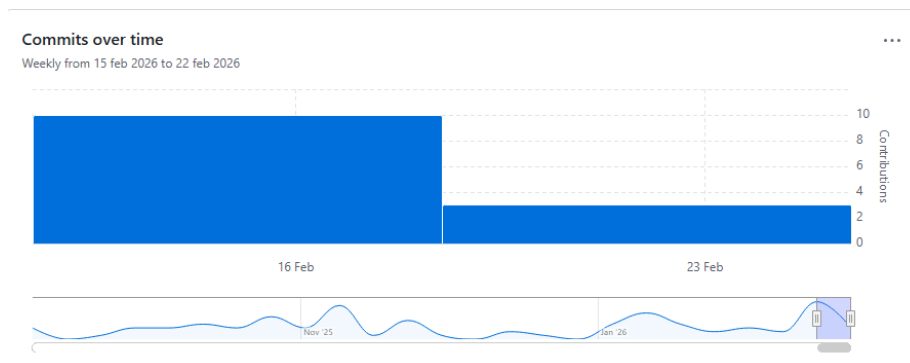


Figura A.13: Gráfico de los commits realizados durante el Sprint 13

Sprint 14 (25/02/2026-04/03/2026)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Comenzar a documentar el apartado de trabajos relacionados, y el de conceptos teóricos.
2. Añadir el id como atributo en la tabla Aula y modificar forma de calcular el color del aula.
3. Modificar la ventana de gestión de reservas, haciendo los filtros visibles y que por defecto al cargar la página se establezca el rango de fecha desde la fecha actual de cuando se carga.
4. Cambiar de color los submenús para que se vea mejor la diferencia.
5. En la página de la ocupación realizada con el calendario, que pulsando un día te cambie a la vista de día del mismo.
6. Mirar funcionamiento y comenzar a trastear con openpyxl para pasar los datos del excel de horarios a la base de datos.

■ Revisión del Sprint

Durante este sprint se ha dispuesto de menos tiempo, con lo que hay tareas que no se han terminado de completar, que en este caso son la documentación, faltan de cargar los cambios en la base de datos, y se ha investigado el funcionamiento de openpyxl, falta trastear tanto con ello como con el analizador léxico, con lo que estas tareas o la parte de ellas que queda pasan al siguiente sprint. Respecto a las modificaciones realizadas en el menú se ha propuesto que el cambio de color de los submenús sea más notable y se ha propuesto una mejora de que se cierren el resto de submenús al abrir uno nuevo de forma que no queden abiertas muchas pestañas a la vez para que se más limpio y claro.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura [A.14](#)

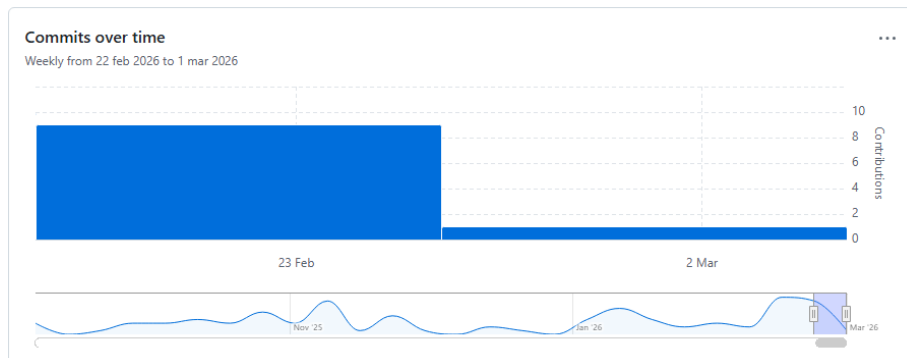


Figura A.14: Gráfico de los commits realizados durante el Sprint 14

Sprint 15 (04/03/2026-11/03/2026)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Comenzar a documentar el apartado de trabajos relacionados, y el de conceptos teóricos.
2. Cargar cambios en la base de datos y modificar forma de calcular el color del aula.
3. Realizar modificaciones nuevas relativas al menú.
4. Comenzar a trabajar y a trastear tanto como con `openpyxl` como con un analizador léxico de python.

■ Revisión del Sprint

Durante la reunión de este sprint, ya se ha determinado el correcto funcionamiento del menú, y de los colores de los aulas al ver la ocupación. Además, se ha visto como se ha trastearado y trabajado en un *notebook* con `openpyxl` y un analizador léxico de python. Por último, se han propuesto un par de cambios en la ventana de solicitar reserva, de añadir el nombre y apellidos del responsable si este no existe en base de datos, y hacer una paginador en la de gestión de reservas.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.15

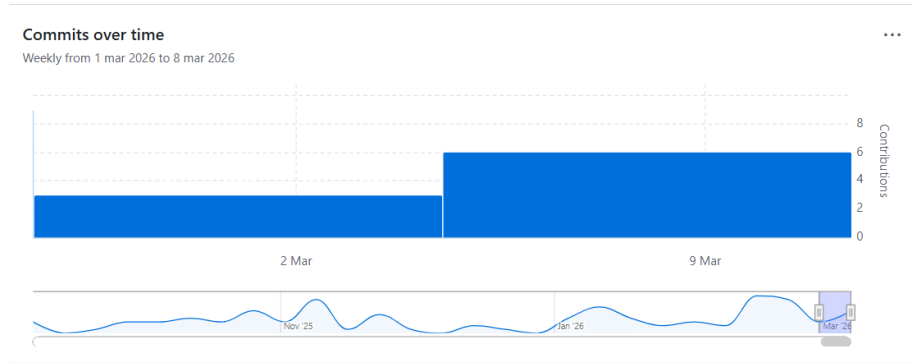


Figura A.15: Gráfico de los commits realizados durante el Sprint 15

Sprint 16 (11/03/2026-25/03/2026)

Duración: 2 semanas

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Comenzar con la extracción de datos de los horarios.
2. Programar modal en Solicitar reserva, y el paginador en Gestión de reservas.
3. Si da tiempo investigar el login.

■ Revisión del Sprint

Durante la reunión de este sprint se ha determinado que se ha cumplido con el objetivo de realizar la modal en la solicitud de reserva si no se encuentra el responsable introducido en base de datos, y la implementación del paginador en la página de gestión de reservas. Ha

quedado pendiente mirar el login y tratar de terminar la extracción de datos de los horarios.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.16

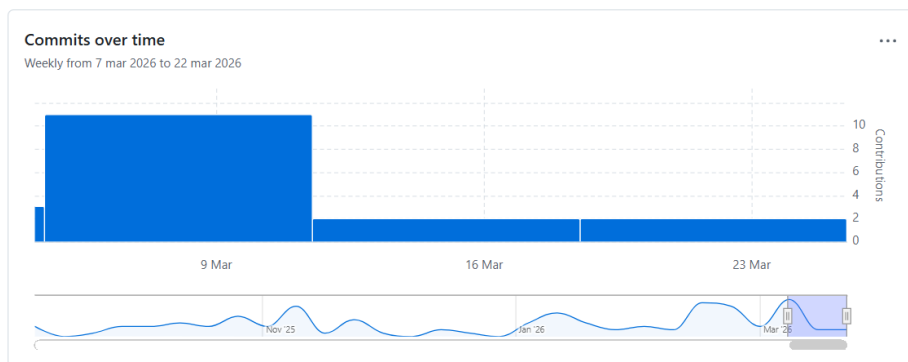


Figura A.16: Gráfico de los commits realizados durante el Sprint 16

Sprint 17 (25/03/2026-08/04/2026)

Duración: 2 semanas

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Completar la extracción de clases del excel de horarios.
2. Implementar la funcionalidad del login
3. Modificar el input de las horas para que vaya de 15 minutos en 15 minutos.

■ Revisión del Sprint

La funcionalidad para conectar el login con el api del moodle de la universidad se ha quedado a medias, es decir, no se ha terminado. Se

ha conseguido que el analizador léxico de momento extraiga casi todos los tipos de clases, y se habían realizado dos tipos de input para elegir la hora en el formulario de la reserva y se ha determinado durante la reunión cuál de los dos es el que se va a utilizar.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.17

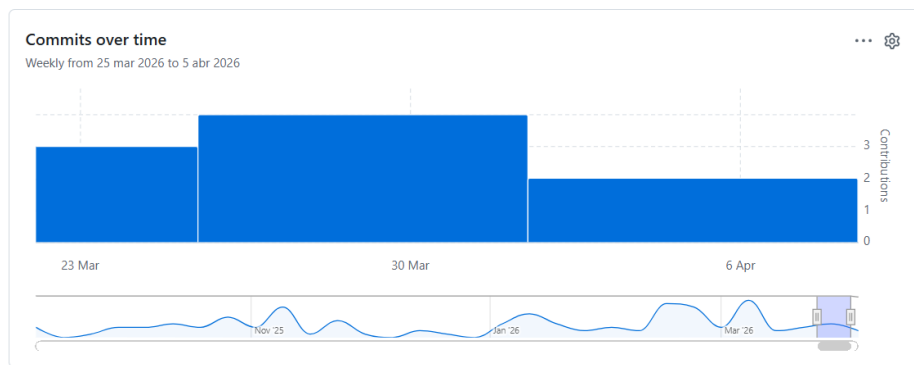


Figura A.17: Gráfico de los commits realizados durante el Sprint 17

Sprint 18 (08/04/2026-15/04/2026)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Extraer datos de los tramos horarios para la reserva, extraer las dos aulas cuando hay una clase en que aparecen separadas por “,” o “/”, y extraer las clases teóricas cuando hay dos grupos teóricos.
2. Programar frontend inicio de sesión, y probar la conexión con el api del moodle.

3. En el formulario de la reserva añadir el campo de observaciones que no sea obligatorio, hace falta añadir también el atributo en base de datos.
4. Conectar sonarqube al repo.
5. Corregir anexos, y avanzar algo de documentación.

■ Revisión del Sprint

Durante esta revisión se han comentado algunos problemas que se ha tenido con el analizador léxico y el excel que se han resuelto, pero se han conseguido todos los objetivos salvo el de conectar sonarqube con el repo que pasa al siguiente sprint. Y se ha visto como se ha avanzado algo de documentación del apéndice A de anexos.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.18

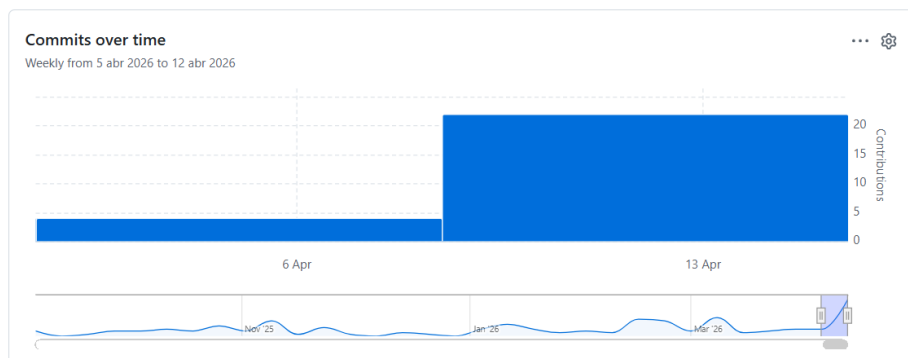


Figura A.18: Gráfico de los commits realizados durante el Sprint 18

Sprint 19 (15/04/2026-22/04/2026)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Cambiar el analizador para coger las asignaturas de otra forma, y hacer validador de lo que me tiene que devolver y lo que me devuelve para ver si funciona correctamente.
2. Sacar código de asignatura, y ver como obtener el id del grupo.
3. Quitar el nombre excel de la tabla aulas
4. Conectar analizador léxico con el proyecto para ver si funciona.
5. Añadir las combinaciones de aulas que faltan en el Excel, y terminar de añadir reglas.
6. Cargar aulas y asignaturas en base de datos.
7. Conectar sonarqube al repo.
8. Corregir memoria, y avanzar algún apartado 6 de la memoria.
9. Investigar para comenzar a hacer tests y cobertura del back.

■ Revisión del Sprint

Este sprint no se han podido llevar a cabo muchos de los objetivos propuestos porque el validador y corregir el Excel ha llevado mucho más tiempo del esperado y aún no se ha terminado de realizar. Lo que sí que se ha hecho ha sido conectar el sonarqube al repositorio, modificar la forma de encontrar las asignaturas y modificar la estructura para encontrar cuando todavía no se ha asignado un aula a las asignaturas.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura [A.19](#)

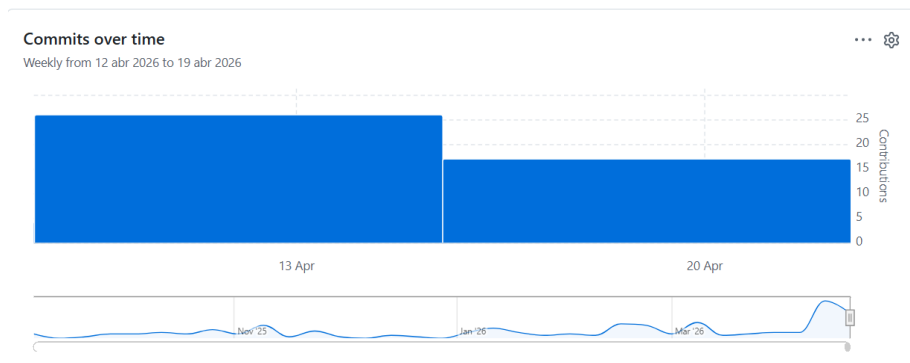


Figura A.19: Gráfico de los commits realizados durante el Sprint 19

Sprint 20 (22/04/2026-06/05/2026)

Duración: 2 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Modificar la tabla de Reservas periódicas, añadiendo como clave foránea IdGrupo y eliminando la clave foránea IdAsignatura.
2. Sacar código de asignatura, y ver como obtener el id del grupo y el aula id.
3. Quitar el nombre excel de la tabla aulas
4. Conectar analizador léxico con el proyecto para ver si funciona.
5. Añadir las combinaciones de aulas que faltan en el Excel, y terminar de añadir reglas.
6. Cargar aulas y asignaturas en base de datos.
7. Dividir cada tipo de estructura de la clase en funciones.
8. Generar reserva periódica.
9. Pasarlo a un script en python.
10. Investigar tema de migraciones de roles y usuarios.
11. Corregir memoria, y avanzar algún apartado 6 de la memoria.
12. Investigar para comenzar a hacer tests y cobertura del back.

■ Revisión del Sprint

Durante la revisión de este sprint, se han comentado las dudas que han ido surgiendo a lo largo del sprint. Y se ha visto cómo se han cargado todos los aulas y asignaturas en base de datos, esta también ha sido modificada con los cambios que vimos en la reunión anterior. Se ha pasado todo el código del analizador a django a falta de modificar alguna cosa que se ha determinado durante la reunión. Ha faltado la corrección de la memoria, ya que en cuanto a documentación lo que se ha avanzado han sido los requisitos funcionales que se han terminado y falta de documentar algún requisito no funcional. Además, se ha comentado el flujo de la carga de un excel de horarios para implementar en el siguiente sprint.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.20

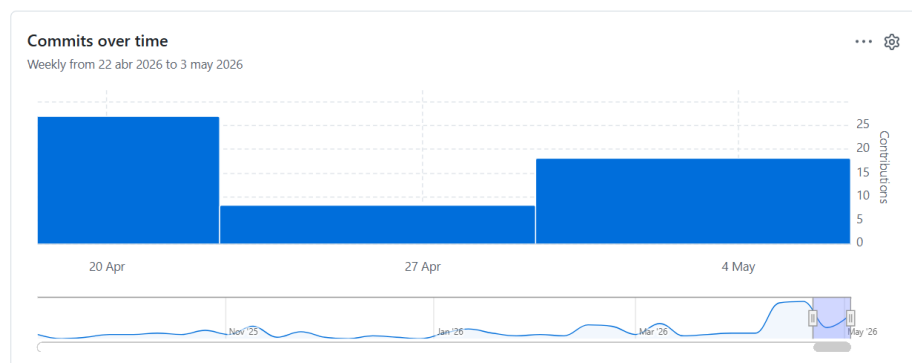


Figura A.20: Gráfico de los commits realizados durante el Sprint 20

Sprint 21 (06/05/2026-13/05/2026)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Modificar las aulas en base de datos con la nomenclatura determinada en la reunión.
2. Implementar flujo de carga del excel de horarios si existe el curso académico al que quiere asociarse.
3. Hacer flujo cuando no existe el curso académico para que permita crearlo.
4. Crear la vista que muestre el calendario con los días.
5. Quitar el atributo de capacidad máxima de la tabla grupo.
6. En el Excel terminar de unificar las asignaturas, y hacer una hoja con el listado de grados del que quiere extraerse la información.
7. Modificar entidad-relación haciendo una ISA de los grupo, ya que solamente añaden id de aulas a los grupos teóricos.
8. Implementar que en la carga de datos a la base de datos, si no encuentra un grupo lo cree.
9. Desarrollar vista de calendario con los diferentes colores.
10. Corregir memoria, y avanzar apartado de aspectos relevantes y de conceptos teóricos. Y terminar de documentar los requisitos no funcionales.
11. Añadir la excepción al parser de los grupos teóricos.

■ Revisión del Sprint

Durante la reunión se han visto los resultados asociados a los objetivos propuestos para el sprint. Ha faltado por una duda surgida, comprobar que se realizan las reservas periódicas del análisis de manera correcta.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura [A.21](#)

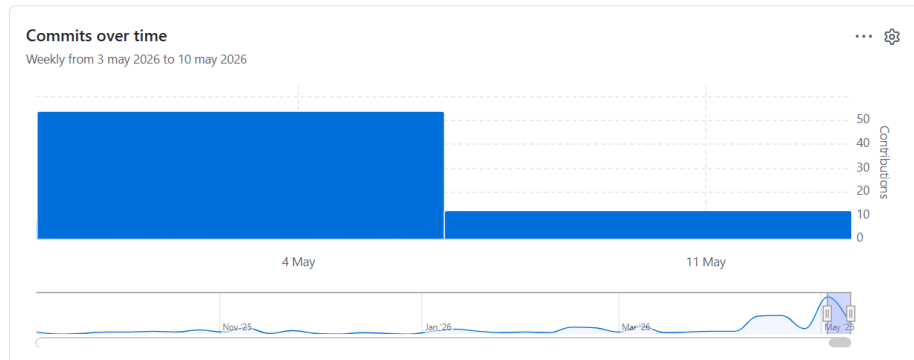


Figura A.21: Gráfico de los commits realizados durante el Sprint 21

Sprint 22 (13/05/2026-22/05/2026)

Duración: 1 semana y 2 días

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Hacer que cuando se quita el check de Añadir festivos se vacíe la lista.
2. Si hay más de 1 día seleccionado no permita escoger la opción de cambio de docencia.
3. Comprobar que el sábado y el domingo no se seleccionen.
4. Comprobar que tras cargar el horario se crean las reservas periódicas correctamente.
5. Realizar la sesión de usuario y que compruebe si está iniciado sesión cuando trata de acceder a través de URL.
6. Delegar para que Django se encargue de roles y permisos.
7. Corregir memoria, y avanzar apartado de trabajos relacionados y de conceptos teóricos.

■ Revisión del Sprint

Durante este sprint por ser fechas de exámenes no se han podido cumplir todos los objetivos. Como se ha mostrado en la revisión, se

han podido cumplir los primeros 4 objetivos, con lo que los otros 3 pasan al siguiente sprint.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.22

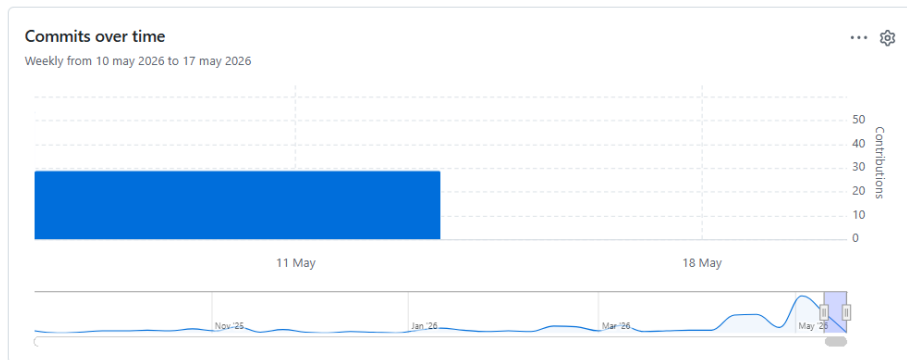


Figura A.22: Gráfico de los commits realizados durante el Sprint 22

Sprint 23 (22/05/2026-27/05/2026)

Duración: 5 días

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. El diseño del formulario de la creación de un nuevo calendario tengan todos los campos la misma sangría.
2. Poner el color del texto de las reservas periódicas en la ocupación de aulas en negro y ver si es viable poner el nombre de la abreviatura de la asignatura y con el cursor aparezca el nombre completo.
3. Mirar porque aparecen los números de los sábados del mes anterior e intentar que los backgrounds que no pertenecen a ese mes sean blancos.

4. Programar el formulario para crear la reserva periódica y ver los detalles de la misma.
5. Ventana de las reservas periódicas por grado y por semestre.
6. Realizar la sesión de usuario y que compruebe si está iniciado sesión cuando trata de acceder a través de URL.
7. Delegar para que Django se encargue de roles y permisos.
8. Corregir memoria, y avanzar apartado de trabajos relacionados y de conceptos teóricos.

■ Revisión del Sprint

En la revisión del sprint se han visto los avances realizados durante la semana, a falta de terminar de realizar que django se encargue de la gestión de usuarios y roles, y avanzar los trabajos relacionados. Respecto a los objetivos cumplidos se han propuesto algunas mejoras en cuanto a usabilidad del usuario de la aplicación que pasan a ser objetivos del siguiente sprint.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.23

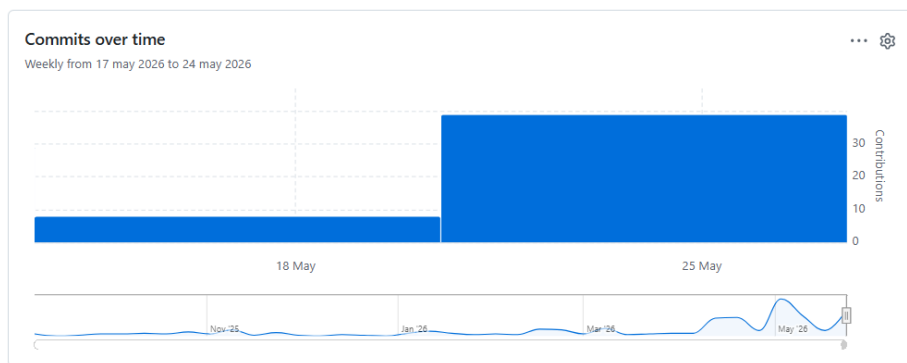


Figura A.23: Gráfico de los commits realizados durante el Sprint 23

Sprint 24 (27/05/2026-03/06/2026)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Implementar desplegable con los grados para los horarios y quitar la página anterior que contiene el listado de cursos
2. Hacer que el analizador léxico recorra todas las filas del tramo horario.
3. Debuguear para ver porqué no coge Exp. Gráfica I del doble grado.
4. Quitar el día actual del horario.
5. Al cargar un horario dar aviso de que se van a eliminar todas las reservas de los grados con docencia de ese año, y eliminarlas previamente.
6. Programar validación de restricciones que compruebe al realizar drag & drop, salga modal si se violan restricciones, y si se da a Continuar modifique todas las reservas periódicas afectadas.
7. Listar los aulas en 3 columnas, se puede hacer división de aulas, laboratorios y otros aulas.
8. Mostrar en el *hover* del horario la asignatura, el grupo y el aula para que se vean completo.
9. Unificar nombres Laboratorio Ing. Eléctrica y Sala de Juntas en la hoja de cálculo y base de datos.
10. Programar el panel de administrador.
11. Delegar para que Django se encargue de roles y permisos.
12. Contrastar usuario y rol contra la base de datos, sino se encuentra se crea la entrada con rol PDI y pasar el rol al *frontend*.
13. Configurar las acciones en función de los permisos.
14. Meter la importación y exportación para realizarla en la interfaz.
15. Meter la solicitud de reserva comonavegación de creación de reserva para el PDI y quitarle el menú.

16. Corregir memoria, y avanzar apartados de trabajos relacionados, conceptos teóricos y aspectos relevantes.

■ Revisión del Sprint

Se ha decretado que durante este sprint se han completado casi todos los objetivos, los que han faltado por terminar pasan al siguiente sprint, que han sido las importaciones y exportaciones, y que al cargar horario de aviso y elimine.

■ Gráfico

El gráfico *burndown chart* se puede ver en la figura A.24

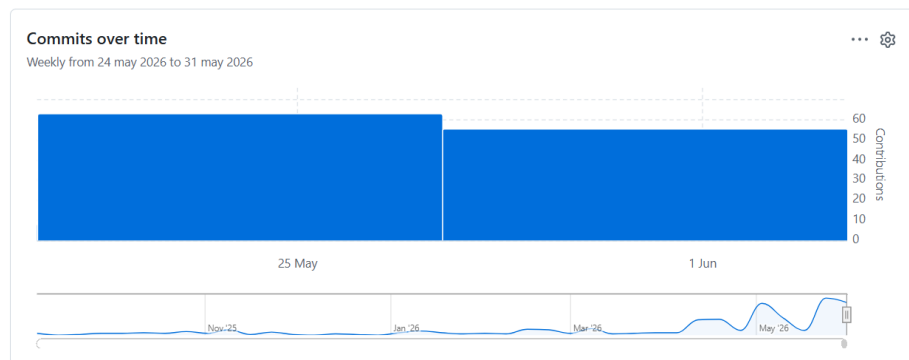


Figura A.24: Gráfico de los commits realizados durante el Sprint 24

Sprint 25 (03/06/2026-10/06/2026)

Duración: 1 semana

■ Objetivos

Durante la reunión de planificación de este sprint se propusieron los siguientes objetivos:

1. Cambiar los ids de las aulas, docentes e imparte autoincrementales.

2. Programar administración de imparte.
3. Permitir editar reserva periódica.
4. Al cargar un horario dar aviso de que se van a eliminar todas las reservas de los grados con docencia de ese año, y eliminarlas previamente.
5. Realizar el despliegue.
6. Mirar problemas sonarqube.
7. Terminar de documentar todo el código.
8. Programar modal reutilizable para cada vez que se quiera eliminar algo.
9. Terminar de documentar técnicas y herramientas.
10. Documentar conclusiones y líneas futuras.
11. Documentar resumen y palabras clave.
12. Documentar correcciones de profesores.
13. Poner imagen y título a la hoja de inicio de sesión.
14. Meter la importación y exportación para realizarla en la interfaz a nivel global y por entidad.
15. Implementar tests base de datos.
16. Implementar tests back.
17. Programar botones para eliminar.
18. Terminar anexos.
19. Diseñar bonito el readme.

■ **Revisión del Sprint**

■ **Gráfico**

A.3. Estudio de viabilidad

En este apartado se analiza la viabilidad del proyecto desde dos perspectivas principales: la viabilidad económica y la viabilidad legal. La primera

permite estimar el coste que tendría el desarrollo de la aplicación en un entorno profesional real, teniendo en cuenta los recursos humanos, materiales, software, infraestructura y otros gastos indirectos necesarios para su realización. La segunda estudia los aspectos relacionados con el uso de herramientas externas, licencias de software, tratamiento de datos personales y utilización de información académica.

Aunque el proyecto se ha desarrollado dentro de un contexto académico, resulta conveniente realizar la estimación para valorar el esfuerzo real que supondría su construcción, despliegue y mantenimiento en un entorno institucional.

Viabilidad económica

Para realizar el estudio económico se han tenido en cuenta los principales recursos necesarios para el desarrollo de la aplicación. Entre ellos se encuentran los costes de personal, el equipo utilizado durante el desarrollo, el software empleado, la infraestructura de despliegue y otros costes indirectos como la conexión a Internet o el consumo eléctrico.

El desarrollo del proyecto se ha llevado a cabo entre el 8 de septiembre y el 10 de junio de 2026, lo que supone una duración aproximada de nueve meses. Durante este periodo, el esfuerzo total estimado a partir de los *issues* definidos en GitHub ha sido de 457 horas. Este valor se utilizará como base para calcular el coste de personal del proyecto.

Costes de personal

El principal coste del proyecto corresponde al tiempo dedicado al análisis, diseño, implementación, pruebas y documentación de la aplicación. Para estimar este coste se ha partido de las 457 horas registradas a partir de las estimaciones de los *issues* del repositorio de GitHub.

Al tratarse de un Trabajo de Fin de Grado, no ha existido un coste real de contratación. Sin embargo, para obtener una aproximación más realista, se puede considerar que el desarrollo habría sido realizado por una persona con perfil de desarrollador *full stack junior*. De esta forma, el coste total de personal se calcularía multiplicando el número total de horas dedicadas por el coste horario estimado para dicho perfil. Investigando el salario medio de un trabajador para este perfil, se ha asignado el coste por hora en 12 euros.

$$\text{Coste de personal} = \text{Horas dedicadas} \times \text{Coste por hora}$$

Sustituyendo las horas dedicadas y el coste por hora:

$$\text{Coste de personal} = 457 \times 12 = 5484 \text{ euros}$$

Además del salario bruto, en un entorno profesional la empresa debe asumir una serie de costes adicionales asociados a las cotizaciones a la Seguridad Social. Para esta estimación se ha una nómina propia, y se ha buscado el porcentaje para los accidentes de trabajo ya que varía dependiendo de la profesión [19]. A través de ambas fuentes se han deducido los siguientes porcentajes:

- Contingencias comunes: 23,6 %
- Desempleo de empresa: 6,7 %
- Accidentes de trabajo: 1,5 %
- Formación profesional: 0,6 %
- FOGASA: 0,2 %
- MEI (Mecanismo de Equidad Intergeneracional): 0,75 %

Siendo la suma total de todos los porcentajes un 33,35 %.

$$\text{Cotizaciones empresariales} = 5484,00 \times 0,3335 = 1828,91 \text{ euros}$$

Por tanto, el coste total de personal para la empresa sería:

$$\text{Coste total de personal} = 5484,00 + 1828,91 = 7312,91 \text{ euros}$$

Como se muestra en la Tabla A.1, el coste estimado de personal asociado al desarrollo del proyecto asciende a 7312,91 euros.

Concepto	Coste
Salario bruto estimado	5484,00 €
Cotizaciones empresariales	1828,91 €
Coste total de personal	7312,91 €

Tabla A.1: Costes de personal del proyecto

Costes de hardware

Para el desarrollo del proyecto se ha utilizado un ordenador portátil Notebook Asus V16 V3607VM, cuyo precio de adquisición fue de 1299,00 euros. Como este equipo no se ha utilizado exclusivamente para el proyecto y su vida útil se extiende durante varios años, no se imputa el coste completo del dispositivo, sino únicamente la parte proporcional correspondiente al periodo de desarrollo.

Se ha considerado una vida útil estimada de 5 años, equivalente a 60 meses. Dado que el proyecto ha tenido una duración aproximada de nueve meses, el coste amortizado del equipo se calcula de la siguiente forma:

$$\text{Coste amortizado} = \frac{1299}{60} \times 9 = 194,85 \text{ euros}$$

En la Tabla A.2 se recoge el cálculo del coste de hardware del proyecto.

Elemento	Coste total	Vida útil	Coste imputado
Notebook Asus V16 V3607VM	1299,00 €	60 meses	194,85 €

Tabla A.2: Costes de hardware del proyecto

Costes de software

Durante el desarrollo se han utilizado principalmente herramientas gratuitas o de código abierto, por lo que el coste directo de software ha sido recudido. Entre estas herramientas se encuentran los entornos de desarrollo, *frameworks* y bibliotecas necesarias para la construcción de la aplicación web.

No obstante, en este caso, el coste del sistema operativo sí debe considerarse de forma independiente, ya que el equipo utilizado para el desarrollo fue adquirido sin sistema operativo incluido. El sistema operativo utilizado durante el desarrollo ha sido Windows 11 Home, cuyo coste se ha obtenido a partir de la tienda oficial de Microsoft [12].

El resto de herramientas utilizadas para el desarrollo, control de versiones, *backend*, *frontend* y gestión de la base de datos no han supuesto un coste directo para el proyecto, al tratarse de herramientas gratuitas o de código abierto.

Para el control de versiones y la gestión de *issues* se ha utilizado GitHub, cuyo plan gratuito permite trabajar con repositorios públicos y privados sin coste mensual [8].

Como se muestra en la Tabla A.3, el único coste directo de software corresponde a la licencia del sistema operativo Windows.

Elemento	Uso	Coste
<i>Windows</i>	Sistema operativo	145,00 €
GitHub	Control de versiones y gestión de <i>issues</i>	0,00 €
Herramientas de desarrollo	Implementación del proyecto	0,00 €
<i>Frameworks</i> y bibliotecas	Desarrollo <i>backend</i> y <i>frontend</i>	0,00 €
Total		145,00 €

Tabla A.3: Costes de software del proyecto

Costes de infraestructura y despliegue

Para el despliegue de la aplicación se ha utilizado Railway. Esta plataforma ofrece un periodo inicial gratuito de 30 días, lo que permite realizar pruebas de despliegue sin coste durante la fase inicial. Sin embargo, para mantener la aplicación disponible de forma continuada en un entorno real, sería necesario contratar un plan de pago una vez finalizado dicho periodo gratuito.

En este proyecto se considera suficiente el plan *Hobby*, ya que la aplicación está orientada a un entorno académico y su uso previsto durante la fase inicial será reducido, principalmente para pruebas, demostración y validación del sistema. Este plan tiene un coste de 5 dólares mensuales e incluye 5 dólares de consumo en recursos [18], por lo que resulta adecuado para mantener la aplicación desplegada durante unos meses sin necesidad de contratar un plan superior.

Para el estudio económico se ha estimado un periodo de despliegue de tres meses, suficiente para cubrir la fase de pruebas, revisión y defensa del proyecto.

$$\text{Coste de despliegue} = 5 \times 3 = 15 \text{ dólares}$$

Tomando una conversión aproximada, el coste total del despliegue durante tres meses sería de unos 14 euros.

Como se recoge en la Tabla A.4, el coste estimado para mantener la aplicación desplegada durante tres meses mediante Railway sería de 15 dólares.

Servicio	Plan	Coste mensual	Coste estimado
Railway	Hobby	5,00 \$/mes	15,00 \$

Tabla A.4: Costes de infraestructura y despliegue

Resumen de costes

Una vez analizados los costes de personal, hardware, software e infraestructura, se obtiene una estimación del coste total del proyecto. Esta estimación no representa un gasto real realizado durante el Trabajo de Fin de Grado, sino una aproximación al coste que tendría desarrollar una aplicación de estas características en un entorno profesional.

No se han incluido costes indirectos de forma independiente, como electricidad o conexión a Internet, debido a que no se dispone de una medición exacta del consumo asociado únicamente al desarrollo del proyecto. Del mismo modo, no se ha aplicado IVA al cálculo final, ya que el objetivo del estudio no es establecer un precio de venta o facturación, sino estimar el coste interno aproximado del desarrollo.

Concepto	Coste estimado
Costes de personal	7312,91 €
Costes de hardware	194,85 €
Costes de software	145,00 €
Costes de infraestructura y despliegue	14,00 €
Coste total estimado	7666,76 €

Tabla A.5: Resumen de costes del proyecto

Como se observa en la Tabla A.5, el coste total estimado del proyecto asciende a 7666,76 euros. La mayor parte de este importe corresponde a los costes de personal, ya que el desarrollo de la aplicación ha requerido tareas de análisis, diseño, implementación, pruebas, despliegue y documentación.

Viabilidad legal

En este apartado se analiza la viabilidad legal del proyecto, teniendo en cuenta las herramientas utilizadas durante el desarrollo, sus respectivas licencias y el tratamiento de datos personales dentro de la aplicación. Este análisis resulta necesario debido a que el sistema gestiona usuarios autenticados, roles de acceso, reservas de aulas, horarios docentes e información asociada al ámbito académico.

Licencias de software

En la Tabla A.6 se muestran las principales herramientas, bibliotecas y servicios utilizados durante el desarrollo del proyecto, junto con su uso dentro de la aplicación y la licencia asociada.[17, 5, 6, 10, 21, 20, 15, 14, 1, 11].

Herramienta / Biblioteca	Uso en el proyecto	Licencia
Python	Lenguaje principal del backend	PSF License
Django	Framework de desarrollo backend	BSD-3-Clause
Django REST Framework	Desarrollo de la API REST	BSD
django-cors-headers	Configuración de CORS	MIT
Requests	Comunicación con servicios externos	Apache License 2.0
openpyxl	Lectura y procesamiento de hojas Excel	MIT
React	Desarrollo de la interfaz frontend	MIT
Vite	Herramienta de construcción del frontend	MIT
Tailwind CSS	Diseño de estilos de la interfaz	MIT
MySQL Community Edition	Sistema gestor de base de datos	GPL
GitHub	Control de versiones y gestión de issues	Servicio externo
Railway	Despliegue de la aplicación	Servicio externo
Windows	Sistema operativo de desarrollo	Licencia propietaria

Tabla A.6: Resumen de licencias de las herramientas utilizadas

Como se observa en la Tabla A.6, la mayor parte de las tecnologías utilizadas se distribuyen bajo licencias permisivas, principalmente MIT, BSD y Apache 2.0. Estas licencias permiten utilizar, modificar y distribuir el software con pocas restricciones, siempre que se respeten las condiciones establecidas, como conservar los avisos de copyright y la información de licencia correspondiente [13, 2, 5].

Además, se han utilizado algunos servicios externos, como GitHub para el control de versiones y Railway para el despliegue de la aplicación. Estos servicios no se incorporan como parte del código fuente del proyecto, sino que se emplean como plataformas de apoyo durante el desarrollo y la puesta en producción de la aplicación [8, 18].

Tipos de licencias utilizadas

Entre las licencias identificadas en el proyecto destacan las siguientes:

- MIT License: es una licencia permisiva que permite usar, copiar, modificar, fusionar, publicar, distribuir y sublicenciar el software, siempre que se mantenga el aviso de copyright y la licencia original [13]. Esta licencia aparece en herramientas como React, Vite, Tailwind CSS, openpyxl y django-cors-headers [10, 21, 20, 14, 1].
- BSD: es una familia de licencias permisivas que permite el uso, modificación y redistribución del software, con la obligación principal de conservar los avisos de copyright. En este proyecto aparece asociada a herramientas como Django y Django REST Framework [5, 6].
- Apache License 2.0: permite el uso, modificación y distribución del software, incluyendo uso comercial. Además, incorpora condiciones relacionadas con la conservación de la licencia y la notificación de cambios realizados [2]. En este proyecto se encuentra asociada a bibliotecas como Requests [9].
- GPL: es una licencia de tipo copyleft, orientada a garantizar la libertad de uso, modificación y distribución del software [7]. En este proyecto aparece asociada a MySQL Community Edition, que se distribuye bajo licencia GPL [15]. Su uso no impide el desarrollo académico de la aplicación, aunque debe tenerse en cuenta si en el futuro se distribuyera el sistema completo o se modificaran componentes cubiertos por esta licencia.

- Licencia propietaria: corresponde al sistema operativo Windows utilizado durante el desarrollo. Su uso está sujeto a las condiciones establecidas por Microsoft y no afecta directamente a la licencia del código desarrollado, ya que se utiliza únicamente como sistema operativo del entorno de trabajo [11].

Elección de la licencia del proyecto

Teniendo en cuenta las licencias de las herramientas utilizadas, se considera adecuado distribuir el código fuente del proyecto bajo una licencia MIT. Esta elección se justifica porque la mayoría de las bibliotecas utilizadas en el *frontend* y en parte del *backend* se encuentran bajo licencias permisivas compatibles con MIT.

La licencia MIT permite que otros usuarios puedan utilizar, modificar y adaptar el código del proyecto, siempre que se mantenga el aviso de copyright correspondiente. Esta característica resulta adecuada para un Trabajo de Fin de Grado, ya que facilita que futuros estudiantes o desarrolladores puedan reutilizar la aplicación como base para proyectos similares, ampliaciones o mejoras.

Además, al tratarse de una licencia sencilla y ampliamente utilizada en proyectos de software libre, favorece la comprensión de las condiciones de uso y reduce la complejidad legal asociada a la distribución del código.

Licencia de la documentación

Para la documentación del proyecto, incluyendo la memoria y los anexos, se considera adecuada una licencia Creative Commons. En concreto, se propone utilizar una licencia CC BY-NC, que permite compartir y adaptar la documentación siempre que se reconozca la autoría original y no se utilice con fines comerciales.

Esta licencia resulta apropiada para un trabajo académico, ya que permite que la documentación pueda ser consultada y utilizada como referencia por otros estudiantes, manteniendo el reconocimiento de la autora y evitando su explotación comercial sin autorización.

Apéndice B

Especificación de Requisitos

B.1. Introducción

En este apéndice se recoge la especificación de requisitos del sistema desarrollado. El objetivo de esta sección es definir de forma clara las funcionalidades que debe proporcionar la aplicación, las restricciones que debe cumplir y los casos de uso que describen la interacción de los distintos tipos de usuario con el sistema.

B.2. Objetivos generales

El objetivo principal del proyecto es desarrollar una aplicación web que facilite la gestión de horarios académicos y reservas de aulas en el entorno universitario, sustituyendo procesos manuales por una solución centralizada, estructurada y adaptada a las necesidades de la planificación docente.

Para alcanzar este objetivo principal, se establecen los siguientes objetivos generales:

- Centralizar la información relacionada con aulas, grados, asignaturas, grupos, docentes, responsables, reservas y calendario académico en un único sistema.
- Permitir la autenticación de usuarios mediante credenciales institucionales, asignando posteriormente los permisos correspondientes en función del rol asociado a cada usuario.

- Gestionar reservas de aulas, diferenciando entre reservas puntuales solicitadas por usuarios autorizados y reservas periódicas asociadas a la docencia de los grados.
- Automatizar la carga de horarios docentes desde hojas de cálculo, transformando la información contenida en dichos ficheros en reservas periódicas dentro de la aplicación.
- Evitar conflictos de ocupación mediante la validación de fechas, horas, aulas disponibles y recursos solicitados antes de crear o modificar una reserva.
- Facilitar la consulta de reservas, horarios de grado, ocupación de aulas y calendario académico mediante interfaces claras y filtros que permitan acceder a la información de forma rápida.
- Gestionar el calendario académico del curso, teniendo en cuenta días lectivos, festivos, cambios de docencia, periodos de exámenes y días destinados a TFG o TFM.
- Controlar el acceso a las distintas funcionalidades mediante un sistema de roles y permisos, diferenciando las acciones disponibles para administradores, gestores y PDI.
- Mejorar la eficiencia en la planificación académica, reduciendo el trabajo manual necesario para crear reservas docentes y disminuyendo la posibilidad de errores en la asignación de aulas.

B.3. Catálogo de requisitos

En este apartado se detallan todos los requisitos que debe satisfacer la aplicación desarrollada. Para ello se realiza la distinción entre los requisitos funcionales, que describen los servicios y comportamientos que se esperan que tenga la aplicación, y los requisitos no funcionales, que establecen las condiciones de calidad, rendimiento, seguridad y uso que deben cumplirse. Estos requisitos sirven como base para el diseño, implementación y validación del sistema.

Requisitos Funcionales

- RF-01. Autenticación de usuarios.
La aplicación deberá permitir que los usuarios accedan al sistema

mediante un mecanismo de autenticación basado en credenciales. El usuario deberá de introducir su usuario y contraseña de la universidad de Burgos para poder iniciar sesión en la aplicación y acceder a las funcionalidades disponibles. Una vez el usuario ha sido autenticado, el sistema identificará el rol asociado al usuario para determinar y restringir las acciones a las que puede y no puede acceder en función a sus permisos.

■ RF-02. Consultar reservas.

La aplicación deberá permitir consultar todas las reservas registradas en el sistema, mostrando la información relevante de cada una de ellas. Entre estos datos se incluirán la fecha y hora de inicio y de fin de la reserva, el título que será el motivo de la reserva, el estado en el que se encuentra, el aula al que se encuentra asociada la reserva y los recursos que se han solicitado como la capacidad, el número de ordenadores,... Además el sistema nos permitirá filtrar estas reservas de manera que solo aparezcan las reservas pendientes o si queremos que aparezcan todas, o filtrar mediante rango de fechas, por motivo o por el correo del responsable de la reserva. De forma que permite a los usuarios autorizados a poder consultar las reservas de forma más rápida y directa.

■ RF-03. Gestionar reservas.

La aplicación deberá permitir la gestión de reservas de aulas, distinguiendo entre reservas puntuales y reservas periódicas. Las reservas puntuales podrán ser solicitadas por los usuarios autorizados que tendrán que rellenar un formulario para ello y asignando un aula, el usuario con permisos de edición, validará los campos de la reserva y modificará el aula asignado si lo cree conveniente. Las reservar periódicas, por el contrario, está dirigida a la organización de las clases impartidas por cada grado y tendrán prioridad sobre las reservas puntuales. Esta funcionalidad es el núcleo de la aplicación.

• RF-03.1. Crear reserva.

El sistema deberá permitir la creación de reservas puntuales a través del formulario correspondiente para ello. Donde se recogerán las mismas características que en el formulario de la solicitud, y en este caso, permitirá también elegir el estado de la reserva por el usuario antes de crearla. La aplicación mostrará las aulas disponibles que estén libres para la fecha y hora introducida y con los recursos solicitados. Una vez creada la reserva, se registra y se actualizan de manera automática la ocupación del aula elegido.

En cuanto a las periódicas, se rellenará el formulario correspondiente seleccionando la asignatura y grupo para la que se quiere impartir, el día de la semana y el rango horario, y el aula libre encontrado.

- RF-03.2. Editar reserva.

La aplicación deberá permitir la modificación de reservas ya existentes, si el usuario dispone de los permisos necesarios para ello. Se pueden modificar todos los campos incluido el estado de la reserva, la fecha y hora y el aula asignado a la reserva, entre otros campos. Antes de guardar los cambios, el sistema deberá de comprobar que los cambios realizados en la reserva no provoquen conflictos con otras reservas existentes o con la ocupación del aula. Si la modificación es correcta, se actualizarán los cambios de manera automática en el sistema.

- RF-03.3. Eliminar reserva.

La aplicación deberá de permitir eliminar reservas existentes en el sistema, siempre que el usuario tenga los permisos adecuados para poder llevar a cabo esta acción. Al eliminar la reserva, el aula quedará disponible para que pueda ser asignado en la fecha y horas correspondientes por otras futuras reservas. De forma que el sistema deberá garantizar que la eliminación de la reserva se refleja de manera correcta en la consulta de la ocupación. Esta operación deberá realizarse de forma controlada para evitar eliminaciones de manera accidental, incluyendo un mensaje de confirmación previo.

- RF-04. Gestión de roles.

La aplicación deberá estar basada en la organización de permisos mediante roles. De forma que cada rol tenga las acciones o permisos limitados evitando que acceda a los de otro rol. Los roles que vamos a encontrar principalmente son el administrador, el gestor editor, el gestor no editor y el PDI (Personal Docente de Investigación) adaptando el comportamiento del sistema en función del perfil o rol, lo que a su vez también reforzará la seguridad y evitará que usuarios sin autorización puedan ejecutar a algunas acciones que puedan ser críticas, lo que hace que sea una funcionalidad muy importante en el proyecto.

- RF-04.1. Crear rol.

La aplicación deberá de permitir al administrador crear nuevos roles cuando lo crea necesario. Cada rol tendrá un nombre que lo

identifique y unos permisos asociados. Dando cierta flexibilidad y evitando que la organización de la aplicación se defina al principio y quede cerrada. Los nuevos roles que se agreguen posteriormente podrán asignarse a los usuarios, haciendo que el sistema pueda evolucionar si la institución así lo requiriese.

- RF-04.2. Editar rol.

La aplicación permitirá realizar modificaciones sobre un rol, tanto modificando su nombre identificativo, como los permisos que tiene asociados. Dando la posibilidad de modificar las acciones que puede realizar un perfil de usuario permitiendo restringir o ampliar las funcionalidades a cada uno de los roles. El sistema deberá de comprobar que estos cambios no produzcan inconsistencia en el sistema dando problemas con el control de accesos.

- RF-04.3. Eliminar rol.

Si un rol deja de ser necesario en el sistema, la aplicación deberá de permitir eliminarlo. Antes de eliminarlo deberá de comprobar el sistema que no se deja ningún usuario sin rol asignado generando problemas de acceso e inconsistencia. En caso de que no sea así, se podrá exigir antes de eliminarlo que se reasignen los usuarios asociados al rol a otro. Esta funcionalidad ayudará a que la estructura de los roles se mantenga limpia y actualizada mientras se utilice la aplicación.

- RF-04.4. Gestionar permisos.

La aplicación deberá de permitir asociar permisos concretos a cada rol definido en el sistema. Estos permisos lo que harán será determinar que operaciones o acciones puede realizar cada usuario sobre las reservas, informes, usuarios,... Permitiendo controlar de forma precisa el acceso a las distintas acciones, implantando un modelo de seguridad basado en los roles.

- RF-06. Gestión de usuarios.

En este caso, como se ha mencionado en el requisito de la autenticación, esta funcionalidad será controlada mediante las credenciales asignadas a cada miembro de la universidad por la misma. De forma, que solo podrán existir usuarios que hayan sido designados por la misma institución. Pudiendo acceder usuarios que no se encuentren en base de datos, o editándolos que en este caso, serían los permisos y el rol asociado.

- RF-06.1. Crear usuario.

El sistema deberá permitir que cuando un usuario no se encuentre

en base de datos podrá iniciar sesión asignándole por defecto el rol de PDI. Aunque el sistema deberá permitir también que el administrador añada a usuarios asignándole el rol que crea necesario.

- RF-06.2. Editar usuario.

Como ya se ha mencionado, el sistema permitirá la edición de los usuarios será respecto a los roles asociado modificando su entrada en base de datos. Debido a que las credenciales no podrán ser modificadas por el sistema, sino que de eso se encargará el sistema de la institución.

- RF-06.3. Eliminar usuario.

El sistema permitirá al usuario con permisos poder eliminar la entrada en base de datos del usuario correspondiente, eliminando así el rol asociado, de forma que cuando entre se le asignará el rol de PDI. Ya que denegar el acceso no dependerá del sistema de este proyecto.

- RF-07. Cargar horarios desde hoja de cálculo.

La aplicación deberá permitir la carga de horarios docentes a partir de una hoja de cálculo, con la estructura específica y unificada con las reglas expuestas. Esta funcionalidad permitirá transformar la información contenida en el fichero en reservas periódicas dentro del sistema, evitando que el gestor tenga que introducir de forma manualmente cada una de las clases. Además, permite automatizar la generación de reservas periódicas asociadas a la docencia a través del analizador léxico, y las generará teniendo en cuenta el calendario académico.

- RF-08. Gestión del calendario académico.

La aplicación deberá permitir la gestión del calendario académico utilizado para la generación y validación de reservas periódicas. Este calendario será la base para determinar los días lectivos, los periodos de docencia de cada cuatrimestre, los cambios de docencia, los días festivos, los periodos de exámenes y los periodos de solamente TFG. De esta forma, el sistema podrá crear las reservas docentes de manera coherente con la planificación oficial de la universidad, evitando generar ocupaciones en fechas en las que no exista actividad académica.

- RF-08.1. Crear calendario académico.

La aplicación deberá permitir al usuario autorizado crear un calendario académico. Para ello, se deberán introducir los datos principales, como el curso académico, la fecha de inicio y de fin

del curso, los periodos de docencia, es decir, las fechas de inicio y de fin de ambos cuatrimestres, y los días lectivos.

- RF-08.2. Editar calendario académico.
El sistema debe permitir modificar el calendario existente, siempre que el usuario tenga los permisos necesarios. Se podrá modificar el tipo de día de uno en uno, o mediante rangos de fechas seleccionadas, permitiendo seleccionar el tipo de día al que quieren actualizarlo.

Requisitos No Funcionales

1. RNF-01. La aplicación debe ser segura.

La aplicación debe proteger tanto la información como las operaciones que se realizan en ella frente a accesos no autorizados. Para lograrlo, cuenta con la autenticación de usuarios, el control de los permisos definidos para cada rol y la validación frente a acciones sensibles. También se deben de proteger de manera adecuada las credenciales de acceso a la aplicación. Al ser una aplicación con gestión de reservas, planificación académica y autenticación de usuarios la seguridad es especialmente relevante.

2. RNF-02. La aplicación debe tener un uso sencillo.

La interfaz de la aplicación deberá de ser clara, intuitiva y fácil de utilizar para los distintos tipos de usuario. El sistema por tanto, debe de presentar la información de forma ordenada, y tratando de reducir lo máximo posible la complejidad de las acciones más frecuentes, como la consulta de reservas o la asignación de aulas a las reservas. Este requisitos es importante para favorecer la adaptación de la herramienta al personal universitario, reduciendo errores humanos y mejorando la eficiencia del trabajo diario.

3. RNF-03. Escalabilidad.

La aplicación debe estar preparada para crecer tanto en el volumen de datos y de usuarios, como para añadir nuevas funcionalidades sin tener que rediseñar todo. Es decir, su arquitectura tiene que permitir facilitar la incorporación de nuevos módulos, restricciones o informes adicionales en un futuro. Permitiendo a su vez soportar un aumento de manera progresiva del número de reservas, aulas, grados o titulaciones, o cursos y horarios académicos. Este requisito es muy importante para su evolución en el futuro, debido a que un proyecto escalable tiene un mayor valor técnico y una mayor vida útil ante su crecimiento.

4. RNF-04. Rendimiento.

La aplicación debe de ofrecer unos tiempos de espera que sean adecuados especialmente en las operaciones más comunes del sistema, como la visualización de las reservas y la visualización de la ocupación de las aulas. Aunque las operaciones de creación o modificación no deben producir mucha demora para el usuario. Este requisito es muy importante para garantizar que la experiencia del usuario utilizando esta herramienta sea más satisfactoria y ayude en la gestión académica.

5. RNF-05. Disponibilidad.

La aplicación debe de estar disponible para su uso cuando los usuarios lo necesiten o requieran. Lo que implica reducir lo máximo posible el número de fallos que puedan ocurrir, principalmente en el acceso al sistema o durante la consulta de información. De esta forma, el sistema tiene que mantener coherencia en los datos incluso ante errores inesperados. Esto es muy importante sobre todo durante los periodos de planificación académica o de gestión masiva de reservas. Si el sistema es fiable y estable aumenta la confianza de los usuarios en la herramienta.

6. RNF-06. Interoperabilidad o compatibilidad.

La aplicación debe de ser compatible con los principales navegadores y funcionar de manera correcta cuando se acceda desde diferentes dispositivos. Además, debe de integrarse adecuadamente con la base de datos y con los diferentes formatos de entrada o salida utilizados para importar o exportar información. Lo que garantiza que la herramienta puede utilizarse sin depender de que se pueda ejecutar en un único entorno, y facilitando el intercambio de datos con otros sistemas. La compatibilidad es esencial en una aplicación web que está destinada a diferentes perfiles de usuarios.

7. RNF-07. Mantenimiento.

La aplicación debe ser desarrollada de forma que tanto su mantenimiento como su evolución resulten sencillos. Para poder conseguirlo, es recomendable seguir una estructura modular, con la separación entre la interfaz, la lógica de negocio y el acceso a datos. También es importante que el código sea claro, esté documentado y sea fácil de modificar. Este requisito nos permite poder corregir errores, adaptar funcionalidades y ampliar el sistema con un menor coste técnico.

B.4. Especificación de casos de uso

Diagrama general de casos de uso

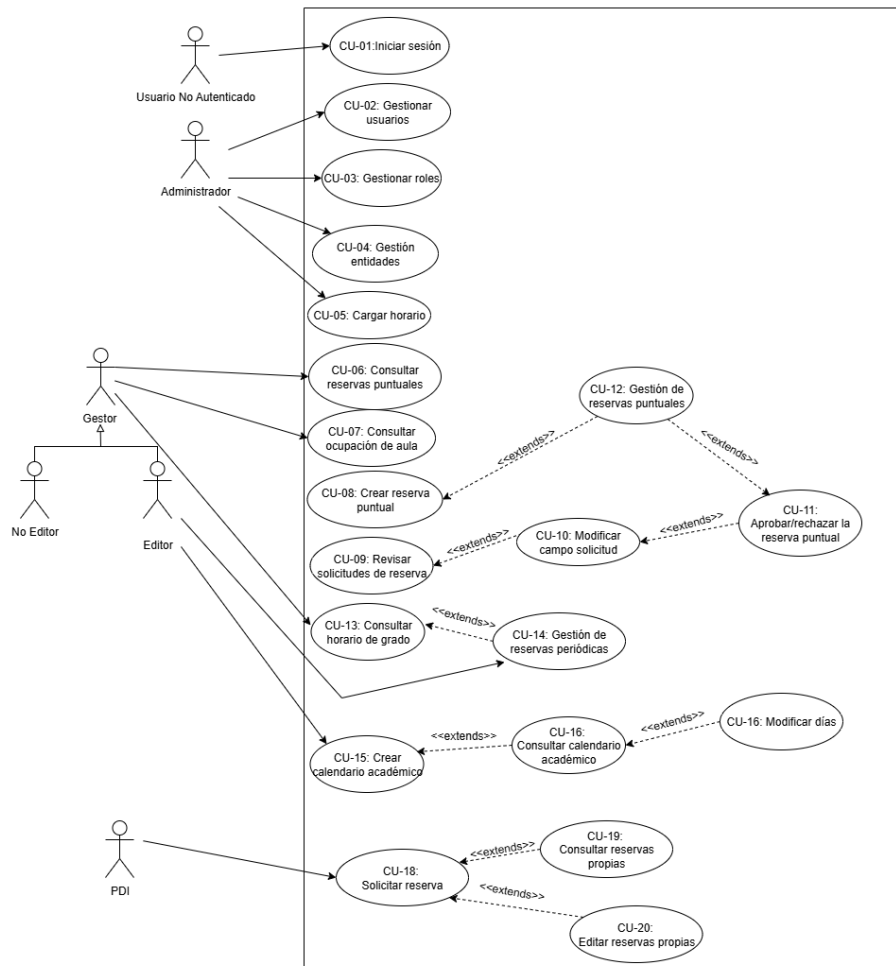


Figura B.1: Diagrama general de casos de uso

Definición de los casos de uso

A continuación, se describen los casos de uso identificados para el sistema. Cada caso de uso se presenta mediante una tabla en la que se recoge su identificador, una breve descripción, las precondiciones necesarias, el flujo principal de ejecución, las postcondiciones, las posibles excepciones y la importancia.

CU-01	Inicio de sesión
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	RF-01
Descripción	Un usuario no autenticado, introduce las credenciales (correo y contraseña), y se mandan a la API del moodle de la universidad. Si son credenciales válidas , se le permite acceder a la aplicación rediriéndole en función de su rol, si las credenciales son inválidas se mostrará el mensaje de error y no se le permitirá el acceso.
Precondición	El usuario está registrado y su cuenta se encuentra activa
Acciones	1. Introducir correo 2. Introducir contraseña 3. El sistema valida el formato 4. El sistema verifica las credenciales 5. El sistema comprueba el rol del usuario 6. Se crea la sesión 7. Se redirige a la pantalla de gestión de reservas
Postcondición	En caso de éxito en el inicio de sesión, el usuario tendrá una sesión activa y el usuario queda en su panel correspondiente. En caso de fallo no se crea la sesión, se incrementa el contador de intentos fallidos y se muestra el mensaje de error.
Excepciones	■ Ex1 Credenciales incorrectas → mensaje genérico y sumar el intento fallido. ■ Ex2 Usuario inactivo o bloqueado → informar del fallo
Importancia	Alta

Tabla B.1: CU-01 Inicio de sesión

CU-02	Gestionar usuarios
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
RF-06 Descripción	El administrador puede crear, modificar y eliminar entradas de usuario del sistema, se corresponderá al rol del usuario, ya que las credenciales no las comprueba la base de datos propia. Incluye búsqueda y listado de los usuarios.
Precondición	El administrador tiene que estar autenticado, tiene que disponer de los permisos para la gestión de usuarios y la base de datos tiene que estar disponible.
Acciones	1. El administrador accede a Administración -> Gestión de Usuarios 2. Visualiza el listado de usuarios (nombre, email, rol), y las opciones (Añadir, editar y eliminar) 3. El administrador pulsa <i>Añadir</i> 3.1. El sistema muestra un formulario con todos los campos correspondientes a los datos necesarios 3.2. El administrador completa estos campos 3.3. El sistema valida el formato del email y que este sea único 3.4. El sistema muestra la confirmación 4. El administrador selecciona al usuario y pulsa <i>Editar</i> 4.1. El sistema muestra los datos actuales 4.2. El administrador modifica los campos 4.3. El sistema valida los campos modificados y guarda los cambios 5. El administrador pulsa <i>Eliminar</i> sobre un usuario 5.1. El sistema solicita confirmación y muestra cambios asociados 5.2. El administrador confirma 5.3. El sistema ejecuta el borrado, y elimina los datos asociados
Postcondición	En caso de éxito el usuario se crea, edita o elimina según la acción realizada. En caso de fallo no se realizan los cambios y se muestra el error
Excepciones	■ Ex1 Email duplicado → Impedir la creación o edición y mostrar el error ■ Ex2 Formato de datos inválidos → mostrar los errores de validación
Importancia	Alta

Tabla B.2: CU-02 Gestionar usuarios

CU-03	Gestionar roles
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
RF-04 Descripción	El administrador asigna, cambio o quitar roles a usuarios para determinar los permisos de acceso.
Precondición	El administrador tiene que estar autenticado, que el usuario sobre el que se actúa exista, y que los roles estén disponibles. Podrá gestionar los permisos de cada rol
Acciones	1. El administrador accede a <i>Administración</i> -> <i>Gestión de roles</i> y selecciona un rol. 2. El sistema muestra los nombres de los roles actuales. 3. El administrador pulsa <i>Añadir rol</i>. 3.1. El usuario rellena el nombre del rol que quiere crear. 3.2. El administrador selecciona los permisos que quiere asignar a ese rol. 3.3. El sistema asigna los permisos al rol. 4. El administrador pulsa <i>Editar roles</i>. 4.1. El sistema muestra el nombre del rol. 4.2. El sistema muestra marcados los permisos de los que dispone. 4.3. El administrador actualiza el listado de permisos y confirma. 4.1. El sistema solicita confirmación y muestra las consecuencias 4.2. El administrador confirma. 4.3. El sistema elimina el rol.
Postcondición	Se actualizan los roles del usuario, pero si falla no hay cambios y se muestra el mensaje de error
Excepciones	■ Ex1. Usuario sin permisos suficientes → el sistema impedirá el acceso a la gestión de roles y mostrará un mensaje de error indicando que no dispone de autorización. ■ Ex2. Nombre de rol duplicado → el sistema no guardará los cambios y mostrará un mensaje indicando que ya existe un rol con ese nombre
Importancia	Alta

Tabla B.3: CU-03 Gestionar roles

CU-04	Gestión de entidades
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	RF-05
Descripción	El administrador gestiona las entidades principales utilizadas por la aplicación, como aulas, grados, asignaturas, docentes, responsables e imparte. Esta gestión permite mantener actualizada la información base sobre la que posteriormente se crearán horarios, reservas puntuales y reservas periódicas.
Precondición	El administrador debe estar autenticado, disponer de permisos de administración y la base de datos debe estar disponible.
Acciones	1. El administrador accede al apartado de <i>Administración</i>. 2. El sistema muestra las entidades disponibles para gestionar. 3. El administrador selecciona la entidad que desea consultar o modificar. 4. El sistema muestra el listado de registros existentes. 5. El administrador puede crear, editar o eliminar registros. 6. El sistema valida los datos introducidos. 7. El sistema guarda los cambios realizados.
Postcondición	La entidad seleccionada queda creada, modificada o eliminada correctamente. En caso de fallo, no se aplican los cambios y se muestra un mensaje de error.
Excepciones	■ Ex1: Datos obligatorios incompletos → impedir la operación y mostrar los errores de validación. ■ Ex2: Registro duplicado → impedir la creación o edición y mostrar un mensaje de error.
Importancia	Alta

Tabla B.4: CU-04 Gestión de entidades

CU-05	Cargar horario
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El administrador o gestor carga en el sistema un horario de otro año
Precondición	Que sea un usuario con permisos y que la base de datos se encuentre disponible para recoger los datos cargados
Acciones	1. El usuario accede al apartado de <i>Horarios</i> 2. El usuario selecciona el grado correspondiente 3. El usuario pulsa en <i>Cargar horario</i> 4. El sistema muestra la opción de cargar un archivo del sistema (csv o Excel) 5. El usuario selecciona el archivo a cargar 6. El sistema convierte los datos del excel a datos SQL que pueda recoger la base de datos 7. El sistema muestra el calendario de exámenes cargado
Postcondición	El calendario se encuentra correctamente cargado en la base de datos, se muestra correctamente en la aplicación y si hay más horarios cargados para el mismo grado de diferentes cursos permite seleccionar el horario en función del curso
Excepciones	■ Ex1: Error al pasar los datos de Excel a SQL → mostrar mensaje de error y cancelar la carga de datos ■ Ex2: Subir un archivo con extensión no válida → mostrar el mensaje y pedir la corrección del archivo
Importancia	Alta

Tabla B.5: CU-05 Cargar horario

CU-06	Consultar reservas puntuales
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	RF-02
Descripción	El gestor o administrador consulta las reservas que existen, y puede aplicar filtros sobre ellas y visualizar los detalles de cada una
Precondición	Que sea un usuario autenticado y que tenga el rol de gestor o de administrador. Que la base de datos esté disponible y que existan reservas registradas
Acciones	1. El usuario accede al módulo <i>Gestionar reservas puntuales</i>. 2. El sistema muestra las reservas que existen en la base de datos 3. El usuario puede aplicar filtros 4. El sistema actualiza la lista con los resultados filtrados 5. El usuario selecciona una reserva específica para ver más información 6. El sistema muestra los detalles de la reserva
Postcondición	Se muestra al usuario la información de las reservas con la posibilidad de realizar filtros. No se permite la modificación de datos.
Excepciones	■ Ex1: No existen reservas que cumplan los criterios → mostrar mensaje “no hay resultados”. ■ Ex2: Error en la conexión a la base de datos → cancelar operación y notificar fallo. ■ Ex3: Filtro inválido o mal formado → restablecer búsqueda y mostrar aviso.
Importancia	Media-Alta

Tabla B.6: CU-06 Consultar reservas puntuales

CU-07	Consultar ocupación de aula
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	RF-04
Descripción	El administrador o el gestor consulta la ocupación del aula para una fecha específica o semanal (solo aparecen las reservas periódicas). El sistema muestra un horario con las horas que se encuentra ocupada o disponible al aula, en función del horario y de las reservas puntuales si así se requiere.
Precondición	Que sea un usuario autenticado con el rol de gestor o de administrador, que los horarios y las reservas se encuentren cargadas en el sistema y que la base de datos esté disponible.
Acciones	1. El usuario accede al apartado de <i>Consultar ocupación de aula</i>. 2. El sistema muestra un menú con filtros de aula, fecha o semanal. 3. El usuario pulsa el aula y el periodo y pulsa <i>Buscar</i> 4. El sistema recoge los datos asociado a la búsqueda realizada por el usuario 5. El sistema genera una vista con las horas ocupadas y disponibles.
Postcondición	El sistema muestra la ocupación del aula seleccionada mostrándose claramente las horas disponibles y las ocupadas.
Excepciones	■ Ex1: Que el aula no exista o no se haya seleccionado correctamente →mostrar mensaje de error. ■ Ex2: Que no existan datos para la fecha o periodo seleccionado →mostrar que no hay existen reservas para ese aula y no hay datos que mostrar ■ Ex3: Que la fecha introducida sea inválida →Mostrar mensaje de error ■ Ex4: Que haya un error al recoger los datos de la consulta →notificar el fallo
Importancia	Media-Alta

Tabla B.7: CU-07 Consultar ocupación de aula

CU-08	Crear reserva puntual
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El administrador o gestor crea una reserva puntual de un aula en una fecha y un horario correcto, validando la disponibilidad.
Precondición	El usuario autenticado tiene que tener los permisos, que exista el aula y esté disponible y la base de datos y las restricciones se encuentren disponibles también.
Acciones	1. El usuario accede al apartado de <i>Gestión de reservas puntuales</i> 2. El usuario pulsa en <i>Crear nueva reserva</i> 3. El sistema muestra un formulario con los campos necesarios 4. El usuario rellena los campos con los datos correspondientes y pulsa <i>Crear reserva</i> 5. El sistema valida que se cumplan todas las restricciones 6. Si hay errores, el sistema muestra los mensajes correspondientes 7. El usuario modifica los datos erróneos 8. El sistema crea la reserva
Postcondición	
Excepciones	■ Ex1: Viola alguna restricción → Impide la creación y muestra el mensaje de error ■ Ex2: Campos obligatorios están incompletos → se muestra el mensaje
Importancia	Alta

Tabla B.8: CU-08 Crear reserva puntual

CU-09	Revisar solicitudes de reserva
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El gestor o administrador revisa el estado de las solicitudes de reserva
Precondición	El usuario tiene que estar autenticado y ha de tener los permisos, deben de existir solicitudes de reservas y la base de datos y las restricciones han de estar disponibles
Acciones	1. El usuario accede al apartado <i>Revisar solicitudes de reservas</i> 2. El sistema muestra la lista de todas las solicitudes de reservas cuyo estado es Pendiente 3. El usuario elige como filtrar si lo necesita 4. El sistema muestra la lista según los filtros 5. El usuario selecciona una solicitud 6. El sistema le muestra los detalles
Postcondición	Se actualiza o muestra correctamente el estado de las solicitudes
Excepciones	■ Ex1: No existen solicitudes para los filtros aplicados → mostrar mensaje "Sin resultados" ■ Ex2: Error al recoger los datos → mostrar aviso de error y no realizar la acción
Importancia	Media-Alta

Tabla B.9: CU-09 Revisar solicitudes de reserva

CU-10	Modificar campo solicitud
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El gestor o administrador decide modificar uno de los campos de la solicitud de reserva, y el sistema avisará al responsable de la reserva del cambio realizado
Precondición	El usuario tiene que estar autenticado y ha de tener los permisos, deben de existir solicitudes de reservas y la base de datos y las restricciones han de estar disponibles
Acciones	1. El usuario accede al apartado <i>Revisar solicitudes de reservas</i> 2. El sistema muestra la lista de todas las solicitudes de reservas cuyo estado es Pendiente 3. El usuario elige como filtrar si lo necesita 4. El sistema muestra la lista según los filtros 5. El usuario selecciona una solicitud 6. El sistema le muestra los detalles 7. El usuario selecciona el campo que quiere modificar y cambia el dato del mismo 8. El sistema notificará al responsable de la solicitud de los cambios realizados en la misma
Postcondición	Se muestra la modificación realizada en la solicitud, y se notifica al responsable de la modificación
Excepciones	■ Ex1: No existen solicitudes para los filtros aplicados → mostrar mensaje "Sin resultados" ■ Ex2: Error al recoger los datos → mostrar aviso de error y no realizar la acción ■ Ex:3 Violación de restricción → los cambios realizados violarían alguna restricción en caso de ser aprobados ■ Ex:4 No llega notificación al responsable → se solicitará volver a notificarlo
Importancia	Media-Alta

Tabla B.10: CU-10 Modificar campo solicitud

CU-11	Aprobar/rechazar reserva puntual
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El administrador o gestor revisa las reservas puntuales solicitadas por los usuarios y decide su aprobación o rechazo según la disponibilidad.
Precondición	El usuario debe de tener permisos para poder aprobar o rechazar las solicitudes, tienen que existir reservas pendientes en el sistema, y la base de datos tiene que estar disponible
Acciones	1. El usuario accede al apartado de <i>Gestión de reservas puntuales</i> 2. El sistema muestra la lista de reservas cuyo estado es Pendiente 3. El usuario selecciona la reserva 4. El sistema le muestra los detalles (aula, fecha, motivo, responsable, periodo, capacidad) 5. El usuario pulsa <i>Aprobar</i> 5.1. El sistema verifica la validación de las restricciones 5.2. Si es válida, se actualiza el estado de la reserva a Aprobada 6. El usuario pulsa <i>Rechazar</i> 6.1. El sistema le da la opción al usuario de notificar el motivo del rechazo 6.2. El usuario introduce el motivo si quiere y confirma 6.3. El sistema cambia el estado de la reserva a Rechazada
Postcondición	Se actualiza el estado de la reserva a Aprobada o Rechazada en función de la acción realizada por el usuario
Excepciones	■ Ex1: La reserva ya ha sido procesada o modificada → impedir la acción y mostrar el aviso ■ Ex2: El aula que estaba asignada se encuentre ocupada tras aprobarla → bloquear acción y sugerir otras aulas disponibles
Importancia	Alta

Tabla B.11: CU-11 Aprobar rechazar reserva puntual

CU-12	Gestión de reservas puntuales
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El gestor o administrador puede crear, modificar o eliminar reservas puntuales existentes, validando que no se violen las restricciones
Precondición	El usuario debe tener los permisos de gestionar las reservas, que estas se encuentren registradas y la base de datos y las validaciones se encuentren disponibles
Acciones	1. El usuario accede al apartado de <i>Gestión de Reservas puntuales</i> 2. El sistema muestra la lista de reservas puntuales existentes permitiendo filtrar por aula, o fechas 3. El usuario selecciona una reserva y pulsa <i>Editar</i> 3.1. El sistema muestra los datos actuales de la reserva 3.2. El usuario modifica los campos que crea correspondientes y pulsa <i>Confirmar</i> 3.3. El sistema valida las restricciones y que los campos no estén vacíos 3.4. El sistema actualiza los datos de la reserva 4. El usuario selecciona una reserva y pulsa <i>Eliminar</i> 4.1. El sistema solicita confirmación 4.2. El usuario confirma la eliminación 4.3. El sistema elimina la reserva
Postcondición	La reserva puntual queda modificada o eliminada de manera correcta, actualizando todos los datos correspondientes
Excepciones	■ Ex1: Reserva inexistente o ya eliminada → impedir la acción y mostrar el error ■ Ex2: Violación de restricciones → impedir la acción y mostrar el error solicitando la corrección de los datos correspondientes
Importancia	Alta

Tabla B.12: CU-12 Gestión de reservas puntuales

CU-13	Consultar horario de grado
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El gestor o administrador puede consultar el horario de grado. El sistema muestra las clases planificadas con su aula y hora.
Precondición	El usuario autenticado tiene permisos , los horarios de los grados deben de estar cargados en el sistema y la base de datos tiene que estar disponible
Acciones	1. El usuario accede al módulo <i>Consultar horarios de grado</i> 2. El sistema muestra los grados por los que filtrar. 3. El usuario selecciona el grado del que quiere 4. El sistema muestra filtros sobre los que aplicar 5. El usuario selecciona si quiere filtrar o no y pulsa <i>Buscar</i> 6. El sistema recoge los datos asociado al grado seleccionado y los filtros
Postcondición	Se muestra el horario con los filtros y grado aplicados, con la informaciónn actualizada y recogiendo los datos de la base de datos
Excepciones	■ Ex1: Error de conexión a la base de datos →mostrar mensaje de error. ■ Ex2: Que no existan horarios para los filtros seleccionados →mostrar mensaje de que no existen resultados
Importancia	Media

Tabla B.13: CU-13 Consultar horario de grado

CU-14	Gestión de reservas periódicas
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	RF-6.2
Descripción	El gestor o administrador crea, edita y elimina las reservas durante el periodo especificado, con la frecuencia especificada.
Precondición	Que sea un usuario autenticado con los permisos de reservas, los aulas y horarios disponibles estén cargados, y la base de datos se encuentre disponible.
Acciones	1. El usuario accede a <i>Horarios</i>. 2. El sistema muestra el horario para el curso y semestre seleccionados. 3. El usuario pulsa <i>Crear reserva periódica</i>. 3.1. El sistema muestra un formulario con el grado, el curso y el semestre preseleccionados del horario en el que se encontraba. 3.2. El usuario modifica los datos precargados si lo cree necesario, añade la asignatura y el grupo para el que va a realizarse la reserva, y la hora de inicio y de fin y el día de la semana que va a ejecutarse, además de seleccionar un aula libre. Y confirma la reserva. 4. El usuario selecciona una reserva periódica 4.1. El sistema permite cambiar los campos relativos al día y hora, y el aula 4.2. El usuario aplica cambios y confirma 5. El sistema valida las restricciones, y actualiza o añade la reserva 6. El usuario selecciona una reserva periódica y pulsa <i>Eliminar</i> 6.1. El usuario confirma 6.2. El sistema elimina todos los datos correspondientes a esa reserva
Postcondición	La reserva periódica queda creada, actualizada o eliminada según la acción
Excepciones	■ Ex1: Viola restricciones → impide la creación o edición y muestra el error
Importancia	Alta

Tabla B.14: CU-14 Gestión de reservas periódicas

CU-15	Crear calendario académico
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	RF-08
Descripción	El gestor editor crea un nuevo calendario académico para un curso concreto, indicando los periodos docentes, días lectivos, festivos.
Precondición	El usuario debe estar autenticado, tener permisos de edición sobre el calendario académico y la base de datos debe estar disponible.
Acciones	1. El gestor editor accede al módulo <i>Calendarios</i>. 2. El usuario pulsa en <i>Cargar nuevo curso</i>. 3. El sistema muestra un formulario con el curso académico y las fechas principales. 4. El usuario introduce los periodos docentes y los días festivos. 5. El sistema valida que las fechas sean correctas y que no existan solapamientos incoherentes. 6. El sistema crea el calendario académico.
Postcondición	El calendario académico queda creado y disponible para su consulta, modificación y uso durante la generación de reservas periódicas.
Excepciones	■ Ex1: Ya existe un calendario para el curso seleccionado → impedir la creación y mostrar un mensaje de aviso. ■ Ex2: Fechas incoherentes o solapadas → impedir la creación hasta que se corrijan. ■ Ex3: Error al guardar el calendario → cancelar la operación y mostrar mensaje de error.
Importancia	Alta

Tabla B.15: CU-15 Crear calendario académico

CU-16	Consultar calendario académico
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El gestor o administrador puede consultar el calendario académico de la politécnica. El sistema mostrará el calendario indicando los días lectivos correspondientes a ambos semestres, los días correspondientes a exámenes de convocatoria, a TFG, los días festivos y los días que se produce un cambio de docencia.
Precondición	El usuario autenticado tiene permisos, los tipos de día deben de estar cargados en el sistema y la base de datos tiene que estar disponible
Acciones	1. El usuario accede al módulo <i>Calendarios</i> 2. El sistema muestra una lista con los cursos que tienen calendario cargado. 3. El usuario selecciona el curso para el que quiere ver el calendario. 4. El sistema muestra el calendario académico con diferentes colores en función del tipo de día.
Postcondición	Se muestra el calendario académico coloreando cada tipo de día con un color diferente (rojo: festivo, morado: cambio de docencia, amarillo: lectivos primer semestre, verde: exámenes convocatoria, azul: lectivos segundo semestre y naranja: días de TFG y TFM), recogiendo los datos de la base de datos
Excepciones	■ Ex1: Error de conexión a la base de datos →mostrar mensaje de error.
Importancia	Media

Tabla B.16: CU-16 Consultar calendario académico

CU-17	Modificar día
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El gestor o administrador modifica el tipo de día en el calendario académico. El sistema cambiará el tipo de día que había anteriormente por el seleccionado, y actualizará la información en la base de datos.
Precondición	El usuario autenticado tenga los permisos, que el calendario se encuentre cargado y que la base de datos esté disponible
Acciones	1. El usuario accede al módulo <i>Calendarios</i> 2. El sistema muestra el listado de los calendarios que se encuentran cargados en la base de datos mostrando el curso académico al que corresponde cada uno 3. El usuario selecciona un curso académico 4. El sistema muestra el calendario del curso correspondiente 5. El usuario selecciona uno o varios días 6. El sistema le mostrará las distintas posibilidades de tipo de día que puede seleccionar 7. El usuario pulsa sobre el tipo de día 8. El sistema cambia el tipo de día, y por tanto se actualiza el color.
Postcondición	El calendario queda actualizado con las modificaciones de los días realizados
Excepciones	■ Ex1: Que se modifique un sábado o domingo y se ponga como día lectivo o cambio de docencia → mostrar error y solicitar nuevo tipo ■ Ex2: Que viole alguna otra restricción → Mostrar el error y no permitir el cambio
Importancia	Alta

Tabla B.17: CU-17 Modificar día

CU-18	Solicitar reserva
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	RF-03, RF-03.1
Descripción	El PDI solicita una reserva puntual de aula indicando los datos necesarios, como fecha, franja horaria, motivo y recursos requeridos. El sistema registra la solicitud con estado pendiente para que posteriormente sea revisada por un gestor.
Precondición	El PDI debe estar autenticado, la base de datos debe estar disponible y deben existir aulas registradas en el sistema.
Acciones	1. El PDI pulsa el botón <i>Crear Reserva</i>. 2. El sistema muestra el formulario de solicitud. 3. El PDI introduce la fecha, hora de inicio, hora de fin, motivo y recursos necesarios. 4. El sistema valida que los campos obligatorios estén completos. 5. El sistema busca un aula compatible con los datos introducidos. 6. El PDI confirma la solicitud. 7. El sistema registra la reserva con estado <i>Pendiente</i>.
Postcondición	La solicitud de reserva queda registrada con estado pendiente y asociada al usuario que la ha realizado.
Excepciones	■ Ex1: No existe ningún aula disponible → impedir el envío y solicitar modificar los datos. ■ Ex2: Campos obligatorios incompletos → mostrar los errores de validación. ■ Ex3: Fecha u hora no válida → impedir la creación de la solicitud.
Importancia	Alta

Tabla B.18: CU-18 Solicitar reserva

CU-19	Consultar reservas propias
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	RF-02
Descripción	El PDI consulta las reservas y solicitudes que ha realizado, pudiendo comprobar su estado y visualizar la información asociada a cada una de ellas.
Precondición	El PDI debe estar autenticado y deben existir reservas o solicitudes asociadas a su usuario.
Acciones	1. El sistema muestra el listado de reservas propias. 2. El PDI puede filtrar por fecha, estado o motivo. 3. El sistema actualiza el listado según los filtros aplicados. 4. El PDI puede ver los campos de la reserva de forma compacta en la lista de reservas.
Postcondición	El PDI visualiza sus reservas y el estado en el que se encuentran, sin modificar los datos.
Excepciones	■ Ex1: No existen reservas propias → mostrar un mensaje indicando que no hay resultados. ■ Ex2: Error al recuperar los datos → mostrar mensaje de error y cancelar la consulta.
Importancia	Media

Tabla B.19: CU-19 Consultar reservas propias

CU-20	Editar reservas propias
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	RF-03.2
Descripción	El PDI modifica los datos de una reserva propia siempre que esta todavía no haya sido procesada por un gestor, es decir mientras su estado sea Pendiente. Esta funcionalidad permite corregir errores o adaptar la solicitud antes de su aprobación o rechazo.
Precondición	El PDI debe estar autenticado, la reserva debe pertenecerle y debe encontrarse en estado Pendiente.
Acciones	1. El sistema muestra el listado de reservas propias. 2. El PDI selecciona una reserva editable. 3. El sistema muestra los datos actuales de la reserva. 4. El PDI modifica los campos necesarios. 5. El sistema valida los nuevos datos introducidos. 6. El sistema actualiza la solicitud de reserva.
Postcondición	La reserva propia queda actualizada correctamente. Si falla la operación, se mantienen los datos anteriores.
Excepciones	■ Ex1: La reserva ya ha sido aprobada o rechazada → impedir la modificación. ■ Ex2: La reserva no pertenece al usuario → impedir el acceso. ■ Ex3: Los nuevos datos incumplen restricciones → no guardar los cambios y mostrar el error.
Importancia	Media-Alta

Tabla B.20: CU-20 Editar reservas propias

CU-22	Gestión solicitud de reservas
Versión	1.0
Autor	Rebeca Cuesta Estépar
Requisitos asociados	
Descripción	El profesorado (PDI) puede solicitar reservas puntuales de aulas, y consultar el estado de sus solicitudes para ver si han sido procesadas, y en el caso de que sí, ver si han sido aprobadas o rechazadas
Precondición	El usuario debe de estar autenticado, el aula solicitada debe de estar disponible y la base de datos y las restricciones disponibles
Acciones	1. El PDI accede al apartado <i>Mis solicitudes de reserva</i> 2. El sistema le muestra las opciones disponibles que son solicitar una nueva reserva o consultar el estado de las solicitudes 3. El PDI pulsa en <i>Nueva solicitud</i> 3.1. El sistema muestra un formulario con los campos: fecha, hora inicio y fin, motivo, capacidad 3.2. El PDI completa los campos y envía la solicitud 3.3. El sistema añade la primera aula que cumpla con las restricciones 3.4. El sistema registra la solicitud con el estado <i>Pendiente</i> 4. El PDI selecciona la opción de <i>Consultar estado</i> 4.1. El sistema muestra un listado con las solicitudes del usuario 4.2. El usuario puede filtrar por fechas o estado 4.3. El sistema actualiza la lista de acuerdo a los filtros
Postcondición	La solicitud de reserva queda creada con estado <i>Pendiente</i> , y el usuario puede consultar el estado de las solicitudes realizadas
Excepciones	■ Ex1: Ningún aula disponible para los datos introducidos → impedir envío y solicitar cambiar datos ■ Ex2: Solicitud duplicada → informar y evitar creación
Importancia	Media-Alta

Tabla B.21: CU-22 Gestión solicitud de reservas

Apéndice C

Especificación de diseño

C.1. Introducción

En este apéndice se recoge la especificación de diseño del sistema desarrollado. A partir de los requisitos funcionales y no funcionales definidos previamente, se describen las decisiones de diseño tomadas para estructurar la aplicación y representar la información que debe gestionar.

El diseño de la aplicación se ha centrado especialmente en la organización de los datos, ya que el sistema trabaja con información relacionada con aulas, grados, asignaturas, docentes, grupos, reservas puntuales, reservas periódicas y calendario académico. Por este motivo, se ha considerado necesario representar primero el modelo conceptual mediante un diagrama entidad-relación y, posteriormente, su transformación al modelo relacional utilizado en la base de datos.

C.2. Diseño de datos

El diseño de datos define la estructura de la información que será almacenada y gestionada por la aplicación. En este proyecto, la base de datos debe permitir almacenar la información académica necesaria para la carga de horarios, la creación de reservas, la consulta de ocupación de aulas y la gestión del calendario académico.

Para representar este diseño se han utilizado dos modelos complementarios: el diagrama entidad-relación y el modelo relacional. El primero permite describir de forma conceptual las entidades principales del sistema y las relaciones existentes entre ellas, mientras que el segundo muestra cómo se

transforma dicho diseño conceptual en tablas, atributos, claves primarias y claves ajenas.

Diagrama entidad-relación

El diagrama entidad-relación representa el modelo conceptual de la base de datos. Este tipo de diagrama permite identificar las entidades principales del sistema, sus atributos más relevantes y las relaciones existentes entre ellas, sin entrar todavía en detalles concretos de implementación [4]. Podemos ver su representación en la Figura C.1

En el caso de esta aplicación, el diagrama entidad-relación permite representar elementos como las aulas, los grados, las asignaturas, los grupos, los docentes, las reservas y los días del calendario académico. Además, recoge relaciones importantes como la asociación entre reservas y aulas, la relación entre asignaturas y grupos, la vinculación entre docentes y asignaturas, y la clasificación de las reservas en puntuales o periódicas.

También se representa la especialización de algunos conceptos. Por ejemplo, una reserva puede ser puntual o periódica, y un día del calendario académico puede clasificarse como lectivo, festivo, día de examen, día de cambio de docencia o periodo asociado a TFG/TFM. De esta forma, el diagrama permite obtener una visión general de la estructura lógica del sistema antes de su transformación al modelo relacional.

Modelo relacional

El modelo relacional representa la transformación del diagrama entidad-relación en una estructura preparada para su implementación en una base de datos relacional. En este modelo, cada entidad o relación relevante se convierte en una tabla, cuyos atributos se representan mediante columnas [16].

Además, el modelo relacional permite definir las claves primarias, encargadas de identificar de forma única cada registro, y las claves ajenas, que permiten establecer relaciones entre las distintas tablas. Gracias a estas claves se garantiza la integridad referencial de la información, evitando inconsistencias entre elementos relacionados, como reservas asociadas a aulas inexistentes o grupos vinculados a asignaturas no registradas.

Este modelo representado en la Figura C.2 constituye la base sobre la que se ha construido la persistencia de datos de la aplicación.

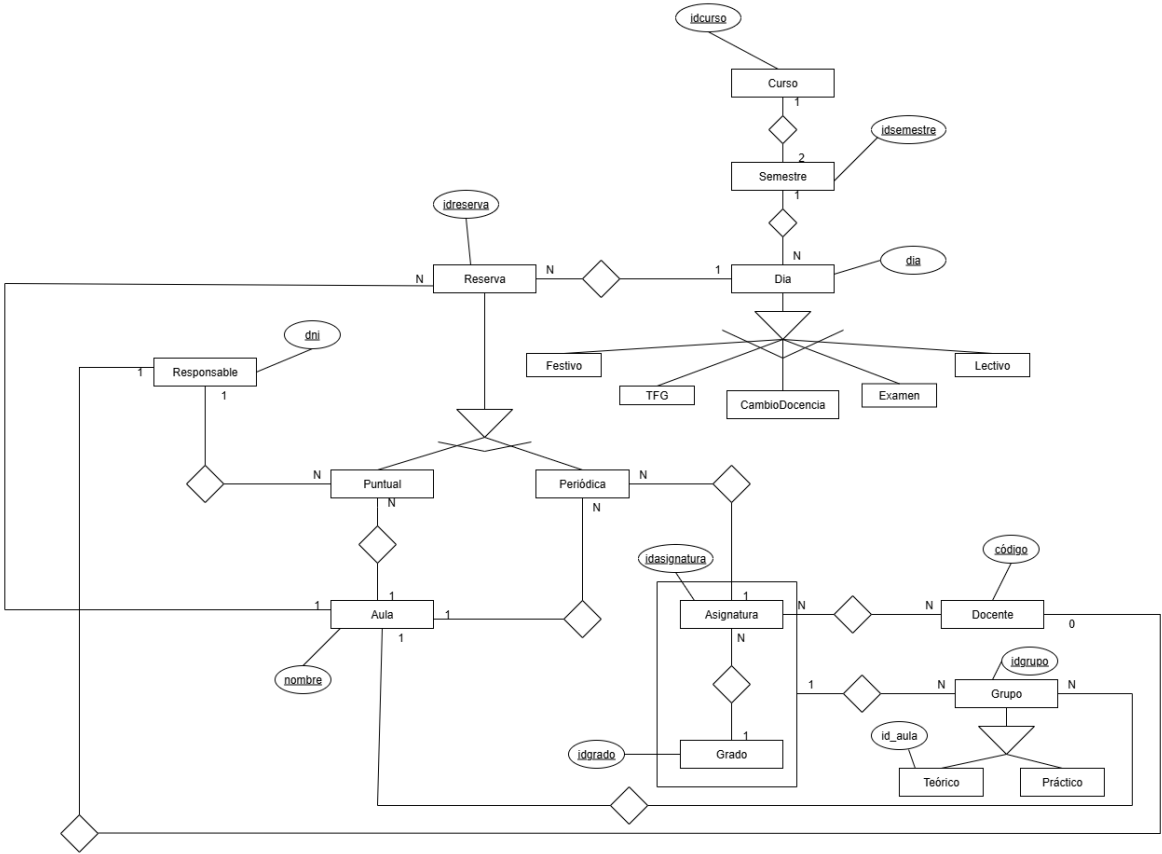


Figura C.1: Diagrama entidad-relación

Diccionario de datos

El diccionario de datos recoge la descripción detallada de las tablas que forman la base de datos de la aplicación. Para cada tabla se especifican sus atributos principales, el tipo de dato utilizado, las claves primarias y ajenas, las restricciones más relevantes y una breve descripción de la información que almacena cada campo.

Este apartado permite complementar el modelo relacional, ya que ofrece una visión más precisa de la estructura interna de cada tabla y facilita la comprensión de cómo se almacena la información dentro del sistema.

Tabla AULA

La tabla *AULA* **C.1** almacena la información relativa a las aulas disponibles en el sistema. Incluye datos identificativos del aula, su ubicación, capacidad y recursos disponibles, como ordenadores, proyector, altavoces o enchufes.

Campo	Tipo	Restricciones	Descripción
ID	INT	PK	Identificador único del aula. Se genera automáticamente.
NOMBRE	VARCHAR(100)	UNIQUE, NOT NULL	Nombre identificativo del aula.
EDIFICIO	VARCHAR(6)		Edificio en el que se encuentra el aula.
PLANTA	NUMERIC(1)		Planta en la que se encuentra el aula.
CAPACIDAD	NUMERIC(3)	CHECK	Capacidad máxima del aula. Debe ser mayor o igual que 0.
NUM_ORDENADORES	NUMERIC(2)	CHECK	Número de ordenadores disponibles en el aula. Debe ser mayor o igual que 0.
ALTAVOCES	BOOLEAN	DEFAULT	Indica si el aula dispone de altavoces. Por defecto, su valor es falso.
PROYECTOR	BOOLEAN	DEFAULT	Indica si el aula dispone de proyector. Por defecto, su valor es falso.
CAMARA	BOOLEAN	DEFAULT	Indica si el aula dispone de cámara. Por defecto, su valor es falso.
ENCHUFES	BOOLEAN	DEFAULT	Indica si el aula dispone de enchufes. Por defecto, su valor es falso.
M2	NUMERIC(6,2)	CHECK	Superficie aproximada del aula en metros cuadrados. Debe ser mayor o igual que 0.
OTROS	VARCHAR(30)		Información adicional sobre el aula.
GRADO_ID	VARCHAR(10)	FK	Identificador del grado asociado o prioritario para el aula. Referencia a la tabla <i>GRADO</i> .
SEMESTRE_ACADEMICO	NUMERIC(2)	CHECK	Semestre académico asociado al aula. Debe estar comprendido entre 1 y 12.
CAMPUS	CHAR(1)		Campus al que pertenece el aula.

Tabla C.1: Diccionario de datos de la tabla AULA

Tabla DOCENTE

Como se muestra en la Tabla C.2, la tabla *DOCENTE* recoge los datos identificativos y de contacto del profesorado.

Campo	Tipo	Restricciones	Descripción
CODIGO	INT	PK	Identificador único del docente. Se genera automáticamente.
NOMBRE	VARCHAR(50)	NOT NULL	Nombre del docente.
APELLIDOS	VARCHAR(100)	NOT NULL	Apellidos del docente.
CORREO	VARCHAR(50)	NOT NULL	Correo electrónico del docente.
TELEFONO	VARCHAR(9)		Número de teléfono de contacto del docente.

Tabla C.2: Diccionario de datos de la tabla DOCENTE

Tabla ASIGNATURAS

La tabla *ASIGNATURAS* almacena la información relativa a las asignaturas registradas en el sistema. Esta tabla permite identificar cada asignatura, asociarla a un grado concreto y almacenar información académica relevante, como el curso, el semestre, los créditos ECTS, el tipo de asignatura y las horas prácticas. Como se muestra en la Tabla C.3, algunos campos presentan restricciones adicionales para garantizar la coherencia de los datos.

Campo	Tipo	Restricciones	Descripción
IDASIGNATURA	VARCHAR(10)	PK	Identificador único de la asignatura.
NOMBRE	VARCHAR(100)	NOT NULL	Nombre completo de la asignatura.
ABREVIATURA	VARCHAR(20)		Abreviatura utilizada para identificar la asignatura de forma resumida.
CURSO_GRADO	NUMERIC(1)	NOT NULL, CHECK	Curso del grado en el que se imparte la asignatura. Debe estar comprendido entre 1 y 6.
SEMESTRE_ACADEMICO	NUMERIC(2)	NOT NULL, CHECK	Semestre académico en el que se imparte la asignatura. Debe estar comprendido entre 1 y 12.
ECTS	NUMERIC(2)	NOT NULL, CHECK	Número de créditos ECTS de la asignatura. Debe ser mayor o igual que 0.
GRADO_ID	VARCHAR(10)	NOT NULL, FK	Identificador del grado al que pertenece la asignatura. Referencia a la tabla <i>GRADO</i> .
TIPO	CHAR(1)		Tipo de asignatura, por ejemplo obligatoria u optativa.
HORAS_PRACTICAS	NUMERIC(1)		Número de horas prácticas asociadas a la asignatura.

Tabla C.3: Diccionario de datos de la tabla ASIGNATURAS

Tabla IMPARTE

La tabla *IMPARTE* representa la relación entre los docentes y las asignaturas que imparten. Esta tabla actúa como tabla intermedia, ya que un docente puede impartir varias asignaturas y una asignatura puede estar asociada a varios docentes. Como se muestra en la Tabla C.4, se incluye una restricción de unicidad para evitar que se repita la misma combinación de docente y asignatura.

Campo	Tipo	Restricciones	Descripción
ID	INT	PK	Identificador único de la relación. Se genera automáticamente.
CODIGO_ DOCENTE	VARCHAR(9)	NOT NULL, FK, UNIQUE	Identificador del docente que imparte la asignatura. Referencia a la tabla <i>DOCENTE</i> .
ID_ ASIGNATURA	VARCHAR(10)	NOT NULL, FK, UNIQUE	Identificador de la asignatura impartida por el docente. Referencia a la tabla <i>ASIGNATURAS</i> .

Tabla C.4: Diccionario de datos de la tabla IMPARTE

Tabla GRADO

La tabla *GRADO* almacena la información relativa a los grados o titulaciones registrados en el sistema. Esta tabla permite asociar asignaturas y aulas a una titulación concreta, facilitando la organización de los horarios y reservas por grado. Como se muestra en la Tabla C.5, cada grado se identifica mediante un código único.

Campo	Tipo	Restricciones	Descripción
IDGRADO	VARCHAR(10)	PK	Identificador único del grado.
NOMBRE	VARCHAR(150)	NOT NULL	Nombre completo del grado o titulación.
ABREVIATURA	VARCHAR(20)	NOT NULL	Abreviatura utilizada para identificar el grado de forma resumida.
COORDINADOR	VARCHAR(150)		Nombre del coordinador asociado al grado.

Tabla C.5: Diccionario de datos de la tabla GRADO

Tabla GRUPO

La tabla *GRUPO* almacena la información relativa a los grupos asociados a cada asignatura. Esta tabla permite diferenciar los distintos grupos docentes de una misma asignatura y, de forma opcional si el grupo es teórico, asociarlos a un aula concreta. Como se muestra en la Tabla C.6, cada grupo queda vinculado obligatoriamente a una asignatura.

Campo	Tipo	Restricciones	Descripción
GRUPOID	INT	PK	Identificador único del grupo. Se genera automáticamente.
NOMBRE	CHAR(3)		Nombre o código breve utilizado para identificar el grupo.
ID_ ASIGNATURA	VARCHAR(10)	NOT NULL, FK	Identificador de la asignatura a la que pertenece el grupo. Referencia a la tabla <i>ASIGNATURAS</i> .
ID_AULA	INT	FK	Identificador del aula preferente asociada al grupo, solo grupos teóricos. Referencia a la tabla <i>AULA</i> .

Tabla C.6: Diccionario de datos de la tabla GRUPO

Tabla RESPONSABLE

La tabla *RESPONSABLE* almacena la información de las personas responsables de realizar o gestionar reservas puntuales dentro del sistema. Esta tabla permite identificar a cada responsable mediante su correo electrónico y, de forma opcional, asociarlo con un docente registrado en la aplicación. Como se muestra en la Tabla C.7, el campo *CODIGO_DOCENTE* permite vincular un responsable con la tabla *DOCENTE*.

Campo	Tipo	Restricciones	Descripción
CORREO	VARCHAR(50)	PK	Correo electrónico del responsable. Actúa como identificador único.
NOMBRE	VARCHAR(50)	NOT NULL	Nombre del responsable.
APELLIDOS	VARCHAR(100)	NOT NULL	Apellidos del responsable.
TELEFONO	NUMERIC(9)		Número de teléfono de contacto del responsable.
CODIGO_ DOCENTE	INT	UNIQUE, FK	Identificador del docente asociado al responsable. Referencia a la tabla <i>DOCENTE</i> .

Tabla C.7: Diccionario de datos de la tabla RESPONSABLE

Tabla RESERVA

La tabla *RESERVA* almacena la información común de todas las reservas registradas en el sistema, independientemente de si son reservas puntuales o periódicas. En ella se recoge el aula reservada, el día de la reserva, el estado, el tipo y la franja horaria correspondiente. Como se muestra en la Tabla C.8, esta tabla permite centralizar los datos compartidos por los distintos tipos

de reserva.

Campo	Tipo	Restricciones	Descripción
IDRESERVA	INT	PK	Identificador único de la reserva. Se genera automáticamente.
ID_AULA	INT	NOT NULL, FK	Identificador del aula asociada a la reserva. Referencia a la tabla <i>AULA</i> .
ID_DIA	DATE	NOT NULL, FK	Fecha asociada a la reserva. Referencia a la tabla <i>DIA</i> .
ESTADO	CHAR(1)		Estado actual de la reserva. Permite diferenciar, reservas pendientes (P), aprobadas (A) o rechazadas (R).
TIPO	CHAR(1)		Tipo de reserva registrada, permitiendo distinguir entre reservas puntuales (P) y periódicas (R).
HORA_INICIO	TIME	NOT NULL	Hora de inicio de la reserva.
HORA_FIN	TIME	NOT NULL, CHECK	Hora de finalización de la reserva. Debe ser posterior a la hora de inicio.

Tabla C.8: Diccionario de datos de la tabla RESERVA

Tabla RESERVA_PUNTUAL

La tabla *RESERVA_PUNTUAL* almacena la información específica de las reservas puntuales realizadas en el sistema. Esta tabla amplía los datos generales almacenados en la tabla *RESERVA*, incorporando información propia de una solicitud concreta, como el responsable, la capacidad solicitada, los recursos requeridos, el motivo de la reserva y las observaciones asociadas. Como se muestra en la Tabla C.9, cada reserva puntual se identifica mediante el mismo identificador de la reserva general.

Campo	Tipo	Restricciones	Descripción
ID_ RESERVA	INT	PK, FK	Identificador de la reserva puntual. Referencia a la tabla <i>RESERVA</i> .
ID_ RESPONSABLE	VARCHAR(30)	NOT NULL, FK	Identificador del responsable asociado a la reserva. Referencia a la tabla <i>RESPONSABLE</i> .
CAPACIDAD_ SOLICITADA	NUMERIC(3)	CHECK	Capacidad solicitada para la reserva. Debe ser mayor o igual que 0.
MOTIVO	VARCHAR(90)		Motivo o título asociado a la reserva puntual.
INICIO	TIMESTAMP	NOT NULL, DEFAULT	Fecha y hora de inicio solicitada para la reserva. Por defecto, toma la fecha actual.
FIN	TIMESTAMP	NOT NULL, DEFAULT, CHECK	Fecha y hora de finalización solicitada para la reserva. Debe ser posterior o igual al inicio.
NUM_ ORDENADORES_ SOLICITADOS	NUMERIC(2)	CHECK	Número de ordenadores solicitados. Debe ser mayor o igual que 0.
ALTAVOCES_ SOLICITADOS	BOOLEAN	NOT NULL, DEFAULT	Indica si se solicitan altavoces. Por defecto, su valor es falso.
PROYECTOR_ SOLICITADO	BOOLEAN	NOT NULL, DEFAULT	Indica si se solicita proyector. Por defecto, su valor es falso.
CAMARA_ SOLICITADA	BOOLEAN	NOT NULL, DEFAULT	Indica si se solicita cámara. Por defecto, su valor es falso.
ENCHUFES_ SOLICITADOS	BOOLEAN	NOT NULL, DEFAULT	Indica si se solicitan enchufes. Por defecto, su valor es falso.
MOMENTO_ RESERVA	TIMESTAMP	NOT NULL, DEFAULT	Fecha y hora en la que se registra la solicitud de reserva.
OBSERVACIONES	VARCHAR(300)		Observaciones adicionales introducidas sobre la reserva puntual.

Tabla C.9: Diccionario de datos de la tabla RESERVA_PUNTUAL

Tabla RESERVA_PERIODICA

La tabla *RESERVA_PERIODICA* almacena la información específica de las reservas que se repiten durante un periodo de tiempo determinado. Este tipo de reserva está especialmente orientado a la planificación docente, ya que permite asociar una reserva a un grupo concreto y definir su repetición semanal dentro de un intervalo de fechas. Como se muestra en la Tabla C.10, cada reserva periódica se vincula con una reserva general y con un grupo académico.

Campo	Tipo	Restricciones	Descripción
ID_ RESERVA	INT	PK, FK	Identificador de la reserva periódica. Referencia a la tabla <i>RESERVA</i> .
ID_ GRUPO	VARCHAR(10)	NOT NULL, FK	Identificador del grupo asociado a la reserva periódica. Referencia a la tabla <i>GRUPO</i> .
DIA_ SEMANA	SMALLINT	NOT NULL, CHECK	Día de la semana en el que se repite la reserva. Debe estar comprendido entre 1 y 7.
FECHA_ INICIO	DATE	NOT NULL, DEFAULT	Fecha de inicio del periodo en el que se repite la reserva. Por defecto, toma la fecha actual.
FECHA_ FIN	DATE	NOT NULL, DEFAULT, CHECK	Fecha de finalización del periodo en el que se repite la reserva. Debe ser posterior o igual a la fecha de inicio.
INTERVALO_ SEMANAS	SMALLINT	NOT NULL, DEFAULT, CHECK	Intervalo de repetición de la reserva expresado en semanas. Debe ser mayor o igual que 1.

Tabla C.10: Diccionario de datos de la tabla RESERVA_PERIODICA

Tabla CURSO

La tabla *CURSO* almacena la información relativa a los cursos académicos registrados en el sistema. Esta tabla permite definir el intervalo temporal de cada curso y controlar si el horario asociado a dicho curso ya ha sido cargado en la aplicación. Como se muestra en la Tabla C.11, cada curso se identifica mediante un código único.

Campo	Tipo	Restricciones	Descripción
IDCURSO	VARCHAR(10)	PK	Identificador único del curso académico, por ejemplo 2025-2026.
FECHA_ INICIO	DATE	NOT NULL, DEFAULT	Fecha de inicio del curso académico. Por defecto, toma la fecha actual.
FECHA_ FIN	DATE	NOT NULL, DEFAULT	Fecha de finalización del curso académico. Por defecto, toma la fecha actual.
HORARIO_ CARGADO	BOOLEAN	NOT NULL, DEFAULT	Indica si el horario correspondiente al curso académico ya ha sido cargado en el sistema. Por defecto, su valor es falso.

Tabla C.11: Diccionario de datos de la tabla CURSO

Tabla SEMESTRE

La tabla *SEMESTRE* almacena la información relativa a los semestres académicos que forman parte de un curso. Esta tabla permite dividir el

curso académico en periodos docentes diferenciados, indicando el número de semestre y las fechas de inicio y fin correspondientes. Como se muestra en la Tabla C.12, cada semestre queda asociado obligatoriamente a un curso académico.

Campo	Tipo	Restricciones	Descripción
IDSEMESTRE	VARCHAR(15)	PK	Identificador único del semestre académico.
CURSO_ID	VARCHAR(10)	NOT NULL, FK	Identificador del curso académico al que pertenece el semestre. Referencia a la tabla <i>CURSO</i> .
NUMERO	NUMERIC(1)	CHECK	Número del semestre. Solo puede tomar los valores 1 o 2.
FECHA_INICIO	DATE	NOT NULL, DEFAULT	Fecha de inicio del semestre académico. Por defecto, toma la fecha actual.
FECHA_FIN	DATE	NOT NULL, DEFAULT	Fecha de finalización del semestre académico. Por defecto, toma la fecha actual.

Tabla C.12: Diccionario de datos de la tabla SEMESTRE

Tabla DIA

La tabla *DIA* almacena las fechas que forman parte del calendario académico. Cada día puede estar asociado a un semestre y contiene información sobre el día de la semana correspondiente. Esta tabla sirve como base para clasificar posteriormente las fechas como lectivas, festivas, días de examen, cambios de docencia o periodos asociados a TFG/TFM. Como se muestra en la Tabla C.13, la fecha actúa como identificador único de cada registro.

Campo	Tipo	Restricciones	Descripción
DIA	DATE	PK	Fecha concreta del calendario académico. Actúa como identificador único del día.
ID_SEMESTRE	VARCHAR(15)	FK	Identificador del semestre al que pertenece el día. Referencia a la tabla <i>SEMESTRE</i> .
DIA_SEMANA	NUMERIC	CHECK	Número correspondiente al día de la semana. Debe estar comprendido entre 1 y 7.

Tabla C.13: Diccionario de datos de la tabla DIA

Tabla LECTIVO

La tabla *LECTIVO* almacena los días del calendario académico que tienen carácter lectivo. Esta tabla permite diferenciar las fechas en las que

se desarrolla actividad docente ordinaria respecto a otros tipos de día, como festivos, exámenes o cambios de docencia. Como se muestra en la Tabla C.14, cada día lectivo se identifica mediante una fecha registrada previamente en la tabla *DIA*.

Campo	Tipo	Restricciones	Descripción
ID_DIA	DATE	PK, FK	Fecha identificativa del día lectivo. Referencia a la tabla <i>DIA</i> .

Tabla C.14: Diccionario de datos de la tabla LECTIVO

Tabla CAMBIO_DOCENCIA

La tabla *CAMBIO_DOCENCIA* almacena los días del calendario académico en los que se produce una modificación de la docencia habitual. Esta tabla permite indicar que una fecha concreta sustituye la planificación correspondiente a otro día de la semana. Como se muestra en la Tabla C.15, cada cambio de docencia queda asociado a un día registrado en la tabla *DIA*.

Campo	Tipo	Restricciones	Descripción
ID_DIA	DATE	PK, FK	Fecha identificativa del cambio de docencia. Referencia a la tabla <i>DIA</i> .
SUSTITUYE_DIA	NUMERIC	CHECK	Día de la semana cuya docencia se sustituye. Debe estar comprendido entre 1 y 5.

Tabla C.15: Diccionario de datos de la tabla CAMBIO_DOCENCIA

Tabla EXAMEN

La tabla *EXAMEN* almacena los días del calendario académico destinados a la realización de exámenes. Esta tabla permite identificar la convocatoria asociada a cada fecha, diferenciando entre primera y segunda convocatoria. Como se muestra en la Tabla C.16, cada registro queda vinculado a una fecha existente en la tabla *DIA*.

Campo	Tipo	Restricciones	Descripción
ID_DIA	DATE	PK, FK	Fecha identificativa del día de examen. Referencia a la tabla <i>DIA</i> .
CONVOCATORIA	NUMERIC(1)	CHECK	Convocatoria asociada al examen. Solo puede tomar los valores 1 o 2.

Tabla C.16: Diccionario de datos de la tabla EXAMEN

Tabla FESTIVO

La tabla *FESTIVO* almacena los días del calendario académico que tienen carácter festivo. Esta tabla permite registrar el nombre del festivo y su alcance, indicando si afecta a nivel nacional, autonómico, local, universitario o de centro. Como se muestra en la Tabla C.17, cada festivo se identifica mediante una fecha registrada previamente en la tabla *DIA*.

Campo	Tipo	Restricciones	Descripción
ID_DIA	DATE	PK, FK	Fecha identificativa del día festivo. Referencia a la tabla <i>DIA</i> .
NOMBRE	VARCHAR(60)		Nombre descriptivo del festivo.
ALCANCE	VARCHAR(20)		Ámbito o alcance del festivo.

Tabla C.17: Diccionario de datos de la tabla FESTIVO

Tabla TFG

La tabla *TFG* almacena los días del calendario académico destinados a la defensa o gestión de Trabajos de Fin de Grado o Trabajos de Fin de Máster. Esta tabla permite indicar la convocatoria asociada a cada fecha. Como se muestra en la Tabla C.18, cada registro queda vinculado a una fecha existente en la tabla *DIA*.

Campo	Tipo	Restricciones	Descripción
ID_DIA	DATE	PK, FK	Fecha identificativa del día asociado a TFG o TFM. Referencia a la tabla <i>DIA</i> .
CONVOCATORIA	NUMERIC(1)	CHECK	Convocatoria asociada al periodo de TFG o TFM. Solo puede tomar los valores 1 o 2.

Tabla C.18: Diccionario de datos de la tabla TFG

C.3. Diseño arquitectónico

En esta sección se describe la arquitectura general seguida durante el desarrollo de la aplicación. El objetivo es mostrar cómo se organizan los distintos componentes del sistema, qué responsabilidades tiene cada uno de ellos y cómo se comunican entre sí para ofrecer las funcionalidades definidas en la especificación de requisitos.

La aplicación se ha diseñado siguiendo una arquitectura web cliente-servidor desacoplada. En este modelo, el *frontend* y el *backend* se desarrollan como dos partes independientes que se comunican mediante peticiones HTTP. Esta separación permite mantener diferenciada la capa de presentación, encargada de la interacción con el usuario, de la capa de negocio, encargada de procesar las operaciones, validar datos y acceder a la base de datos.

Arquitectura general de la aplicación

La arquitectura general del sistema está formada por cuatro elementos principales: el usuario, el *frontend*, el *backend* y la base de datos. Además, el sistema se comunica con el servicio institucional de autenticación de la Universidad de Burgos para validar las credenciales de los usuarios.

El usuario accede a la aplicación a través de un navegador web. Desde este navegador interactúa con la interfaz desarrollada en React, que actúa como cliente de la aplicación. Cuando el usuario realiza una acción, el *frontend* envía una petición HTTP al *backend*.

El *backend*, desarrollado con Django, recibe la petición, comprueba los permisos del usuario, valida los datos recibidos y ejecuta la lógica de negocio correspondiente. Si la operación requiere consultar o modificar información persistente, el *backend* accede a la base de datos. Una vez procesada la operación, devuelve una respuesta al *frontend*, que actualiza la interfaz y muestra el resultado al usuario.

De forma general, el flujo de comunicación de la aplicación es el siguiente:

1. El usuario interactúa con una pantalla de la aplicación.
2. El *frontend* envía una petición HTTP al *backend*.
3. El *backend* valida la petición y comprueba los permisos del usuario.
4. Si es necesario, el *backend* consulta o modifica la base de datos.

5. El *backend* devuelve una respuesta al *frontend*.
6. El *frontend* actualiza la vista y muestra el resultado de la operación.

Esta organización facilita la mantenibilidad del sistema, ya que cada componente tiene responsabilidades claramente diferenciadas. Además, permite que el *frontend* y el *backend* puedan evolucionar de forma independiente, siempre que se mantenga la interfaz de comunicación entre ambos.

En la Figura C.3 se muestra la arquitectura general de la aplicación.

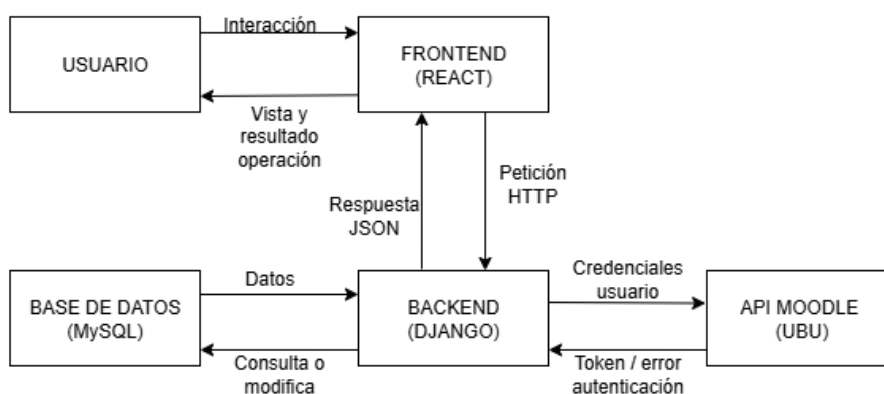


Figura C.3: Arquitectura general de la aplicación

Arquitectura del backend

El *backend* constituye la capa encargada de la lógica de negocio de la aplicación. Está desarrollado con Django y se encarga de recibir las peticiones procedentes del *frontend*, aplicar las reglas del sistema y acceder a la base de datos cuando resulta necesario.

La arquitectura del *backend* se organiza siguiendo una separación por responsabilidades. Por un lado, las vistas o controladores reciben las peticiones HTTP y devuelven las respuestas correspondientes. Por otro lado, los modelos representan las entidades persistentes de la aplicación y permiten interactuar con la base de datos. Además, se incluyen módulos auxiliares encargados de procesos específicos, como la autenticación mediante el servicio institucional, la carga de horarios desde hojas de cálculo o la validación de reservas.

Las responsabilidades principales del *backend* son las siguientes:

- Validar las credenciales del usuario mediante el sistema institucional de la Universidad de Burgos.
- Gestionar la sesión del usuario autenticado.
- Comprobar el rol asociado al usuario y aplicar las restricciones de acceso según permisos.
- Procesar las operaciones de creación, consulta, modificación y eliminación de reservas.
- Validar la disponibilidad de aulas antes de crear o modificar una reserva.
- Gestionar la carga de horarios desde hojas de cálculo.
- Transformar los horarios cargados en reservas periódicas.
- Aplicar las restricciones derivadas del calendario académico, como festivos, días no lectivos y cambios de docencia.
- Proporcionar al *frontend* los datos necesarios para mostrar horarios, reservas, aulas y calendario académico.

Esta separación permite que la lógica de negocio no quede mezclada con la interfaz de usuario y que las operaciones críticas se centralicen en el servidor. De esta forma, aunque el usuario interactúe desde el navegador, las validaciones importantes se realizan siempre en el *backend*.

Arquitectura del frontend

El *frontend* constituye la capa de presentación de la aplicación. Está desarrollado con React y se encarga de mostrar la interfaz de usuario, gestionar la navegación entre pantallas y enviar las peticiones correspondientes al *backend*.

El diseño del *frontend* se basa en una arquitectura orientada a componentes. Cada pantalla o funcionalidad se divide en componentes reutilizables, como formularios, tablas, botones, filtros, cabeceras o elementos de navegación. Esta organización permite construir interfaces más mantenibles y facilita la reutilización de elementos comunes en distintas partes de la aplicación.

Además de los componentes visuales, el *frontend* dispone de servicios encargados de centralizar las llamadas al *backend*. Estos servicios permiten

separar la lógica de comunicación con la API de la lógica visual de los componentes. De esta forma, si cambia la ruta de un endpoint o la estructura de una petición, la modificación puede realizarse en un único punto sin afectar directamente a todas las pantallas.

Las responsabilidades principales del *frontend* son las siguientes:

- Mostrar las pantallas disponibles según el rol del usuario autenticado.
- Permitir la navegación entre los distintos módulos de la aplicación.
- Gestionar formularios de creación y edición de reservas, usuarios, roles, entidades y calendario académico.
- Mostrar listados de reservas, horarios y ocupación de aulas.
- Aplicar filtros de búsqueda para facilitar la consulta de información.
- Enviar peticiones al *backend* y procesar las respuestas recibidas.
- Mostrar mensajes de éxito, error o validación al usuario.

Esta arquitectura basada en componentes favorece la modularidad y permite que las distintas partes de la interfaz puedan evolucionar de forma independiente.

Autenticación y control de acceso

La aplicación incorpora un sistema de autenticación basado en las credenciales institucionales de la Universidad de Burgos. El usuario introduce su nombre de usuario y contraseña en la interfaz de inicio de sesión, y el *backend* se encarga de validar dichas credenciales mediante el servicio externo correspondiente.

La contraseña introducida por el usuario no se almacena en la base de datos de la aplicación. Una vez validado el acceso, el sistema identifica al usuario y consulta el rol asociado en la base de datos interna. Este rol determina las funcionalidades a las que el usuario puede acceder y las acciones que puede realizar dentro del sistema.

El control de acceso se basa en la existencia de varios perfiles de usuario. Entre ellos se encuentran el administrador, el gestor editor, el gestor no editor y el PDI. Cada uno de estos perfiles tiene permisos diferenciados, pero siguen un modelo en cascada, un perfil tiene sus permisos diferencias y los mismos

que los siguientes. Por ejemplo, el administrador puede gestionar usuarios, roles y entidades; el gestor editor puede modificar reservas y calendario académico; el gestor no editor puede consultar información sin modificarla; y el PDI puede solicitar reservas puntuales y consultar sus propias solicitudes.

Este diseño permite proteger las operaciones críticas de la aplicación y evita que usuarios sin autorización puedan realizar cambios en horarios, reservas, aulas, usuarios o calendario académico.

Despliegue de la aplicación

El despliegue de la aplicación se ha planteado utilizando Railway como plataforma de alojamiento. Esta plataforma permite desplegar el *backend*, el *frontend*, la base de datos MySQL y los servicios necesarios para que la aplicación pueda estar disponible en un entorno accesible a través de Internet.

En un entorno de producción, el usuario accedería a la aplicación mediante un navegador web. El *frontend* serviría la interfaz de usuario y se comunicaría con el *backend* mediante peticiones HTTP. El *backend*, por su parte, se conectaría con la base de datos para consultar o modificar la información persistente del sistema.

El despliegue debe tener en cuenta la configuración de variables de entorno, especialmente aquellas relacionadas con la conexión a la base de datos, la configuración del servicio de autenticación institucional, las claves secretas de Django y los parámetros de seguridad necesarios para ejecutar la aplicación fuera del entorno local.

En la Figura C.4 se muestra el esquema general de despliegue de la aplicación.

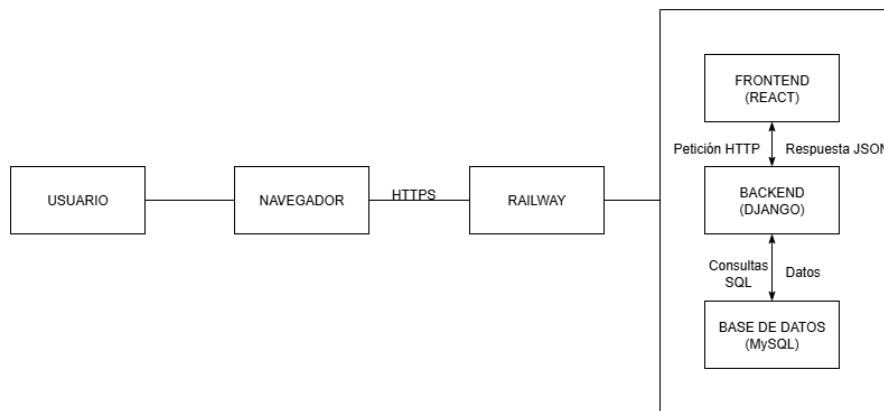


Figura C.4: Diagrama de despliegue de la aplicación

C.4. Diseño procedimental

En esta sección se describe el comportamiento interno de algunos de los procesos principales de la aplicación. Mientras que el diseño arquitectónico permite conocer la estructura general del sistema y la comunicación entre sus componentes, el diseño procedimental se centra en explicar la secuencia de pasos que se siguen para realizar determinadas operaciones.

Para ello, se han seleccionado los procedimientos más relevantes del sistema, especialmente aquellos relacionados con la autenticación de usuarios, la gestión de reservas, la carga de horarios desde hojas de cálculo y la generación de reservas periódicas. Estos procesos representan el núcleo funcional de la aplicación y permiten comprender cómo interactúan el usuario, el *frontend*, el *backend* y la base de datos durante la ejecución de las principales funcionalidades.

Cada procedimiento se describe de forma textual y puede complementarse mediante diagramas de flujo o diagramas de secuencia, con el objetivo de facilitar la comprensión del funcionamiento interno del sistema.

Inicio de sesión

El proceso de inicio de sesión permite autenticar a un usuario mediante sus credenciales institucionales y determinar posteriormente las funcionalidades a las que puede acceder dentro de la aplicación. Este procedimiento es fundamental, ya que el resto de operaciones dependen del rol asignado al usuario autenticado.

En este proceso el usuario introduce sus credenciales, las cuales verifica la API del moodle, si son incorrectas devuelve un error y se mantiene en la página de login. Si son correctas, el sistema comprueba si existe una entrada para ese usuario en la base de datos, si no existe crea una nueva entrada asignándole por defecto el rol de PDI, tras este proceso en ambos casos envía el rol al *frontend*. El *frontend* ajusta las rutas y elementos accesibles en función del rol, y redirige a la página de gestión de reservas.

Este procedimiento garantiza que la aplicación no almacene la contraseña del usuario, ya que la validación de credenciales se realiza mediante el servicio institucional externo. La aplicación únicamente almacena la información necesaria para identificar al usuario y gestionar sus permisos internos.

En la Figura C.5 se muestra el diagrama correspondiente al proceso de inicio de sesión.

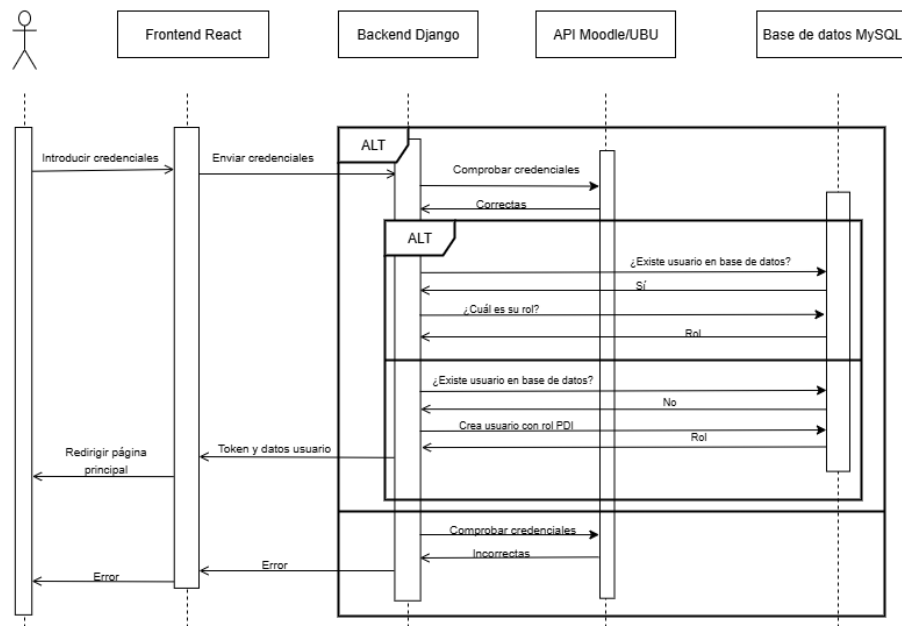


Figura C.5: Diagrama de secuencia correspondiente al inicio de sesión

Carga de horario desde hoja de cálculo

La carga de horario desde hoja de cálculo es uno de los procedimientos principales de la aplicación, ya que permite transformar la información contenida en la hoja de cálculo de horarios en reservas periódicas dentro del sistema. Este proceso evita introducir manualmente cada clase, reduciendo el

tiempo de gestión y disminuyendo la posibilidad de errores en la planificación académica.

En este proceso el usuario selecciona el curso para el que quiere cargar el horario, se abre un desplegable con un área donde cargar el archivo, el usuario lo carga y pulsa el botón “Enviar”. El archivo se manda al *backend*, el analizador léxico extraer las clases que se encuentran en la hoja de cálculo y el *backend* crea una reserva periódica para cada día de docencia de cada clase extraída.

Este procedimiento permite automatizar una parte fundamental de la gestión académica, ya que convierte los horarios definidos en hojas de cálculo en información estructurada dentro de la base de datos. Además, al realizarse las validaciones desde el *backend*, se garantiza que las reservas generadas respeten las restricciones del calendario académico.

En la Figura C.6 se muestra el diagrama correspondiente al proceso de carga de horario desde hoja de cálculo.

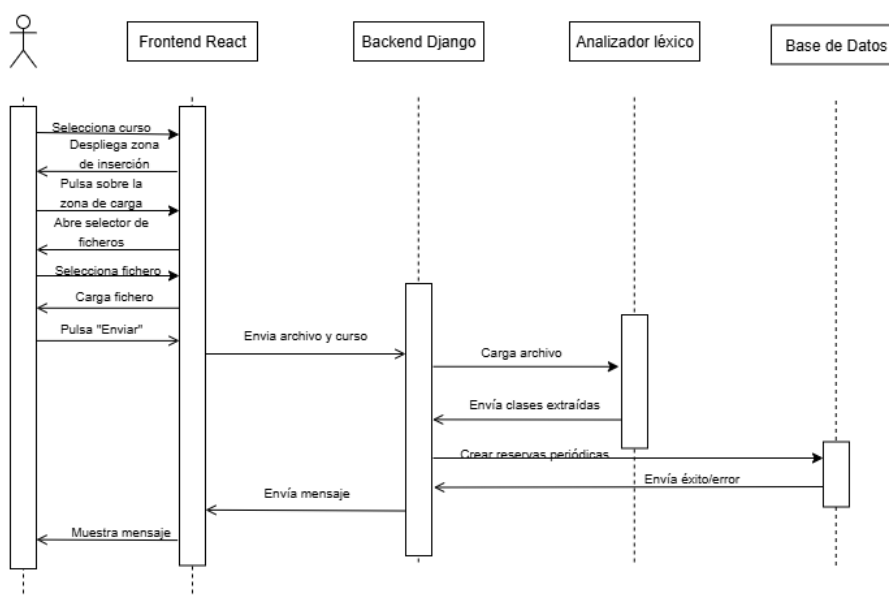


Figura C.6: Diagrama de secuencia correspondiente a la carga de horarios desde hoja de cálculo

Creación de reserva puntual

La creación de una reserva puntual permite registrar una solicitud de uso de un aula para una fecha y franja horaria concreta. Este procedimiento está

orientado principalmente a reservas no periódicas, como charlas, reuniones, actividades docentes concretas, exámenes u otros eventos que no forman parte del horario semanal habitual.

Para este procedimiento, el usuario deberá pulsar “Crear Nueva Reserva”, se abrirá el formulario, y rellenará los campos obligatorios y los campos opcionales que crea necesarios. Luego pulsará “Buscar Aulas Candidatas”, el sistema le mostrará la lista de aulas con los recursos que ha solicitado y que se encuentran disponibles para la fecha y hora seleccionada. El usuario pulsará en “Guardar”, y se generará la reserva o mostrará un mensaje de error.

Este procedimiento garantiza que las reservas puntuales se creen de forma controlada, comprobando previamente la validez de los datos introducidos y la disponibilidad de aulas.

En la Figura C.7 se muestra el diagrama correspondiente al proceso de creación de una reserva puntual.

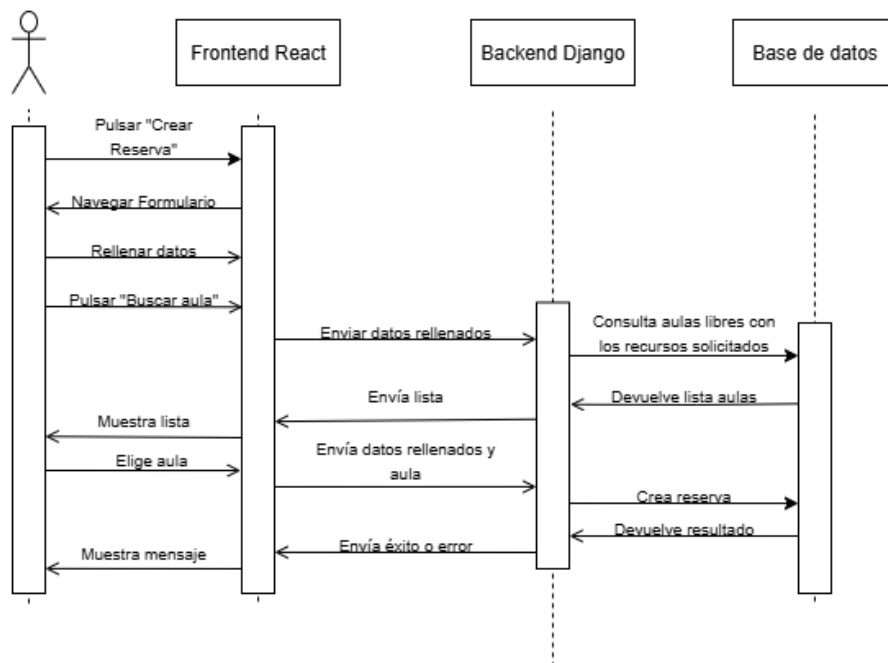


Figura C.7: Diagrama de secuencia correspondiente a la creación de una reserva puntual

Modificación de reserva periódica

La modificación de una reserva periódica permite actualizar una reserva asociada a la docencia de un grupo durante un periodo determinado del calendario académico. Este procedimiento es especialmente relevante para no permitir que se solapen grupos o que haya varias reservas periódicas que ocupen el mismo aula para la misma franja horaria de la semana.

En este procedimiento el usuario selecciona la reserva periódica que desea editar. Si modifica los campos relativos al horario y le da a guardar, el sistema valida que el aula seleccionada esté libre para el nuevo horario frente a otras reservas periódicas, y a su vez también valida que no hay otra reserva periódica que imparta clase para ese grupo del mismo grado y mismo semestre en el nuevo horario. Si alguna de las dos condiciones anteriores no la cumple da aviso, y permite al usuario cancelar o seguir. En el caso de que busque un nuevo aula, el sistema busca los aulas disponibles de reservas periódicas para ese tramo horario, y devuelve una lista de ellos. Tras los cambios realizados el usuario pulsa "Guardar" se actualizan los datos de la reserva periódica.

Este procedimiento garantiza que los cambios realizados sobre una reserva periódica no generen inconsistencias en la planificación académica. Al comprobar la disponibilidad del aula, la coherencia de las fechas, la franja horaria y las restricciones del calendario académico, el sistema evita que se produzcan solapes con otras reservas o que se asignen clases en días no válidos.

En la Figura C.8 se muestra el diagrama correspondiente al proceso de modificación de una reserva periódica.

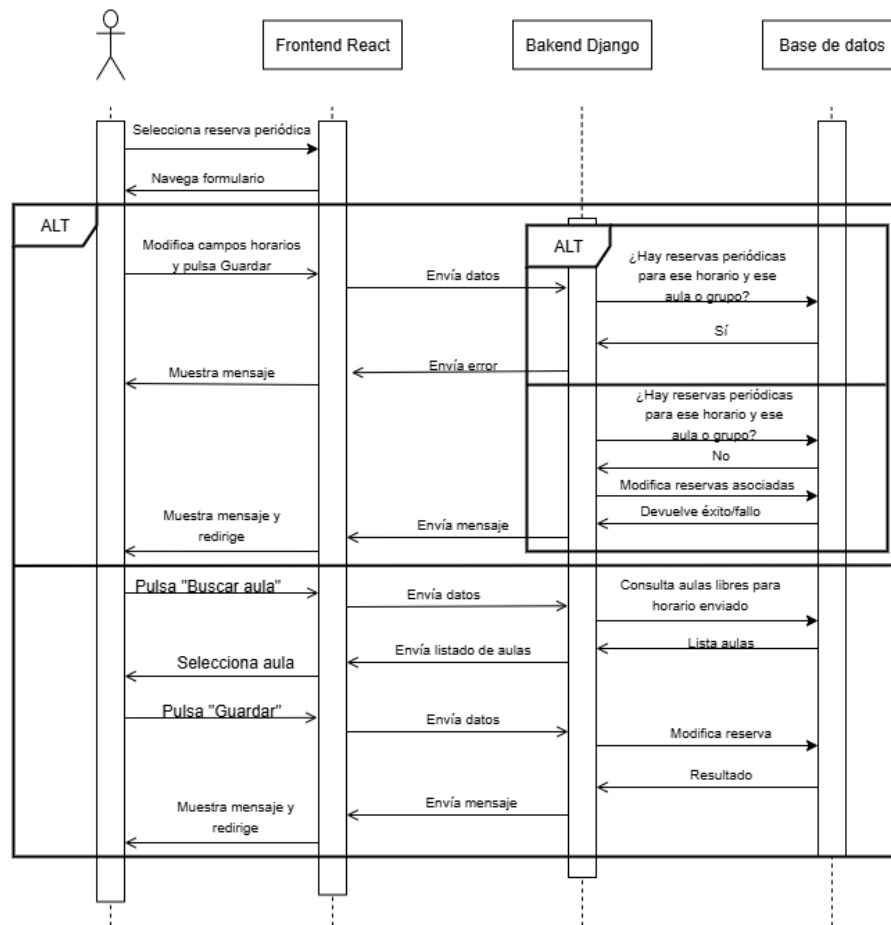


Figura C.8: Diagrama de secuencia correspondiente a la modificación de una reserva periódica

Consulta de ocupación de aula

La consulta de ocupación de aula permite visualizar las reservas asociadas a una o varias aulas concretas dentro de un intervalo de tiempo determinado. Esta funcionalidad resulta útil para conocer la disponibilidad real de los espacios docentes, comprobar si un aula se encuentra libre en una fecha y franja horaria concreta, y facilitar la toma de decisiones antes de crear o modificar una reserva.

El usuario selecciona el aula o aulas para el que quiere consultar la ocupación. Pulsa “Ver Ocupación”, el sistema carga todas las reservas asociadas a esas aulas y las representa en el calendario, permitiendo ver las reservas para rangos determinados de tiempo.

En la Figura C.9 se muestra el diagrama correspondiente al proceso de consulta de ocupación de aula.

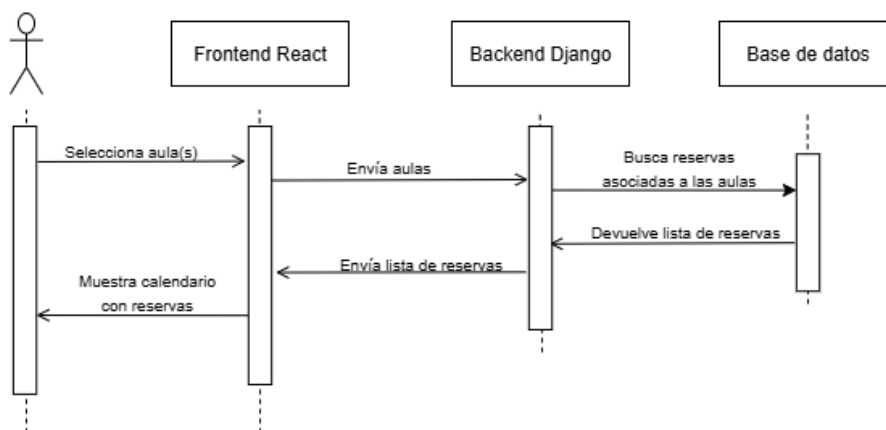


Figura C.9: Diagrama de secuencia correspondiente a ver la ocupación de una o varias aulas

Modificar día

La modificación de un día permite actualizar la información asociada a una fecha concreta del calendario académico. Esta funcionalidad es necesaria para reflejar situaciones especiales dentro del curso, como días lectivos, festivos, periodos de exámenes, cambios de docencia o fechas destinadas a TFG/TFM.

El usuario cuando está en la vista del calendario de un curso académicos, pulsa sobre uno o varios de los días del calendario. El sistema abre una modal con las opciones a las que cambiar el día, el usuario selecciona la opción, rellena los datos asociados y pulsa “Guardar”. El sistema valida que no sea sábado y domingo y n día distinto de festivo, modifica el tipo de día asociado a esa fecha y se actualiza la vista del calendario con los nuevos datos.

Este procedimiento permite mantener actualizado el calendario académico y garantiza que las modificaciones realizadas sobre una fecha concreta sean coherentes con la planificación docente. Además, resulta relevante para la generación y gestión de reservas periódicas, ya que estas dependen de la existencia de días lectivos, festivos y posibles cambios de docencia.

En la Figura C.10 se muestra el diagrama correspondiente al proceso de modificación de un día del calendario académico.

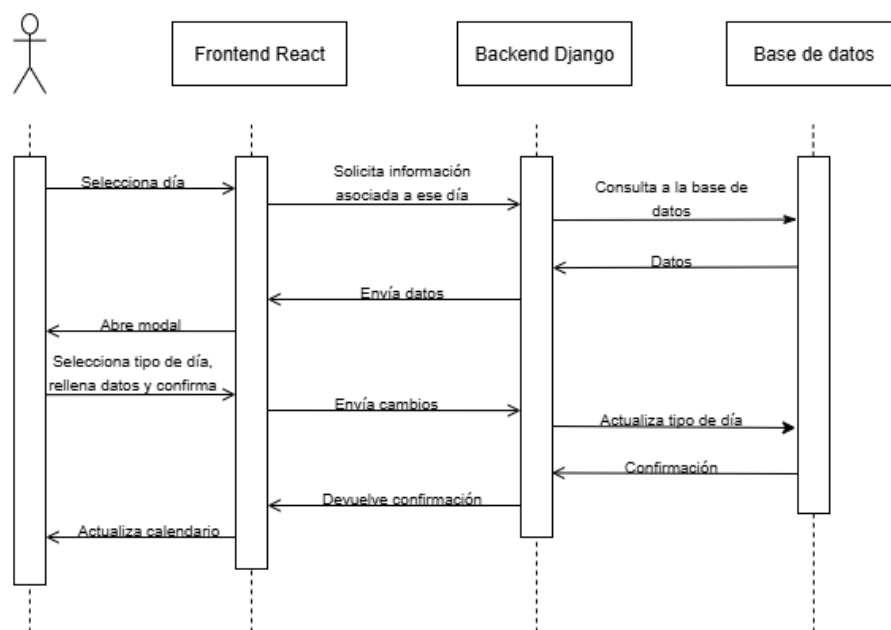


Figura C.10: Diagrama de secuencia correspondiente a la modificación del tipo de día en el calendario académico

Apéndice *D*

Documentación técnica de programación

D.1. Introducción

En este apéndice se recoge la documentación técnica de programación del proyecto. Su objetivo es facilitar la comprensión de la estructura interna de la aplicación, así como proporcionar la información necesaria para que otro desarrollador pueda instalar, ejecutar, mantener o ampliar el sistema.

La aplicación desarrollada está formada por un *backend* implementado con Django, un *frontend* desarrollado con React y Vite, y una base de datos MySQL. El *backend* se encarga de la lógica de negocio, la autenticación, la gestión de reservas, la carga de horarios y el acceso a la base de datos. El *frontend*, por su parte, proporciona la interfaz de usuario desde la que se accede a las distintas funcionalidades del sistema.

Esta documentación describe la estructura de directorios del proyecto, las principales partes del código, los pasos necesarios para instalar y ejecutar la aplicación, y las pruebas realizadas para comprobar el correcto funcionamiento del sistema.

El proyecto se organiza separando el código del *backend* y del *frontend*, lo que permite mantener diferenciadas la lógica de negocio y la interfaz de usuario. Esta separación facilita el mantenimiento del sistema y permite que ambas partes puedan evolucionar de forma independiente.

D.2. Estructura de directorios

La estructura del proyecto se organiza separando la parte correspondiente al *backend* y la parte correspondiente al *frontend*. Esta división permite diferenciar claramente la lógica de negocio y acceso a datos, desarrollada con Django, de la interfaz de usuario, desarrollada con React. Por otro lado, tenemos el *script SQL* de la base de datos llamado "BaseDeDatosHorario.sql", que se encuentra a la misma altura que el directorio del *frontend* y del *backend*.

En los siguientes apartados se describe la organización de ambos bloques, indicando la finalidad de los principales directorios y ficheros que forman parte del proyecto. Esta explicación facilita la comprensión del código fuente y permite que otros desarrolladores puedan localizar de forma más sencilla los módulos principales de la aplicación.

Backend

El *backend* de la aplicación se encuentra desarrollado con Django y contiene la lógica principal del sistema. En esta parte del proyecto se gestionan los modelos de datos, las rutas, las vistas, la autenticación, la comunicación con la base de datos y las operaciones relacionadas con horarios, reservas, calendario académico, aulas, docencia y usuarios.

En la Figura [D.1](#) se muestra la estructura principal de directorios correspondiente al *backend* del proyecto.

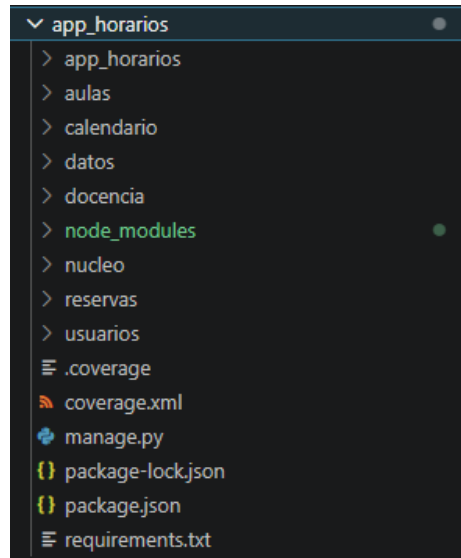


Figura D.1: Estructura de directorios del backend

Como se observa en la Figura D.1, el directorio del *backend* está formado por diferentes aplicaciones de Django, cada una de ellas orientada a una parte funcional concreta del sistema. Esta organización modular permite separar responsabilidades y facilita el mantenimiento del código.

El directorio *nucleo* contiene la configuración principal del proyecto Django. En él se encuentran los archivos generales de configuración, como la configuración del proyecto, las rutas principales y otros elementos necesarios para el arranque de la aplicación.

El módulo *aulas* agrupa la lógica relacionada con la gestión de aulas y su ocupación. Este módulo permite trabajar con la información relativa a capacidad, ubicación y equipamiento disponible en cada aula, y hacer las llamadas para obtener las reservas asociadas a cada aula.

El módulo *calendario* contiene las funcionalidades relacionadas con el calendario académico. En él se gestiona la información de días lectivos, festivos, exámenes, cambios de docencia y fechas asociadas a TFG/TFM. Se encarga de cargar el calendario académico obteniendo los tipos de día y de manejar el cambio de tipo de día, además de la generación de un nuevo calendario académico.

El módulo *docencia* agrupa la información académica relacionada con grados, asignaturas, grupos y docentes. Esta parte resulta fundamental para la generación y consulta de horarios, ya que permite asociar las reservas

periódicas con la estructura docente correspondiente. En este módulo, se obtienen los cursos con horario cargado, los grados con docencia y sus semestres, las reservas periódicas asociadas a todo lo anterior, en cuanto a mostrar el horario. En cuanto a las reservas periódicas, maneja la creación de una nueva reserva periódica, la edición, su eliminación, las restricciones, la búsqueda de aulas disponibles frente a otras reservas periódicas.

El módulo *app_horarios* contiene las funcionalidades de importación y exportación de la base de datos conjunta o por entidades.

El módulo *reservas* contiene la lógica relacionada con la gestión de reservas puntuales. En él se implementan las operaciones de creación, modificación, consulta, eliminación, y la carga de un horario creando con el analizador léxico y creando todas las reservas periódicas asociadas a la hoja de cálculo cargada.

El módulo *usuarios* se encarga de las funcionalidades relacionadas con la autenticación, los usuarios y los roles. Esta parte permite controlar el acceso a las distintas funcionalidades de la aplicación en función del perfil asociado a cada usuario. Por lo que en esta parte se encuentra la conexión con la API del moodle de la universidad.

Además de estos directorios, aparecen varios ficheros relevantes en la raíz del proyecto. El archivo *manage.py* es el punto de entrada para ejecutar comandos propios de Django, como iniciar el servidor de desarrollo, aplicar migraciones o crear usuarios administradores. El fichero *requirements.txt* contiene las dependencias de Python necesarias para ejecutar el *backend*. Por otro lado, los archivos *package.json* y *package-lock.json* recogen las dependencias asociadas a Node.js, utilizadas por la parte de *frontend*. Finalmente, el directorio *node_modules* se genera automáticamente al instalar dichas dependencias.

A continuación, se adjunta la Tabla [D.1](#) a modo resumen de las funcionalidades que abarca cada módulo.

Directorio/Fichero	Descripción
<i>nucleo</i>	Configuración principal del proyecto Django.
<i>aulas</i>	Gestión de aulas, capacidad, ubicación y recursos disponibles.
<i>calendario</i>	Gestión del calendario académico, días lectivos, festivos, exámenes y cambios de docencia.
<i>docencia</i>	Gestión de grados, asignaturas, grupos y docentes y horarios.
<i>app_horarios</i>	Importación y exportación base de datos.
<i>reservas</i>	Gestión de reservas puntuales y analizador léxico.
<i>usuarios</i>	Gestión de autenticación, usuarios, roles y permisos.
<i>manage.py</i>	Archivo principal para ejecutar comandos de administración de Django.
<i>requirements.txt</i>	Fichero con las dependencias Python necesarias para el backend.
<i>package.json</i>	Fichero de configuración de dependencias de Node.js.
<i>node_modules</i>	Directorio generado automáticamente al instalar dependencias de Node.js.

Tabla D.1: Descripción de la estructura principal del backend

Frontend

El *frontend* de la aplicación se encuentra desarrollado con React + Vite y contiene la parte visual del sistema. Esta parte del proyecto se encarga de mostrar las pantallas de la aplicación, gestionar la interacción con el usuario y comunicarse con el *backend* mediante peticiones HTTP.

En la Figura [D.2](#) se muestra la estructura principal de directorios correspondiente al *frontend* del proyecto.

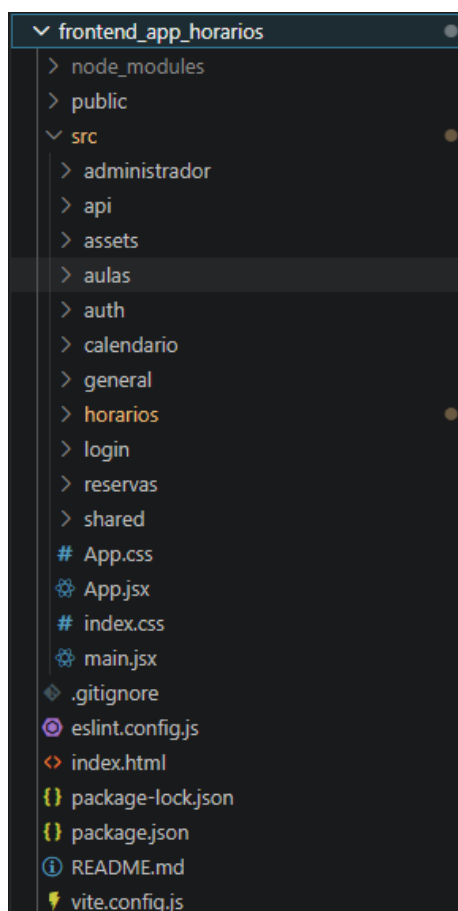


Figura D.2: Estructura de directorios del frontend

Como se observa en la Figura D.2, el directorio del *frontend* está formado por una carpeta principal *src*, donde se encuentra el código fuente de la interfaz, y por varios ficheros de configuración necesarios para ejecutar y construir la aplicación.

El directorio *public* contiene recursos estáticos públicos que pueden ser utilizados directamente por la aplicación. Estos archivos se sirven sin necesidad de ser procesados por el sistema de construcción.

Dentro de *src* se organiza la mayor parte del código del *frontend*.

El directorio *administrador* agrupa las pantallas y componentes relacionados con el panel de administración. En este módulo se encuentran las funcionalidades destinadas a la gestión de entidades, usuarios, roles y el mantenimiento global del sistema.

El directorio *api* contiene las funciones encargadas de centralizar la comunicación con el *backend*. Esta organización permite separar las llamadas HTTP de los componentes visuales, facilitando el mantenimiento del código y evitando duplicidades.

El directorio *assets* almacena recursos estáticos utilizados por la interfaz, como imágenes, logotipos u otros elementos gráficos.

El directorio *aulas* agrupa los componentes y páginas relacionados con la consulta de ocupación de aulas.

El directorio *auth* contiene la lógica para proteger rutas de acceso sin autenticación, y para validar permisos.

El directorio *calendario* agrupa las pantallas y componentes relacionados con la consulta y modificación del calendario académico.

El directorio *horarios* contiene las pantallas y componentes necesarios para cargar nuevos horarios, consultar horarios existentes.

El directorio *login* contiene la pantalla de inicio de sesión y los elementos visuales relacionados con el acceso a la aplicación.

El directorio *reservas* agrupa las pantallas y componentes relacionados con la gestión de reservas puntuales y periódicas, incluyendo listados, formularios de creación y edición, y acciones asociadas a las solicitudes.

Finalmente, el directorio *shared* contiene elementos compartidos por diferentes módulos del *frontend*, permitiendo reutilizar código común y mantener una estructura más organizada.

Además de estos directorios, aparecen varios ficheros relevantes en la raíz del proyecto. El archivo *App.jsx* define la estructura principal de la aplicación React y la organización general de sus rutas o componentes principales. El archivo *main.jsx* actúa como punto de entrada de la aplicación. Los ficheros *App.css* e *index.css* contienen estilos globales utilizados por la interfaz.

El fichero *package.json* recoge la configuración del proyecto, los scripts de ejecución y las dependencias necesarias. El archivo *package-lock.json* almacena la versión exacta de las dependencias instaladas. El archivo *vite.config.js* contiene la configuración de Vite, herramienta utilizada para ejecutar y construir el *frontend*. Por último, el directorio *node_modules* se genera automáticamente al instalar las dependencias de Node.js y no forma parte del código fuente desarrollado.

Se ha realizado una Tabla [D.2](#) a modo resumen visual de lo que realiza cada directorio.

Directorio/Fichero	Descripción
<i>public</i>	Recursos estáticos públicos de la aplicación.
<i>src</i>	Directorio principal del código fuente del frontend.
<i>administrador</i>	Pantallas y componentes relacionados con el panel de administración.
<i>api</i>	Funciones encargadas de realizar peticiones al backend.
<i>assets</i>	Imágenes, logotipos y otros recursos gráficos.
<i>aulas</i>	Componentes relacionados con aulas y ocupación de espacios.
<i>auth</i>	Lógica de autenticación y gestión de sesión.
<i>calendario</i>	Componentes relacionados con el calendario académico.
<i>general</i>	Componentes comunes de la interfaz.
<i>horarios</i>	Pantallas de carga, consulta y gestión de horarios.
<i>login</i>	Pantalla y componentes de inicio de sesión.
<i>reservas</i>	Gestión de reservas puntuales.
<i>shared</i>	Componentes o utilidades compartidas por varios módulos.
<i>App.jsx</i>	Componente principal de la aplicación React.
<i>main.jsx</i>	Punto de entrada del frontend.
<i>vite.config.js</i>	Configuración de Vite.
<i>package.json</i>	Dependencias y scripts del proyecto frontend.
<i>node_modules</i>	Dependencias instaladas automáticamente mediante Node.js.

Tabla D.2: Descripción de la estructura principal del frontend

D.3. Manual del programador

Esta sección está dirigida a futuros desarrolladores que necesiten mantener, ampliar o modificar la aplicación. En ella se describen los aspectos más relevantes del desarrollo, la organización interna del código y las partes principales que deben conocerse antes de realizar cambios en el sistema.

La aplicación está formada por dos bloques principales: el *backend*, desarrollado con Django, y el *frontend*, desarrollado con React. El *backend* se encarga de la lógica de negocio, la autenticación, la comunicación con la base de datos y la validación de las operaciones. El *frontend*, por su

parte, proporciona la interfaz visual y se comunica con el servidor mediante peticiones HTTP.

Organización interna del backend

Como se ha indicado en la sección anterior, el *backend* se encuentra dividido en varios módulos funcionales, como *aulas*, *calendario*, *docencia*, *reservas* y *usuarios*. Cada uno de estos módulos agrupa la lógica correspondiente a una parte concreta del sistema, lo que permite separar responsabilidades y facilita el mantenimiento del código.

Dentro de estos módulos se ha seguido una organización basada en distintos tipos de ficheros, entre los que destacan los modelos, vistas, serializadores y servicios.

Modelos

Los modelos representan las entidades de la base de datos dentro de Django. Cada modelo define los campos de una tabla, sus tipos de datos, relaciones y restricciones principales. Gracias a estos modelos, el *backend* puede trabajar con la información persistente de la aplicación sin necesidad de escribir directamente todas las consultas SQL.

En este proyecto, los modelos representan elementos como aulas, docentes, asignaturas, grupos, reservas, cursos, semestres y días del calendario académico. Cualquier modificación en la estructura de datos debe revisarse cuidadosamente, ya que puede afectar a las vistas, serializadores, servicios y pantallas del *frontend* que dependan de dicha información.

Vistas

Las vistas son las encargadas de recibir las peticiones procedentes del *frontend*, procesarlas y devolver una respuesta. En ellas se definen los puntos de entrada de la API y se controla qué operación debe ejecutarse en función de la petición recibida.

El *frontend* envía una petición HTTP al *backend*. La vista correspondiente recibe esa petición, comprueba los datos recibidos y coordina el resto de operaciones necesarias.

Aunque las vistas pueden contener validaciones sencillas, se ha procurado que la lógica más compleja se separe en servicios auxiliares. De esta forma, las vistas se mantienen más claras y se evita concentrar demasiada lógica en un único fichero.

Serializadores

Los serializadores se utilizan para transformar los datos entre el formato interno de Django y el formato intercambiado con el *frontend*, normalmente JSON. Su función es especialmente importante en una aplicación con *frontend* y *backend* desacoplados, ya que permiten controlar qué información se envía al cliente y cómo se reciben los datos introducidos por el usuario.

Además de convertir objetos en respuestas JSON, los serializadores permiten realizar validaciones sobre los datos recibidos antes de crear o actualizar registros en la base de datos. Por ejemplo, pueden comprobar que los campos obligatorios estén presentes, que los tipos de datos sean correctos o que una estructura enviada desde el *frontend* cumpla el formato esperado.

Servicios

Los servicios agrupan funciones auxiliares o procesos de negocio que no deben estar directamente dentro de las vistas. Esta separación permite reutilizar código y mantener una estructura más organizada.

En este proyecto, los servicios son especialmente importantes en procesos como la carga de horarios desde hojas de cálculo, la validación de disponibilidad de aulas, la comprobación de solapes entre reservas y el tratamiento del calendario académico.

Por ejemplo, en lugar de implementar toda la lógica de lectura de Excel dentro de una vista, esta se puede delegar en funciones o clases de servicio encargadas de procesar el fichero, extraer la información relevante y devolver los datos estructurados para que posteriormente sean validados y almacenados.

Rutas

Las rutas definen las direcciones de la API disponibles para el *frontend*. Cada ruta se asocia con una vista o conjunto de vistas que implementan una funcionalidad concreta. Mantener las rutas organizadas por módulos facilita localizar los puntos de entrada de cada parte del sistema.

Si en el futuro se añade una nueva funcionalidad, será necesario definir su ruta correspondiente, implementar la vista asociada, crear o reutilizar los serializadores necesarios y, en caso de que exista lógica compleja, trasladarla a un servicio específico.

Organización interna del frontend

El *frontend* está desarrollado con React y se organiza en módulos funcionales relacionados con las principales partes de la aplicación, como reservas, horarios, calendario, aulas, autenticación y administración.

Cada módulo contiene componentes y pantallas relacionados con una funcionalidad concreta. Esta organización permite que el código sea más mantenible y que los cambios realizados en una parte de la interfaz no afecten directamente al resto del sistema.

Componentes

Los componentes son la base de la interfaz en React. Cada componente representa una parte visual o funcional de la aplicación, como formularios, tablas, botones, modales, menús o pantallas completas.

En este proyecto se han utilizado componentes para construir pantallas como la gestión de reservas puntuales, la ocupación de aulas, el calendario académico, la carga de horarios o el panel de administración. La reutilización de componentes permite evitar duplicidades y mantener una apariencia coherente en toda la aplicación.

Estado de los componentes

Para gestionar la información interna de los componentes se ha utilizado el *hook* *useState*. Este mecanismo permite almacenar valores que pueden cambiar durante la ejecución de la aplicación, como filtros seleccionados, datos de formularios, mensajes de error, resultados de una consulta o el estado de apertura de un modal.

Efectos y carga de datos

Para ejecutar acciones asociadas al ciclo de vida de los componentes se ha utilizado el *hook* *useEffect*. Este mecanismo permite realizar operaciones cuando se carga una pantalla o cuando cambia algún dato del que depende el componente.

En este proyecto, *useEffect* se utiliza principalmente para cargar información inicial desde el *backend*. También puede emplearse para actualizar la información mostrada cuando el usuario modifica filtros o selecciona una nueva opción.

De esta forma, las pantallas pueden solicitar datos al *backend* en el momento adecuado y actualizar la interfaz cuando la respuesta ha sido recibida.

Comunicación con el backend

La comunicación entre el *frontend* y el *backend* se realiza mediante peticiones HTTP. Para evitar duplicar código, las llamadas a la API se centralizan en ficheros específicos dentro del directorio *api*.

Estos ficheros contienen funciones encargadas de realizar operaciones concretas, como consultar reservas, crear una reserva puntual, modificar una reserva periódica, cargar un horario o actualizar un día del calendario académico. Los componentes de React llaman a estas funciones cuando necesitan comunicarse con el servidor.

Esta organización permite separar la lógica visual de la lógica de comunicación. Si en el futuro cambia una ruta del *backend*, solo será necesario actualizar el fichero correspondiente de la API, sin modificar todas las pantallas que utilizan esa operación.

Gestión de formularios y eventos

Las pantallas que permiten crear o editar información utilizan formularios controlados mediante estado. Cada campo del formulario se asocia a una variable de estado y se actualiza mediante eventos, como *onChange* o *onSubmit*.

Cuando el usuario confirma una operación, el componente recoge los valores almacenados en el estado, realiza las comprobaciones necesarias en la interfaz y envía los datos al *backend*. Posteriormente, en función de la respuesta recibida, se muestra un mensaje de confirmación o de error.

Control de autenticación y permisos

El sistema diferencia las funcionalidades disponibles en función del rol asociado al usuario autenticado. Por este motivo, el programador debe tener en cuenta que algunas pantallas, botones o acciones solo deben mostrarse a determinados perfiles.

En el *frontend*, esta restricción se utiliza para adaptar la interfaz al rol del usuario, ocultando opciones que no le corresponden. Sin embargo, estas comprobaciones no son suficientes por sí solas, ya que un usuario podría intentar acceder manualmente a una ruta o enviar una petición no permitida.

Por este motivo, las validaciones importantes deben realizarse también en el *backend*, que es el encargado de garantizar que cada operación solo pueda ser ejecutada por usuarios autorizados.

D.4. Compilación, instalación y ejecución del proyecto

En esta sección se describen los pasos necesarios para instalar y ejecutar el proyecto en un entorno local de desarrollo. La aplicación está formada por dos partes principales: el *backend*, desarrollado con Django, y el *frontend*, desarrollado con React y Vite. Además, es necesario disponer de una base de datos MySQL correctamente configurada.

El proceso de instalación consiste en preparar el entorno del *backend*, instalar las dependencias necesarias, configurar la conexión con la base de datos, preparar el entorno del *frontend* y ejecutar ambos servidores de desarrollo.

Requisitos previos

Antes de ejecutar el proyecto, es necesario tener instaladas las siguientes herramientas:

- **Python:** necesario para ejecutar el proyecto Django.
- **pip:** gestor de paquetes de Python.
- **Node.js:** necesario para ejecutar el proyecto React.
- **npm:** gestor de paquetes de Node.js.
- **MySQL:** sistema gestor de base de datos utilizado por la aplicación.
- **Git:** recomendado para clonar el repositorio y gestionar el código fuente.

Además, se recomienda utilizar un entorno virtual de Python para aislar las dependencias del proyecto y evitar conflictos con otras aplicaciones instaladas en el equipo.

Obtención del código fuente

En primer lugar, se debe obtener el código fuente del proyecto desde el repositorio correspondiente. Para ello, se puede clonar el repositorio mediante Git:

```
git clone <URL_DEL_REPOSITORIO>
```

Una vez clonado el repositorio, se accede al directorio principal del proyecto:

```
cd Trabajo-Fin-de-Grado-2025-2026
```

En caso de disponer del proyecto comprimido en un fichero, bastará con descomprimirlo y acceder a la carpeta raíz del proyecto.

Configuración de la base de datos

La aplicación utiliza MySQL como sistema gestor de base de datos. Antes de ejecutar el *backend*, es necesario crear una base de datos vacía y configurar las credenciales de acceso en el proyecto Django.

Desde MySQL, se puede crear la base de datos con un comando similar al siguiente:

```
CREATE DATABASE nombre_base_datos;
```

A continuación, se debe comprobar que el usuario configurado dispone de permisos suficientes sobre dicha base de datos. Estos datos de conexión deberán coincidir con los valores definidos en la configuración del proyecto.

En caso de partir de un script SQL inicial, este deberá ejecutarse sobre la base de datos creada para generar las tablas necesarias o cargar datos iniciales.

```
mysql -u usuario -p nombre_base_datos < BaseDeDatosHorario.sql
```

Instalación y ejecución del backend

Para instalar el *backend*, se accede al directorio en el que se encuentra el proyecto Django:

```
cd app_horarios
```

A continuación, se crea un entorno virtual de Python:

```
python -m venv .venv
```

En Windows, el entorno virtual se activa con el siguiente comando:

```
.venv\Scripts\activate
```

Una vez activado el entorno virtual, se instalan las dependencias necesarias a partir del fichero *requirements.txt*:

```
pip install -r requirements.txt
```

Antes de iniciar el servidor, se debe revisar la configuración del proyecto, especialmente los datos de conexión a la base de datos, la clave secreta de Django, el modo de depuración y los parámetros relacionados con la autenticación institucional.

Como el proyecto utiliza migraciones de Django para la gestión de usuarios, roles y permisos, se aplican mediante los siguientes comandos:

```
python manage.py mmakemigrations
```

```
python manage.py migrate
```

Finalmente, se inicia el servidor de desarrollo del *backend*:

```
python manage.py runserver
```

Si la ejecución se realiza correctamente, el servidor quedará disponible por defecto en la siguiente dirección:

```
http://127.0.0.1:8000/
```

Una se ha inicializado el proyecto la primera vez correctamente, si el sistema operativo es Windows, puede ejecutar el archivo *runserver.ps1* que inicia el servidor desde la carpeta principal del repositorio, al ejecutar los comandos necesarios.

Para instalar el *frontend*, se accede al directorio correspondiente a la aplicación React:

```
cd frontend_app_horarios
```

A continuación, se instalan las dependencias definidas en el fichero *package.json*:

```
npm install
```

Una vez instaladas las dependencias, se puede iniciar el servidor de desarrollo de Vite:

```
npm run dev
```

Si la ejecución se realiza correctamente, la aplicación indicará en la consola la dirección local en la que se encuentra disponible. Habitualmente, será una dirección similar a la siguiente:

```
http://localhost:5173/
```

Desde esta dirección se podrá acceder a la interfaz web de la aplicación mediante un navegador.

Ejecución completa del sistema

Para que la aplicación funcione correctamente en local, deben estar activos al mismo tiempo la base de datos, el servidor del *backend* y el servidor del *frontend*.

El orden recomendado de ejecución es el siguiente:

1. Iniciar el servicio de MySQL.
2. Comprobar que la base de datos existe y que las credenciales de conexión son correctas.
3. Activar el entorno virtual del *backend*.
4. Ejecutar el servidor de Django mediante *python manage.py runserver*.
5. Instalar, si no se ha hecho previamente, las dependencias del *frontend* mediante *npm install*.
6. Ejecutar el servidor de desarrollo del *frontend* mediante *npm run dev*.
7. Acceder a la aplicación desde el navegador mediante la URL indicada por Vite.

Durante la ejecución, el *frontend* enviará peticiones HTTP al *backend*, y este se encargará de procesarlas, aplicar la lógica de negocio y consultar o modificar la base de datos cuando sea necesario.

Compilación del frontend para producción

En caso de querer generar una versión optimizada del *frontend* para despliegue, se puede ejecutar el siguiente comando dentro del directorio del *frontend*:

```
npm run build
```

Este comando genera una versión compilada de la aplicación en el directorio de salida configurado por Vite, normalmente denominado *dist*. Esta versión contiene los ficheros estáticos optimizados para su despliegue en un servidor.

Consideraciones sobre variables de entorno

Para evitar incluir información sensible directamente en el código fuente, se recomienda utilizar variables de entorno para almacenar datos de configuración. Entre los valores que pueden configurarse mediante variables de entorno se encuentran:

- Clave secreta de Django.

- Nombre de la base de datos.
- Usuario y contraseña de la base de datos.
- Host y puerto de MySQL.

El uso de variables de entorno facilita el despliegue en distintos entornos, como desarrollo local, pruebas o producción, sin necesidad de modificar directamente el código fuente.

D.5. Pruebas del sistema

En esta sección se recogen las pruebas realizadas para comprobar el correcto funcionamiento de algunas de las funcionalidades principales de la aplicación. El objetivo de estas pruebas es verificar que el sistema responde de forma adecuada ante distintos escenarios, tanto correctos como erróneos, y que la lógica interna implementada en el *backend* mantiene la coherencia de los datos.

Durante el desarrollo del proyecto se han implementado pruebas unitarias. Las pruebas unitarias son aquellas que permiten comprobar el funcionamiento de una parte concreta del sistema de forma aislada, normalmente una función, método o servicio específico. Este tipo de pruebas resultan útiles para validar que una unidad de código produce el resultado esperado ante unos datos de entrada determinados.

Las pruebas se han desarrollado utilizando la clase *TestCase* de Django. Esta clase permite crear una base de datos de prueba independiente, de forma que los datos utilizados durante la ejecución de los tests no afectan a la base de datos real de la aplicación. Además, en algunos casos se han utilizado *mocks* mediante *unittest.mock.patch*, lo que permite simular el comportamiento de determinadas funciones auxiliares y comprobar cómo responde el servicio principal ante distintos resultados.

Casos de prueba unitarios

En este apartado se van a presentar algunos de los casos de prueba implementados. Para cada caso de prueba se ha definido un identificador, título, prioridad, descripción, precondiciones, pasos de ejecución, resultado esperado, resultado obtenido y estado final. Estos campos han sido seleccionado parecidos a los que se utilizaron en la asignatura de Validación y Pruebas de cuarto.

Estos casos se han seleccionado porque cubren las funcionalidades más críticas del sistema, especialmente aquellas relacionadas con la generación de reservas periódicas, la validación de restricciones, la carga de horarios desde hoja de cálculo y el análisis de la información extraída del fichero.

Campo	Descripción
ID	CP-01
Título	Validación de restricciones con conflicto de aula y grupo
Prioridad	Alta
Descripción	Comprobar que el sistema detecta correctamente los conflictos cuando se intenta mover una reserva periódica a una franja horaria en la que el aula está ocupada y el grupo ya tiene otra docencia asignada.
Precondición	Debe existir una reserva periódica previa asociada a un aula, grupo, día y franja horaria concreta.
Postcondición	La reserva periódica no se modifica, ya que la validación detecta conflictos.
Pasos	1. Crear los datos de prueba necesarios: curso, semestre, día, grado, asignatura, grupo, aula y reserva periódica. 2. Preparar una firma de serie válida para identificar la reserva. 3. Simular mediante <i>mocks</i> que la validación de ocupación de aula falla. 4. Simular mediante <i>mocks</i> que la validación de solapamiento de grupo falla. 5. Ejecutar la función <i>validar_restricciones_movimiento</i>. 6. Comprobar que el resultado indica que la modificación no es válida.
Datos de prueba	Reserva periódica del grupo A1 en el aula A-01, lunes de 09:00 a 11:00. Movimiento simulado al martes de 10:00 a 12:00.
Resultado esperado	El sistema debe devolver la validación como incorrecta e indicar los motivos del conflicto.
Resultado actual	El sistema devuelve la validación como incorrecta e incluye los mensajes de aula ocupada y grupo solapado.
Estado	Correcto

Tabla D.3: Caso de prueba CP-01: validación de restricciones con conflicto

Campo	Descripción
ID	CP-02
Título	Validación correcta de restricciones
Prioridad	Alta
Descripción	Comprobar que el sistema permite la modificación de una reserva periódica cuando no existen conflictos de aula ni solapamientos de grupo.
Precondición	Debe existir una reserva periódica previa y una firma válida que permita identificarla.
Postcondición	La operación se considera válida y puede ejecutarse la modificación de la serie de reservas.
Pasos	1. Crear una reserva periódica asociada a un grupo y aula. 2. Preparar una firma válida de la serie de reservas. 3. Simular mediante <i>mocks</i> que el aula está disponible. 4. Simular mediante <i>mocks</i> que el grupo no tiene solapamientos. 5. Ejecutar la función <i>validar_restricciones_movimiento</i>. 6. Comprobar que el resultado de la validación es correcto.
Datos de prueba	Movimiento de la reserva del grupo 101 al martes de 10:00 a 12:00 sin conflictos simulados.
Resultado esperado	El sistema debe indicar que la modificación es válida y no devolver motivos de error.
Resultado actual	El sistema devuelve la validación como correcta y la lista de motivos aparece vacía.
Estado	Correcto

Tabla D.4: Caso de prueba CP-02: validación correcta de restricciones

Campo	Descripción
ID	CP-03
Título	Movimiento correcto de una serie de reservas periódicas
Prioridad	Alta
Descripción	Comprobar que el sistema actualiza correctamente una serie de reservas periódicas cuando se modifica su día de la semana y su franja horaria.
Precondición	Debe existir una reserva periódica registrada en la base de datos de prueba y una firma válida que permita identificar la serie.
Postcondición	La reserva periódica queda actualizada con el nuevo día de la semana y la reserva asociada queda actualizada con la nueva franja horaria.
Pasos	1. Crear una reserva periódica asociada a una reserva base. 2. Preparar una firma válida para identificar la serie. 3. Simular el cálculo de nuevas fechas mediante un <i>mock</i>. 4. Ejecutar la función <i>mover_serie_reservas</i>. 5. Refrescar los objetos desde la base de datos. 6. Comprobar que el día de la semana se ha actualizado. 7. Comprobar que la hora de inicio y la hora de fin se han actualizado correctamente.
Datos de prueba	Reserva original el lunes de 09:00 a 11:00. Nueva reserva el martes de 10:00 a 12:00.
Resultado esperado	La reserva periódica debe cambiar al día 2 y la reserva asociada debe actualizar sus horas a 10:00 y 12:00.
Resultado actual	La reserva periódica cambia correctamente al día 2 y la reserva asociada actualiza su hora de inicio y fin.
Estado	Correcto

Tabla D.5: Caso de prueba CP-03: movimiento correcto de una serie de reservas

Campo	Descripción
ID	CP-04
Título	Generación correcta de reservas desde hoja de cálculo
Prioridad	Alta
Descripción	Comprobar que el sistema genera correctamente reservas periódicas a partir de una clase extraída de una hoja de cálculo.
Precondición	Deben existir en la base de datos de prueba el curso académico, semestre, día lectivo, aula, grado y asignatura correspondientes.
Postcondición	Se crean las reservas periódicas necesarias, se crea el grupo si no existía previamente y el curso queda marcado como horario cargado.
Pasos	1. Crear el calendario académico de prueba con un día lectivo, un festivo y un cambio de docencia. 2. Crear el aula, grado y asignatura necesarios. 3. Simular una clase leída del Excel mediante un objeto <i>Clase</i>. 4. Ejecutar la función <i>generar_reservas_periodicas</i>. 5. Comprobar que el curso queda marcado como cargado. 6. Comprobar que se crea el grupo correspondiente. 7. Comprobar que se crea una reserva en el día lectivo. 8. Comprobar que se tiene en cuenta el cambio de docencia.
Datos de prueba	Clase de Álgebra, código 8797, grupo 1, aula Aula 51-A1, lunes de 09:00 a 11:00.
Resultado esperado	El sistema debe crear el grupo, generar las reservas periódicas correspondientes y marcar el curso como horario cargado.
Resultado actual	El sistema crea el grupo, genera la reserva del lunes, genera la reserva correspondiente al cambio de docencia y marca el curso como cargado.
Estado	Correcto

Tabla D.6: Caso de prueba CP-04: generación de reservas desde hoja de cálculo

Campo	Descripción
ID	CP-05
Título	Carga de Excel con aula inexistente
Prioridad	Alta
Descripción	Comprobar que el sistema no genera reservas cuando la clase extraída del Excel hace referencia a un aula que no existe en la base de datos.
Precondición	Debe existir la asignatura correspondiente, pero no debe existir el aula indicada en la clase extraída del Excel.
Postcondición	No se crea ninguna reserva en la base de datos de prueba.
Pasos	1. Crear el curso, semestre, día lectivo, grado y asignatura necesarios. 2. Crear una clase procedente del Excel con un aula inexistente. 3. Ejecutar la función <i>generar_reservas_periodicas</i>. 4. Comprobar el número de reservas creadas.
Datos de prueba	Clase de Álgebra con aula “Aula Inexistente”.
Resultado esperado	El sistema no debe crear ninguna reserva.
Resultado actual	El número de reservas creadas es 0.
Estado	Correcto

Tabla D.7: Caso de prueba CP-05: carga de Excel con aula inexistente

Campo	Descripción
ID	CP-06
Título	Detección de aula ocupada
Prioridad	Alta
Descripción	Comprobar que el sistema detecta correctamente que un aula ya está ocupada cuando se intenta crear o mover una reserva a una franja horaria solapada.
Precondición	Debe existir una reserva periódica previa en el aula y franja horaria que se desea validar.
Postcondición	La validación devuelve resultado negativo y no se permite utilizar el aula en esa franja.
Pasos	1. Crear un aula y una reserva periódica asociada a ella. 2. Ejecutar la función <i>validar_ocupacion_aula</i> con una franja horaria solapada. 3. Comprobar que la función devuelve <i>False</i>. 4. Comprobar que el mensaje de error indica que el aula está ocupada.
Datos de prueba	Aula 51-A1 ocupada de 09:00 a 11:00. Nueva validación de 10:00 a 12:00.
Resultado esperado	El sistema debe indicar que el aula seleccionada ya está ocupada en ese horario.
Resultado actual	El sistema devuelve la validación como incorrecta y muestra el mensaje de aula ocupada.
Estado	Correcto

Tabla D.8: Caso de prueba CP-06: detección de aula ocupada

Campo	Descripción
ID	CP-07
Título	Validación de aula disponible por cambio de horario
Prioridad	Media
Descripción	Comprobar que el sistema considera disponible un aula cuando la nueva franja horaria no se solapa con reservas existentes.
Precondición	Debe existir una reserva previa en el aula seleccionada, pero en una franja horaria diferente.
Postcondición	La validación devuelve resultado positivo y el aula se considera disponible.
Pasos	1. Crear una reserva periódica previa en un aula. 2. Ejecutar la función <i>validar_ocupacion_aula</i> con una franja horaria no solapada. 3. Comprobar que la función devuelve <i>True</i>. 4. Comprobar que no se devuelve mensaje de error.
Datos de prueba	Aula 51-A1 ocupada de 09:00 a 11:00. Nueva validación de 11:00 a 13:00.
Resultado esperado	El sistema debe considerar el aula disponible.
Resultado actual	El sistema devuelve la validación como correcta y sin mensaje de error.
Estado	Correcto

Tabla D.9: Caso de prueba CP-07: validación de aula disponible

Campo	Descripción
ID	CP-08
Título	Clasificación de celdas mediante expresiones regulares
Prioridad	Alta
Descripción	Comprobar que el analizador léxico clasifica correctamente las celdas procedentes de la hoja de cálculo según su contenido.
Precondición	Debe estar disponible la función <i>clasificar_celda</i> utilizada por el analizador del Excel.
Postcondición	Cada valor analizado se clasifica con el tipo correspondiente.
Pasos	1. Ejecutar la función <i>clasificar_celda</i> con una celda vacía. 2. Ejecutar la función con palabras clave como “Hora” o “Nombre Abreviado”. 3. Ejecutar la función con días de la semana. 4. Ejecutar la función con tramos horarios. 5. Ejecutar la función con grupos teóricos y prácticos. 6. Ejecutar la función con aulas, códigos y abreviaturas de asignaturas. 7. Comprobar que cada celda recibe la clasificación esperada.
Datos de prueba	Valores como “Hora”, “lunes”, “09:00 - 11:00”, “1”, “801”, “Aula 51-A1”, “8797” y “ALG”.
Resultado esperado	El sistema debe clasificar cada celda como comienzo de tabla, día de semana, tramo horario, grupo, aula, código o abreviatura según corresponda.
Resultado actual	Las celdas se clasifican correctamente según las reglas definidas en el analizador léxico.
Estado	Correcto

Tabla D.10: Caso de prueba CP-08: clasificación de celdas mediante expresiones regulares

Campo	Descripción
ID	CP-09
Título	Normalización de nombres de aulas
Prioridad	Alta
Descripción	Comprobar que el sistema normaliza correctamente los nombres de aulas procedentes de la hoja de cálculo para poder compararlos con los nombres almacenados en la base de datos.
Precondición	Debe estar disponible la función <i>limpiar_nombre_aula</i> .
Postcondición	Los nombres de aulas quedan transformados al formato esperado por la aplicación.
Pasos	1. Ejecutar la función con un valor vacío. 2. Ejecutar la función con un aula con prefijo “Aula”. 3. Ejecutar la función con un nombre que no debe modificarse, como Aula Empresa”. 4. Ejecutar la función con abreviaturas como “Inf.”. 5. Ejecutar la función con laboratorios abreviados como “Lab.”. 6. Comprobar que el resultado coincide con el formato normalizado esperado.
Datos de prueba	Valores como <i>None</i> , “Aula 51-A1”, “Aula Empresa”, “Inf. 1”, “Lab. Química” y “Lab 12-A1”.
Resultado esperado	El sistema debe devolver nombres normalizados como “51-A1”, “Aula Empresa”, “Informática 1” y “Laboratorio Química”.
Resultado actual	Los nombres de aulas se normalizan correctamente según las reglas definidas.
Estado	Correcto

Tabla D.11: Caso de prueba CP-09: normalización de nombres de aulas

Apéndice E

Documentación de usuario

E.1. Introducción

Este apéndice recoge la documentación dirigida a los usuarios finales de la aplicación. Su objetivo es explicar los requisitos necesarios para utilizar el sistema, el proceso de acceso a la aplicación y el funcionamiento de las principales funcionalidades disponibles según el rol del usuario autenticado.

La aplicación desarrollada permite gestionar horarios académicos, reservas de aulas, calendario académico, usuarios y roles dentro del entorno universitario. Para ello, proporciona una interfaz web desde la que los usuarios autorizados pueden consultar información, solicitar reservas puntuales, cargar horarios desde hojas de cálculo, modificar reservas periódicas y gestionar la planificación académica.

El acceso a las distintas funcionalidades depende del rol asignado a cada usuario. De esta forma, no todos los usuarios disponen de las mismas opciones dentro de la aplicación. Por ejemplo, un usuario con rol de administrador puede gestionar usuarios, roles y entidades del sistema, mientras que un usuario PDI puede consultar información relativa a sus propias reservas y solicitar reservas puntuales.

E.2. Requisitos de usuarios

Para utilizar la aplicación no es necesario instalar ningún programa adicional en el equipo del usuario, ya que se trata de una aplicación web accesible desde un navegador. No obstante, se deben cumplir una serie de requisitos mínimos para garantizar su correcto funcionamiento.

Requisitos hardware

El usuario únicamente necesita disponer de un equipo con conexión a Internet. La aplicación puede utilizarse desde un ordenador o portátil, o dispositivo móvil, siempre que permita ejecutar un navegador web actualizado.

Los requisitos mínimos recomendados son los siguientes:

- Dispositivo móvil.
- Conexión estable a Internet.
- Pantalla con resolución suficiente para visualizar correctamente tablas, formularios y calendarios.
- Teclado y ratón o dispositivo equivalente para interactuar con la interfaz.

Requisitos software

Al tratarse de una aplicación web, el principal requisito software es disponer de un navegador actualizado. Se recomienda utilizar alguno de los navegadores más habituales, como Google Chrome, Mozilla Firefox, Microsoft Edge o Safari.

Los requisitos software son los siguientes:

- Navegador web actualizado.
- Acceso a Internet.
- Credenciales institucionales de la Universidad de Burgos.
- Permisos adecuados dentro de la aplicación según el tipo de usuario.

Para las funcionalidades relacionadas con la carga de horarios, el usuario deberá disponer además del fichero de hoja de cálculo correspondiente en el formato esperado por la aplicación.

E.3. Instalación

Desde el punto de vista del usuario final, la aplicación no requiere instalación local. El sistema se encuentra desplegado en un servidor externo y puede ser utilizado directamente a través de un navegador web.

Para acceder a la aplicación, el usuario deberá introducir la dirección web proporcionada por el administrador del sistema. Una vez cargada la página, se mostrará la pantalla de inicio de sesión, donde será necesario introducir las credenciales institucionales de la Universidad de Burgos.

En caso de que la aplicación se ejecute en un entorno local o de pruebas, será necesario que el administrador o desarrollador haya iniciado previamente los servicios correspondientes al *frontend*, *backend* y base de datos. No obstante, este proceso no forma parte del uso habitual de la aplicación por parte del usuario final.

E.4. Manual del usuario

En esta sección se describen las principales funcionalidades disponibles en la aplicación. Algunas opciones pueden no estar visibles para todos los usuarios, ya que el acceso a cada funcionalidad depende del rol asignado dentro del sistema.

Inicio de sesión

La pantalla de inicio de sesión permite acceder a la aplicación utilizando las credenciales institucionales de la Universidad de Burgos. Para entrar en el sistema, el usuario debe introducir su correo electrónico asociados y la contraseña en los campos correspondientes y debe pulsar el botón de inicio de sesión.

Una vez enviadas las credenciales, la aplicación valida los datos introducidos mediante el sistema de autenticación institucional. Si las credenciales son correctas, el usuario accede a la pantalla principal de la aplicación. En caso contrario, se muestra un mensaje de error indicando que no ha sido posible iniciar sesión.

Además, tras una autenticación correcta, el sistema identifica el rol asociado al usuario para mostrar únicamente las funcionalidades a las que tiene acceso. De esta forma, la interfaz puede variar en función de si el usuario es administrador, gestor editor, gestor no editor o PDI.

En la Figura E.1 se muestra la pantalla de inicio de sesión de la aplicación.

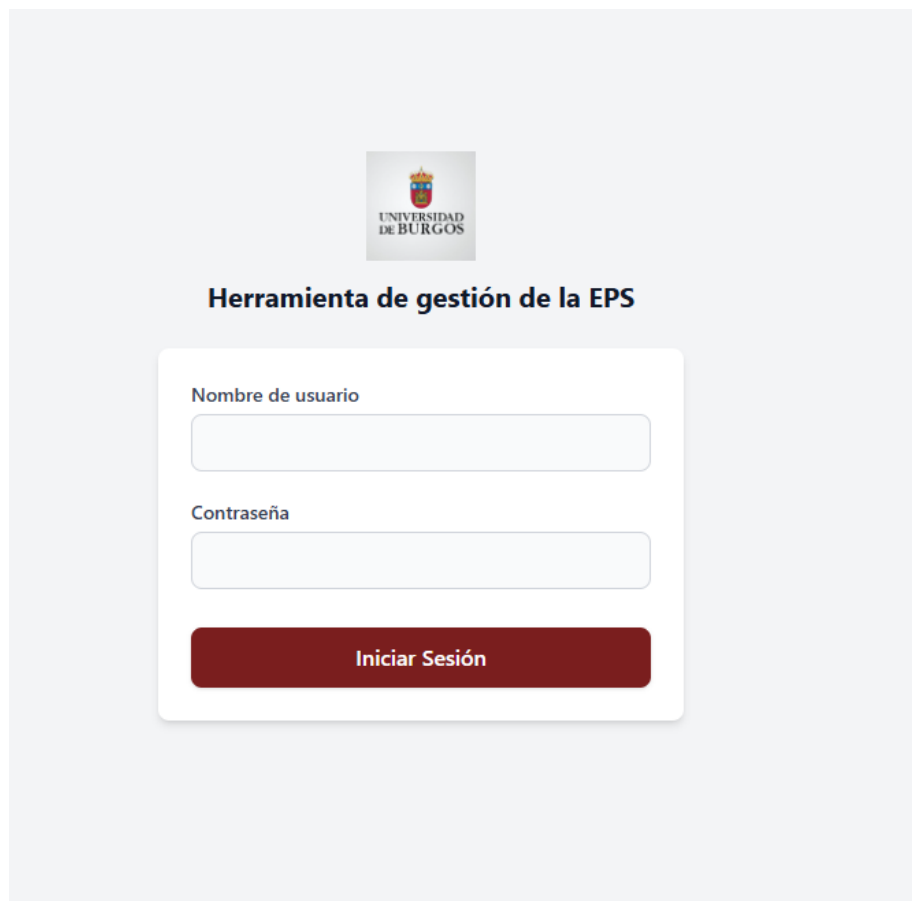
La imagen muestra la interfaz de inicio de sesión de una aplicación. En el centro, sobre un fondo gris claro, hay un formulario blanco con un efecto de sombra. Al principio del formulario está el logo de la Universidad de Burgos, que incluye un escudo con un libro y la inscripción 'UNIVERSIDAD DE BURGOS'. Debajo del logo, el título 'Herramienta de gestión de la EPS' aparece en un color azul oscuro. El formulario contiene dos campos de entrada: el primero está etiquetado 'Nombre de usuario' y el segundo 'Contraseña'. Ambos campos son rectángulos blancos con una fina línea gris. Debajo de estos campos, hay un botón rectangular de color rojo oscuro con el texto 'Iniciar Sesión' en blanco.

Figura E.1: Pantalla de inicio de sesión

Estructura general de la interfaz

Una vez iniciado sesión, el usuario accede a la interfaz principal de la aplicación. Esta interfaz mantiene una estructura común en las distintas pantallas, formada por un encabezado superior, un menú lateral de navegación y una zona central de contenido.

En la parte superior de la pantalla se encuentra el encabezado principal. Este encabezado muestra el logotipo de la Universidad de Burgos, un mensaje de bienvenida con el nombre del usuario autenticado y el botón de cierre de sesión. Mediante este botón, el usuario puede finalizar su sesión de forma segura y volver a la pantalla de inicio de sesión.

En la parte izquierda de la interfaz se encuentra el menú lateral de navegación. Este menú permite acceder a los principales módulos de la aplicación, como reservas, horarios, calendarios, ocupación de aulas y administración. Algunos apartados pueden desplegarse mediante el icono correspondiente, mostrando opciones adicionales relacionadas con la funcionalidad seleccionada. Cuando se despliega un apartado se cierra otro para que no quede un menú abierto muy extenso.

Las opciones visibles en el menú dependen del rol asociado al usuario autenticado. Por este motivo, un usuario con permisos de administración podrá visualizar apartados de gestión avanzada, mientras que otros perfiles solo tendrán acceso a las funcionalidades permitidas para su rol.

La zona central de la pantalla se utiliza para mostrar el contenido de cada módulo. En esta área se presentan formularios, listados, calendarios, tablas de ocupación, horarios o paneles de administración, según la opción seleccionada en el menú lateral.

En la Figura E.2 se muestra la estructura general de la interfaz de la aplicación, incluyendo el encabezado superior, el menú lateral y la zona principal de contenido.



Figura E.2: Estructura general de la interfaz de usuario

Gestión de reservas puntuales

La gestión de reservas puntuales permite consultar todas las reservas registradas en la aplicación. Desde esta pantalla, el usuario puede visualizar la información principal compacta de cada reserva, como el motivo, la fecha, la franja horaria, el aula asociada, el responsable, el estado de la reserva, y los recursos solicitados para la reserva.

La pantalla incluye diferentes filtros que facilitan la búsqueda de reservas concretas. Entre ellos se pueden encontrar filtros por motivo, responsable, estado (ver solo las pendientes), y por rango de fechas. Por defecto, al acceder a la página se encontrará activo el filtro de fechas desde el día actual en adelante porque es más útil ver las reservas que hay desde el día actual en

adelante que las reservas que ya han pasado. De esta forma, el usuario puede localizar rápidamente las reservas que necesita revisar o gestionar.

Los usuarios con permisos adecuados pueden acceder desde esta pantalla a las acciones de creación, edición, aprobación, rechazo o eliminación de reservas puntuales. Sin embargo, los usuarios con permisos únicamente de consulta solo podrán visualizar la información disponible. Además, respecto a las acciones a realizar sobre las reservas de forma individual, se permite la selección de varias reservas para llevar a cabo acciones masivas, es decir, sobre más de una de las reservas. Estas acciones serán eliminar, aprobar y rechazar, estas dos últimas solamente si todas las reservas seleccionadas son pendientes, sino no está permitido.

En la Figura E.3 se muestra la pantalla de gestión de reservas puntuales.

Figura E.3: Pantalla de gestión de reservas puntuales

Crear/Editar reserva puntual

La creación o edición de una reserva puntual se realiza mediante un formulario específico. En este formulario el usuario debe introducir la infor-

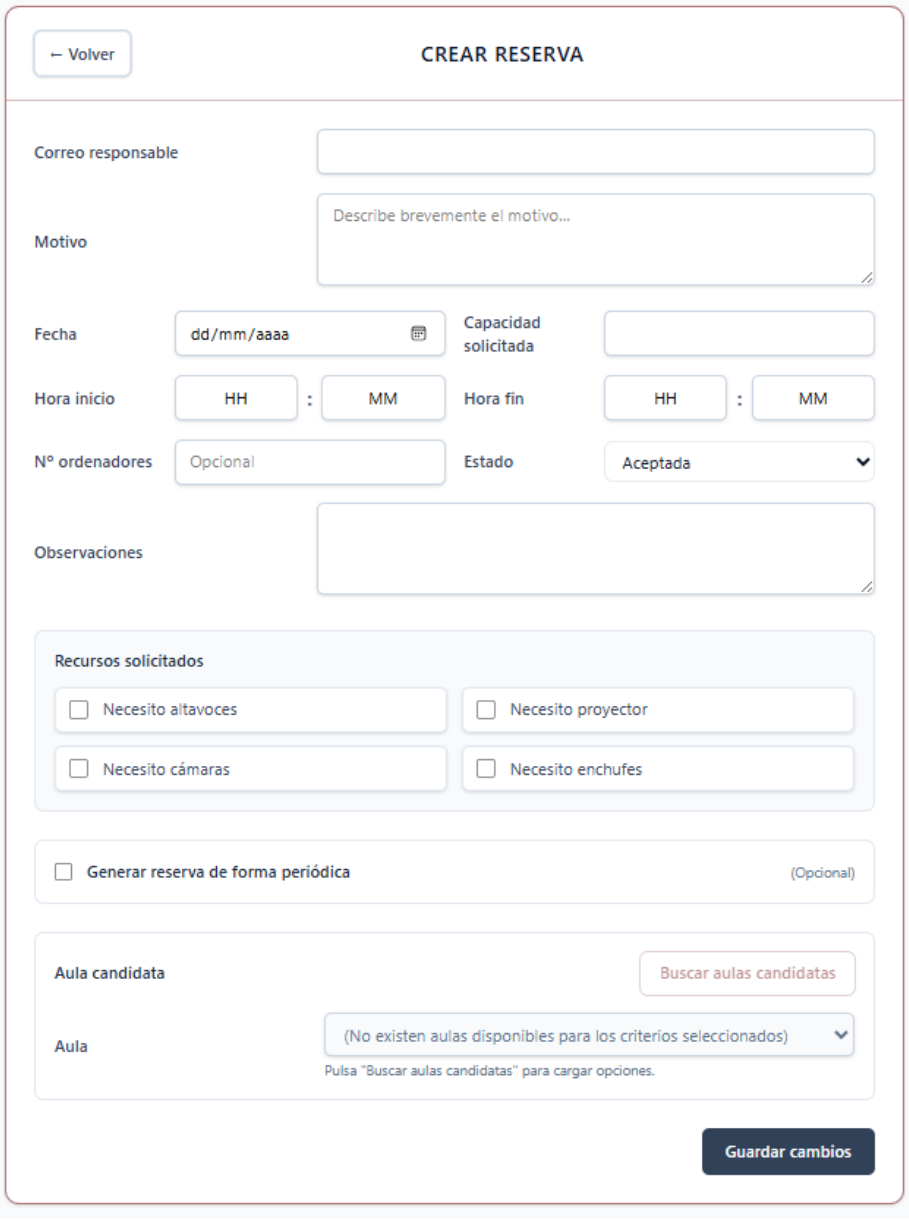
mación necesaria para registrar la reserva, como la fecha, la hora de inicio, la hora de fin, el motivo de la solicitud, la capacidad necesarias y los recursos requeridos. Además al crear o solicitar una reserva se da la opción de generar de forma periódica, por ejemplo si se quiere impartir durante un mes todos los martes, en vez de realizar cuatro solicitudes por separado, te da la opción de generarlas en la misma solicitud.

Entre los recursos que pueden solicitarse se encuentran ordenadores, proyectores, altavoces, cámara o enchufes. Permitiendo en función de estos recursos y de la fecha y hora seleccionada, buscar un aula libre con los recursos solicitados.

Antes de guardar la reserva, el sistema comprueba que los campos obligatorios estén completos, que la hora de fin sea posterior a la hora de inicio y que exista disponibilidad para el aula y franja horaria seleccionadas. Si los datos son correctos, la reserva se registra o actualiza en el sistema. En caso contrario, se muestra un mensaje de error.

La diferencia entre crear una reserva y solicitarla, se basa en que al crear se puede elegir el estado de la reserva, y al solicitar se crea con el estado Pendiente.

En la Figura E.4 se muestra el formulario utilizado para crear o editar una reserva puntual.



Formulario de creación o edición de reserva puntual. El formulario está dividido en varias secciones:

- Correo responsable:** Campo de texto para el correo electrónico.
- Motivo:** Campo de texto para describir brevemente el motivo.
- Fecha:** Selector de fecha con formato dd/mm/aaaa.
- Capacidad solicitada:** Campo de texto para la capacidad solicitada.
- Hora inicio:** Selector de hora con formato HH:MM.
- Hora fin:** Selector de hora con formato HH:MM.
- Nº ordenadores:** Campo de texto con valor predeterminado "Opcional".
- Estado:** Selector de estado con valor predeterminado "Aceptada".
- Observaciones:** Campo de texto para las observaciones.
- Recursos solicitados:** Sección con cuatro campos de texto para solicitar recursos: "Necesito altavoces", "Necesito proyector", "Necesito cámaras" y "Necesito enchufes".
- Generar reserva de forma periódica:** Campo de texto con un checkbox y el texto "(Opcional)".
- Aula candidata:** Sección con un botón "Buscar aulas candidatas" y un selector de aula.
- Aula:** Selector de aula con el texto "(No existen aulas disponibles para los criterios seleccionados)".
- Guardar cambios:** Botón de acción para guardar los cambios.

Figura E.4: Formulario de creación o edición de reserva puntual

En la Figura E.5 se muestra el desplegable en el formulario para realizar reservas puntuales de forma periódica.

The image shows a web form for generating periodic reservations. It is divided into two main sections. The first section, titled 'Recursos solicitados', contains four checkboxes: 'Necesito altavoces', 'Necesito proyector', 'Necesito cámaras', and 'Necesito enchufes'. The second section, titled 'Generar reserva de forma periódica' with a '(Opcional)' label, is expanded. It contains a sub-section 'Configuración de reserva periódica' with fields for 'Fecha inicio' and 'Fecha fin' (both with date pickers), a 'Día de la semana' dropdown menu, and a status message at the bottom stating 'Se generarán 0 reserva(s) con esta configuración.'

Figura E.5: Desplegable para generar reservas puntuales de forma periódica

Ocupación de aulas

La pantalla de ocupación de aulas permite consultar la disponibilidad de una o varias aulas en un periodo determinado. Para realizar la consulta, el usuario debe pulsar el campus al que pertenecen las aulas que quiere ver. Posteriormente selecciona una o varias aulas, y pulsa "Ver Ocupación".

El sistema le redirigirá a una ventana con un calendario a través del cual puede ir pasando los meses, semanas o días depende de la vista escogida para cargar en un rango de fechas las reservas seleccionadas. Además tendrá una leyenda para saber a qué aula pertenece cada reserva, y también tendrá la opción de visualizar solo las reservas puntuales, las periódicas o ambas.

Esta información permite conocer qué tramos están ocupados y cuáles permanecen disponibles, facilitando la planificación de nuevas reservas.

En la Figura E.6 se muestra la pantalla para seleccionar las aulas de las que se quiere ver la ocupación.

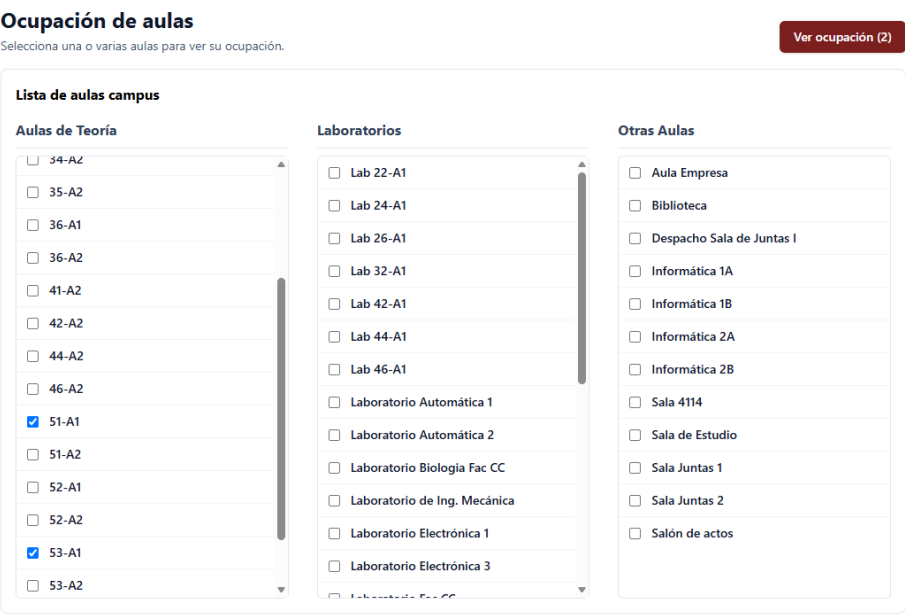


Figura E.6: Pantalla de ocupación de aulas

En la Figura E.7 se muestra la pantalla de calendario de la consulta de ocupación de aulas.

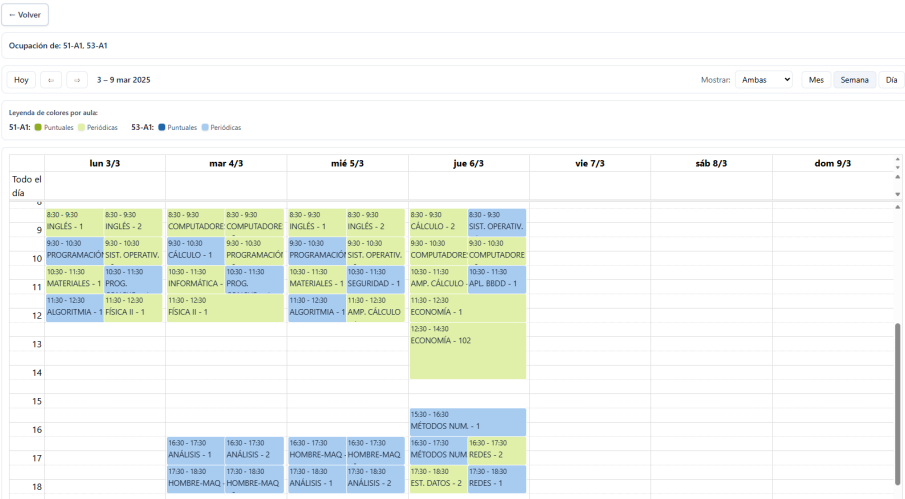


Figura E.7: Pantalla de ocupación de aulas

Calendario académico

El calendario académico permite consultar y gestionar los días que forman parte de un curso académico. Desde esta pantalla se pueden visualizar los

días lectivos, festivos, periodos de exámenes, cambios de docencia y fechas asociadas a TFG/TFM. Antes de entrar a esta pantalla, el usuario deberá seleccionar el curso académico al que quiere acceder. Además, si se quiere añadir un nuevo calendario académico, basta con ir a Cargar Nuevo Curso y rellenar el formulario para crearlo, y posteriormente el usuario tendrá que terminar de modificarlo.

Los usuarios con permisos de edición pueden modificar la clasificación de un día concreto. Para ello, deben seleccionar la fecha en el calendario y actualizar la información correspondiente. El sistema valida los datos introducidos antes de guardar los cambios. En los sábados y domingos solamente permitirá cambiar el tipo de día a festivo, y si se seleccionan más de un día no podrá elegir cambio de docencia esa opción solo estará disponible para un día.

Esta funcionalidad es importante porque el calendario académico influye directamente en la generación y gestión de reservas periódicas. Por ejemplo, los días festivos o no lectivos deben tenerse en cuenta para evitar la creación de reservas en fechas no válidas.

En la Figura E.8 se muestra la pantalla de con el listado de cursos académicos para visualizar.



Lista de Cursos	
Curso 2025-2026	
Curso 2024-2025	

Cargar Nuevo Curso

Figura E.8: Pantalla de listas con cursos académicos cargados

En la Figura E.9 se muestra la pantalla con el formulario para crear un nuevo curso académico.

← Volver

CREAR CALENDARIO ACADÉMICO

Curso académico

Ej: 2023-2024

Formato YYYY-YYYY

Inicio del Curso

dd/mm/aaaa

Fin del Curso

dd/mm/aaaa

Inicio 1º Semestre

dd/mm/aaaa

Fin 1º Semestre

dd/mm/aaaa

Inicio 2º Semestre

dd/mm/aaaa

Fin 2º Semestre

dd/mm/aaaa

☐ Añadir días festivos en el calendario

Guardar Calendario

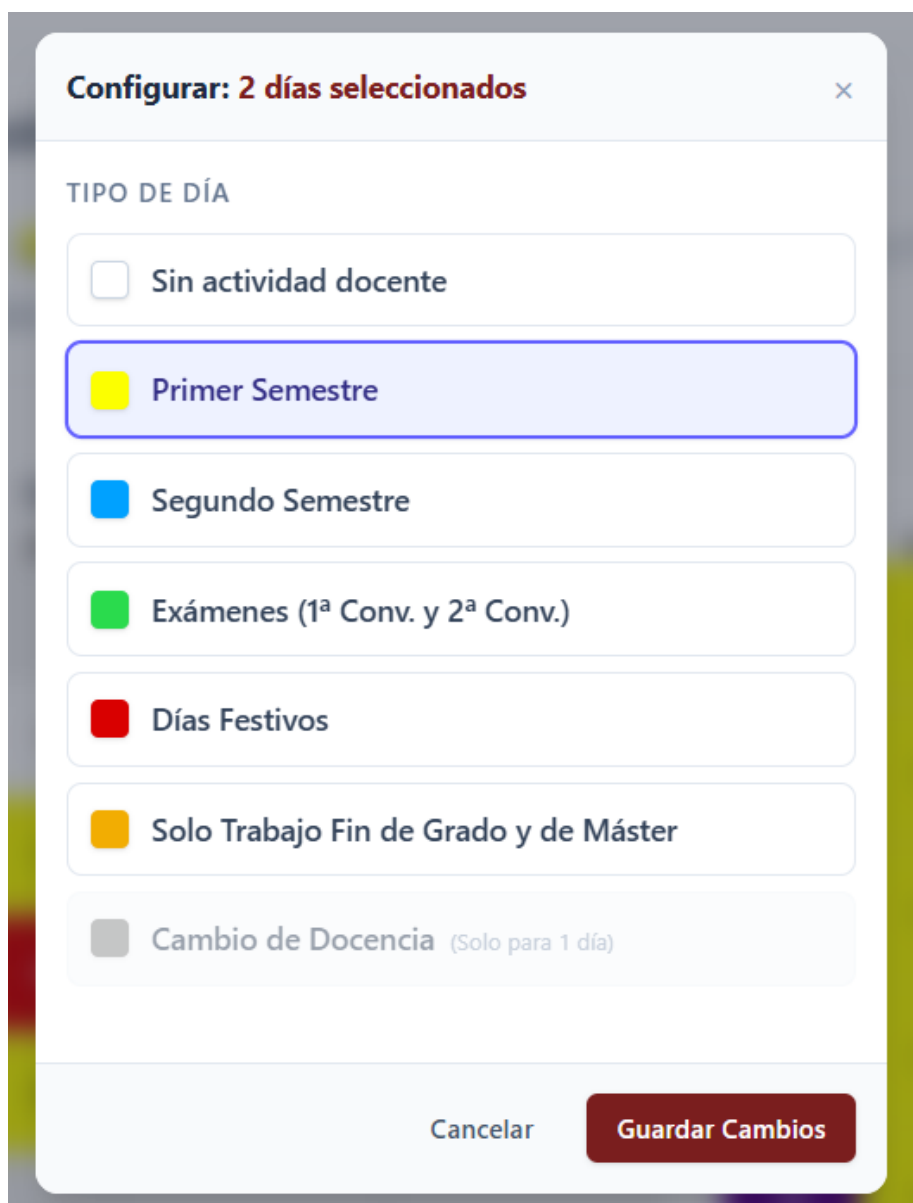
Figura E.9: Pantalla para crear un nuevo curso

En la Figura E.10 se muestra la pantalla de gestión del calendario académico.



Figura E.10: Pantalla del calendario académico

En la Figura E.11 se muestra la modal para cambiar el tipo de día.



The image shows a modal window titled "Configurar: 2 días seleccionados" with a close button (X) in the top right corner. Below the title, the section "TIPO DE DÍA" contains a list of options, each with a colored square icon and a text label. The options are: "Sin actividad docente" (white square), "Primer Semestre" (yellow square, highlighted with a blue border), "Segundo Semestre" (blue square), "Exámenes (1ª Conv. y 2ª Conv.)" (green square), "Días Festivos" (red square), "Solo Trabajo Fin de Grado y de Máster" (orange square), and "Cambio de Docencia (Solo para 1 día)" (gray square). At the bottom of the modal, there are two buttons: "Cancelar" and "Guardar Cambios".

Configurar: 2 días seleccionados

TIPO DE DÍA

- ☐ Sin actividad docente
- ☒ Primer Semestre
- ☐ Segundo Semestre
- ☐ Exámenes (1ª Conv. y 2ª Conv.)
- ☐ Días Festivos
- ☐ Solo Trabajo Fin de Grado y de Máster
- ☐ Cambio de Docencia (Solo para 1 día)

Cancelar Guardar Cambios

Figura E.11: Modal para cambiar el tipo de día

Cargar nuevo horario

La funcionalidad de carga de nuevo horario permite importar la información de horarios académicos desde una hoja de cálculo. Para ello, el usuario debe pulsar el botón de Cargar Nuevo Horario y seleccionar previamente el curso académico para el que lo quiere cargar. Si el curso académico para el que lo quiere cargar no existe en base de datos, podría crearlo pulsando en

Cargar Nuevo Curso, pero conviene que antes configure correctamente el calendario académico para que se carguen las reservas en función de festivos y cambios de docencia.

Una vez confirmada la operación, la aplicación procesa el fichero y extrae la información necesaria para generar las reservas periódicas correspondientes. Durante este proceso se validan los datos del archivo, las asignaturas, los grupos, las aulas y las franjas horarias.

Si el fichero contiene errores, datos incompletos o conflictos de ocupación, el sistema muestra un mensaje de error. Si la carga se completa correctamente, las reservas periódicas se registran en la base de datos y quedan disponibles para su consulta en los horarios correspondientes.

En la Figura E.12 se muestra la pantalla utilizada para cargar un nuevo horario desde hoja de cálculo.

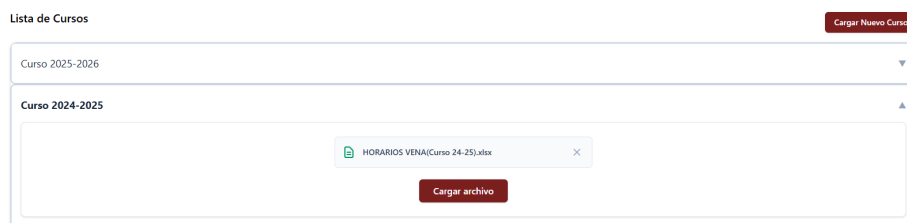


Figura E.12: Pantalla de carga de nuevo horario

Ver horarios

La pantalla de visualización de horarios permite consultar las reservas periódicas generadas para un grado, curso y semestre concreto. El usuario debe seleccionar los filtros correspondientes y la aplicación muestra el horario asociado. Previamente tendrá que seleccionar el curso académico para el que quiere ver el horario.

El horario permite visualizar las clases distribuidas por días de la semana y franjas horarias. Cada reserva muestra información relevante, como la asignatura, el grupo, el aula y el horario correspondiente.

Los usuarios con permisos de edición pueden acceder desde esta pantalla a la modificación de reservas periódicas. Por el contrario, los usuarios con permisos de consulta solo pueden visualizar la información del horario.

En la Figura E.13 se muestra la pantalla de consulta de horarios.

Horarios Académicos

Horario del curso 2024-2025

Figura E.13: Pantalla de visualización de los horarios cargados en la aplicación

En la Figura E.14 se muestra la pantalla de consulta de horarios.

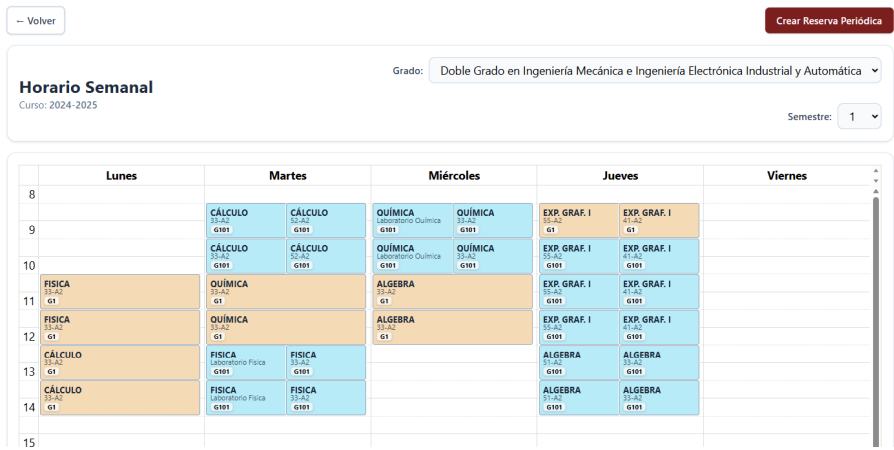


Figura E.14: Pantalla de visualización de horarios

Crear/Editar Reserva Periódica

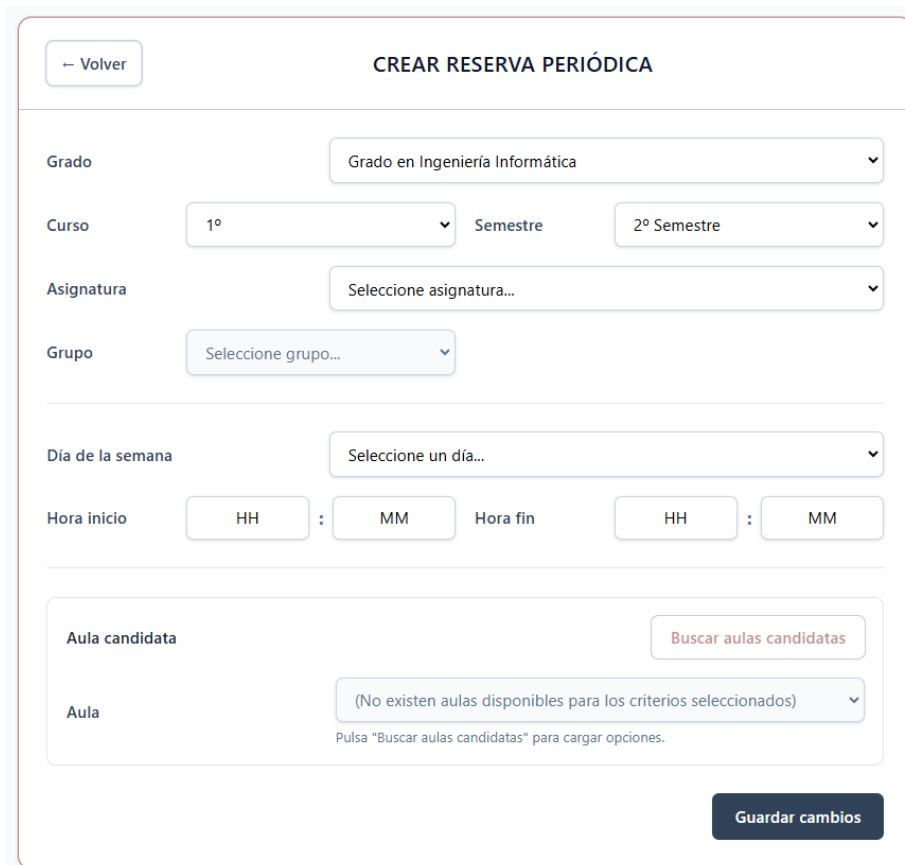
La creación o edición de una reserva periódica permite gestionar reservas asociadas a la docencia de un grupo durante un periodo determinado. Esta funcionalidad se utiliza principalmente para representar clases que se repiten semanalmente dentro de un semestre académico.

En el formulario de la reserva periódica para crearla, el usuario deberá de ir seleccionando el grado, el curso, el semestre, la asignatura y el grupo para el que quiere crearse. Estos son desplegables en cascada, es decir, no se habilitará el desplegable hasta que se rellene el anterior. También tendrá que rellenar el día de la semana y las horas a la que quiere que se imparta. Por último, pulsará Buscar Aula, y el sistema le devuelve la lista de aulas disponibles de reservas periódicas para la franja horaria seleccionada.

Para crear una reserva se podrá hacer desde la vista de horario pulsando sobre Crear Reserva Periódica se precargará el grado, curso y semestre de la vista, y para editarla bastará con tener permisos y pulsar sobre la reserva en le horario, o mover la reserva por el calendario.

En el caso de la edición de reserva, solamente se podrán modificar la franja horaria y el aula. A su vez también se pueden modificar de día y hora mediante *drag & drop*. Al modificar el día y la fecha, el sistema comprobará que no se cumpla ninguna de las restricciones, que en este caso es que el aula esté ocupada por otra reserva periódica a la misma hora y día, y que no se solape con el mismo grupo para otra asignatura del mismo grado y semestre. Si se viola alguna de estas restricciones saltará una modal de aviso, que le permitirá al usuario seguir o cancelar la acción.

En la Figura E.15 se muestra el formulario de creación o edición de una reserva periódica.



El formulario, titulado "CREAR RESERVA PERIÓDICA", incluye un botón "← Volver" en la esquina superior izquierda. Los campos de selección son:

- Grado:** Grado en Ingeniería Informática
- Curso:** 1º
- Semestre:** 2º Semestre
- Asignatura:** Seleccione asignatura...
- Grupo:** Seleccione grupo...
- Día de la semana:** Seleccione un día...
- Hora inicio:** HH : MM
- Hora fin:** HH : MM

En la sección de aula, hay un botón "Buscar aulas candidatas" y un menú desplegable "Aula" que muestra el mensaje "(No existen aulas disponibles para los criterios seleccionados)". Debajo de este menú, se indica: "Pulsa 'Buscar aulas candidatas' para cargar opciones."

El formulario termina con un botón "Guardar cambios" en la esquina inferior derecha.

Figura E.15: Formulario de creación o edición de reserva periódica

En la Figura E.16 se muestra el formulario de creación o edición de una reserva periódica.



Figura E.16: Modal de aviso con las restricciones incumplidas por el cambio

Panel de administración

El panel de administración permite acceder a las funcionalidades de gestión avanzada de la aplicación. Este apartado está destinado a usuarios con permisos de administración, por lo que no estará disponible para todos los perfiles.

Desde el panel de administración se pueden gestionar distintos elementos del sistema, como usuarios, roles, permisos, aulas, grados, asignaturas, docentes, grupos y otros datos necesarios para el funcionamiento de la aplicación.

El objetivo de este panel es centralizar las tareas de mantenimiento de la información base del sistema. Se accede desde el menú.

Entidades

Dentro del apartado de Administración, el usuario podrá ver en el menú las entidades que puede gestionar, es decir, sobre las que puede crear, editar y eliminar. Además dentro de cada entidad se podrá importar y exportar los datos, y descargar la plantilla para saber lo que se tiene que rellenar para importar de manera correcta, por si se desean introducir varias entradas para una misma entidad, a través de un Excel.

Esta funcionalidad permite mantener actualizada la información base del sistema.

En la Figura E.17 se muestra el acceso a las diferentes entidades.



Figura E.17: Acceso a entidades en el panel de administración

En la Figura E.18 se muestra la ventana de gestión de la entidad Aula.

Gestión de Aulas

Ocultar Filtros **Crear Nuevo** Descargar Plantilla Exportar Datos Importar Datos

Búsqueda global
Buscar en cualquier campo de la tabla...

Limpiar

AULA	EDIFICIO	CAPACIDAD	CAMPUS	ACCIONES
21-A1	A1	40	Rio Vena	Editar Eliminar
Lab 22-A1	A1	24	Rio Vena	Editar Eliminar
Lab 24-A1	A1	24	Rio Vena	Editar Eliminar
Lab 26-A1	A1	24	Rio Vena	Editar Eliminar
Lab 32-A1	A1	30	Rio Vena	Editar Eliminar
34-A1	A1	27	Rio Vena	Editar Eliminar
36-A1	A1	24	Rio Vena	Editar Eliminar
Lab 42-A1	A1	24	Rio Vena	Editar Eliminar

Figura E.18: Página con el listado de entradas para la entidad Aula

En la Figura E.19 se muestra el formulario para crear y editar una asignatura.

[← Volver](#)

Editar Asignatura: Cálculo

Código Asignatura)	6302	Nombre de la Asignatura	Cálculo
Abreviatura	CÁLCULO	Grado al que pertenece	166 - Grado en Ingeniería Mecá▼
Curso	1	Semestre	1
Créditos ECTS	6	Tipo de Asignatura	Obligatoria ▼
Horas Prácticas	2		

Guardar Datos

Figura E.19: Formulario para la creación o edición de una asignatura

Mantenimiento Global

Se accede también dentro del apartado de Administración del menú. Esta funcionalidad permitirá hacer mantenimiento sobre la base de datos entera. Permitiendo exportar los datos, así también se pueden tener copias de seguridad de la base de datos. Y también se plantea la opción de Descargar la Plantilla con los campos y hojas a rellenar, y de Importar los datos a la base de datos, esta acción genera un aviso ya que borrará previamente todos los datos de la base de datos.

En la Figura E.20 se muestra el formulario para crear y editar una asignatura.

Mantenimiento del Sistema

Gestiona copias de seguridad e importaciones generales de la base de datos.

Exportar Todo Descarga un backup completo en Excel con todas las tablas y datos del sistema. Descargar .XLSX	Plantilla Global Descarga el formato Excel vacío listo con todas las columnas de las tablas. Descargar Estructura	Restaurar / Importar Sube un Excel válido. Advertencia: Se sobrescribirán los datos de la base de datos. Seleccionar y Subir
---	--	---

Figura E.20: Ventana para el mantenimiento y la gestión global de todo el sistema

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo recoge una reflexión personal sobre los aspectos de sostenibilidad abordados durante el desarrollo del Trabajo de Fin de Grado. El proyecto consiste en una aplicación web orientada a la gestión de horarios, reservas de aulas, ocupación de espacios y calendario académico dentro del ámbito universitario. Aunque se trata de un trabajo principalmente técnico, su aplicación tiene relación con varios Objetivos de Desarrollo Sostenible (ODS), ya que busca mejorar la organización de recursos, reducir procesos manuales y facilitar una gestión académica más eficiente.

Siguiendo las directrices de sostenibilización curricular de la CRUE [3], se analiza a continuación cómo el proyecto se relaciona con la sostenibilidad desde una perspectiva social, ambiental, institucional y técnica.

F.2. ODS 4: Educación de calidad

El proyecto se relaciona directamente con el ODS 4, Educación de calidad, al estar orientado a mejorar la gestión de la actividad académica universitaria. La correcta planificación de horarios, aulas y reservas resulta fundamental para el desarrollo de la docencia, ya que permite organizar los espacios disponibles y evitar conflictos que puedan afectar al alumnado o al profesorado.

La aplicación permite consultar horarios, gestionar reservas periódicas asociadas a la docencia, registrar reservas puntuales y visualizar la ocupación de las aulas. Todo ello contribuye a que la información académica sea más accesible, ordenada y actualizada. De esta forma, se facilita el trabajo de los gestores y se reduce la posibilidad de errores derivados de una gestión manual o dispersa.

Además, la centralización de la información permite que los cambios en el calendario académico, como festivos o cambios de docencia, puedan reflejarse en el sistema de manera más clara. Esto favorece una mejor coordinación entre los distintos agentes implicados en la organización docente.

F.3. ODS 9: Industria, innovación e infraestructura

El proyecto también se alinea con el ODS 9, Industria, innovación e infraestructura, al proponer una solución tecnológica para modernizar un proceso administrativo universitario. La aplicación sustituye parcialmente procedimientos basados en hojas de cálculo y comprobaciones manuales por un sistema web estructurado, con una base de datos centralizada y módulos diferenciados para horarios, reservas, aulas, calendario y usuarios.

Desde el punto de vista técnico, se ha buscado construir una solución mantenible y ampliable. La separación entre *backend* y *frontend*, el uso de una arquitectura modular y la documentación del sistema facilitan que el proyecto pueda evolucionar en el futuro. Esto es importante porque una solución sostenible no solo debe resolver una necesidad inmediata, sino también permitir futuras mejoras sin rehacer todo el sistema.

Asimismo, el proyecto incorpora herramientas de control de versiones y análisis de calidad del código, lo que contribuye a mejorar la mantenibilidad y fiabilidad de la aplicación.

F.4. ODS 12: Producción y consumo responsables

El ODS 12, Producción y consumo responsables, se aborda de forma indirecta mediante la mejora en el uso de los recursos universitarios. Las aulas, laboratorios y espacios docentes son recursos limitados que deben utilizarse de forma eficiente. Una mala planificación puede provocar solapamientos,

infrautilización de espacios o necesidad de realizar correcciones manuales posteriores.

La aplicación ayuda a conocer la ocupación real de las aulas y a validar restricciones antes de crear o modificar reservas. Esto permite evitar conflictos y favorece una utilización más responsable de los espacios disponibles. Además, al digitalizar la gestión de horarios y reservas, se reduce la dependencia de documentos impresos o ficheros duplicados, promoviendo una gestión más ordenada de la información.

F.5. ODS 13: Acción por el clima

Aunque el impacto ambiental del proyecto es indirecto, también puede relacionarse con el ODS 13, Acción por el clima. La digitalización de procesos contribuye a reducir el uso de papel y otros recursos asociados a la gestión tradicional de documentos. Además, una planificación más eficiente de aulas y horarios puede servir como base para futuras decisiones relacionadas con el uso energético de los edificios universitarios.

Si en un futuro la aplicación se ampliara con informes de ocupación o análisis de uso de espacios, podría ayudar a identificar aulas infrautilizadas, franjas horarias con baja ocupación o patrones de uso que permitieran optimizar mejor los recursos físicos de la universidad.

F.6. Conclusión

La realización de este Trabajo de Fin de Grado me ha permitido comprender que la sostenibilidad también está presente en el desarrollo de software. Diseñar una aplicación mantenible, segura y útil para mejorar procesos reales forma parte de una visión responsable de la ingeniería informática.

En conclusión, el proyecto contribuye a la sostenibilidad al mejorar la gestión académica, favorecer el uso eficiente de aulas, reducir procesos manuales y ofrecer una base tecnológica preparada para futuras ampliaciones. Aunque su impacto ambiental no es directo, sí aporta valor desde una perspectiva social, institucional y técnica dentro del contexto universitario.

Bibliografía

- [1] Adam Johnson. django-cors-headers. <https://pypi.org/project/django-cors-headers/>, 2026. Accedido: junio de 2026.
- [2] Apache Software Foundation. Apache license, version 2.0. <https://www.apache.org/licenses/LICENSE-2.0>, 2026. Accedido: junio de 2026.
- [3] CRUE. Directrices para la introducción de la sostenibilidad en el currículum. https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf, 2012. Accedido: 2026.
- [4] Diagramas UML. Diagrama entidad-relación (e/r). <https://diagramasuml.com/%E2%96%B7-diagrama-entidad-relacion-e-r-teoria-y-ejemplos/>, 2024-01-02. Accedido: junio de 2026.
- [5] Django Software Foundation. Faq: General. how is django licensed? <https://docs.djangoproject.com/en/6.0/faq/general/>, 2026. Accedido: junio de 2026.
- [6] Encode OSS Ltd. Django rest framework. <https://pypi.org/project/djangorestframework/>, 2026. Accedido: junio de 2026.
- [7] Free Software Foundation. Gnu general public license, version 2. <https://www.gnu.org/licenses/old-licenses/gpl-2.0.en.html>, 2026. Accedido: junio de 2026.
- [8] GitHub. Github pricing. <https://github.com/pricing>, 2026. Accedido: junio de 2026.
- [9] Kenneth Reitz. Requests: Http for humans. <https://requests.readthedocs.io/>, 2026. Accedido: junio de 2026.

- [10] Meta Platforms, Inc. React license. <https://github.com/facebook/react/blob/main/LICENSE>, 2026. Accedido: junio de 2026.
- [11] Microsoft. Microsoft software license terms for windows 11. https://www.microsoft.com/content/dam/microsoft/usetm/documents/windows/11/oem-%28pre-installed%29/UseTerms_OEM_Windows_11_English.pdf, 2026. Accedido: junio de 2026.
- [12] Microsoft. Windows 11 home. <https://www.microsoft.com/es-es/d/windows-11-home/dg7gmgf0krt0>, 2026. Accedido: junio de 2026.
- [13] Open Source Initiative. The mit license. <https://opensource.org/license/mit>, 2026. Accedido: junio de 2026.
- [14] openpyxl. openpyxl. <https://pypi.org/project/openpyxl/>, 2026. Accedido: junio de 2026.
- [15] Oracle. Mysql community edition. <https://www.mysql.com/products/community/>, 2026. Accedido: junio de 2026.
- [16] Patricio Araneda. El modelo relacional. <https://bookdown.org/paranadagarcia/database/el-modelo-relacional.html>, 2022-10-18. Accedido: junio de 2026.
- [17] Python Software Foundation. History and license. <https://docs.python.org/3/license.html>, 2026. Accedido: junio de 2026.
- [18] Railway. Railway pricing. <https://railway.com/pricing>, 2026. Accedido: junio de 2026.
- [19] Super Contable. Tarifa de primas para la cotización a la seguridad social por accidentes de trabajo y enfermedades profesionales. https://www.supercontable.com/informacion/laboral/Cuadro_Tarifas_de_Accidente_de_Trabajo_y_.html, 2026. [Internet; descargado 07-junio-2026].
- [20] Tailwind Labs. Tailwind css license. <https://github.com/tailwindlabs/tailwindcss/blob/main/LICENSE>, 2026. Accedido: junio de 2026.
- [21] VoidZero Inc. and Vite contributors. Vite license. <https://github.com/vitejs/vite/blob/main/LICENSE>, 2026. Accedido: junio de 2026.